



北京航空航天大学
BEIHANG UNIVERSITY



数据结构与程序设计（信息类）

Data Structure & Programming

北京航空航天大学 数据结构课程组

软件学院 谭火彬

2023年春



提纲：树和二叉树

- ◆4.1 树的基本概念
- ◆4.2 树的存储结构
- ◆4.3 二叉树
- ◆4.4 二叉树的存储结构
- ◆4.5 二叉树的遍历
- ◆*4.6 线索二叉树
- ◆4.7 二叉查找树
- ◆*4.8 堆和表达式树
- ◆4.9 哈夫曼树

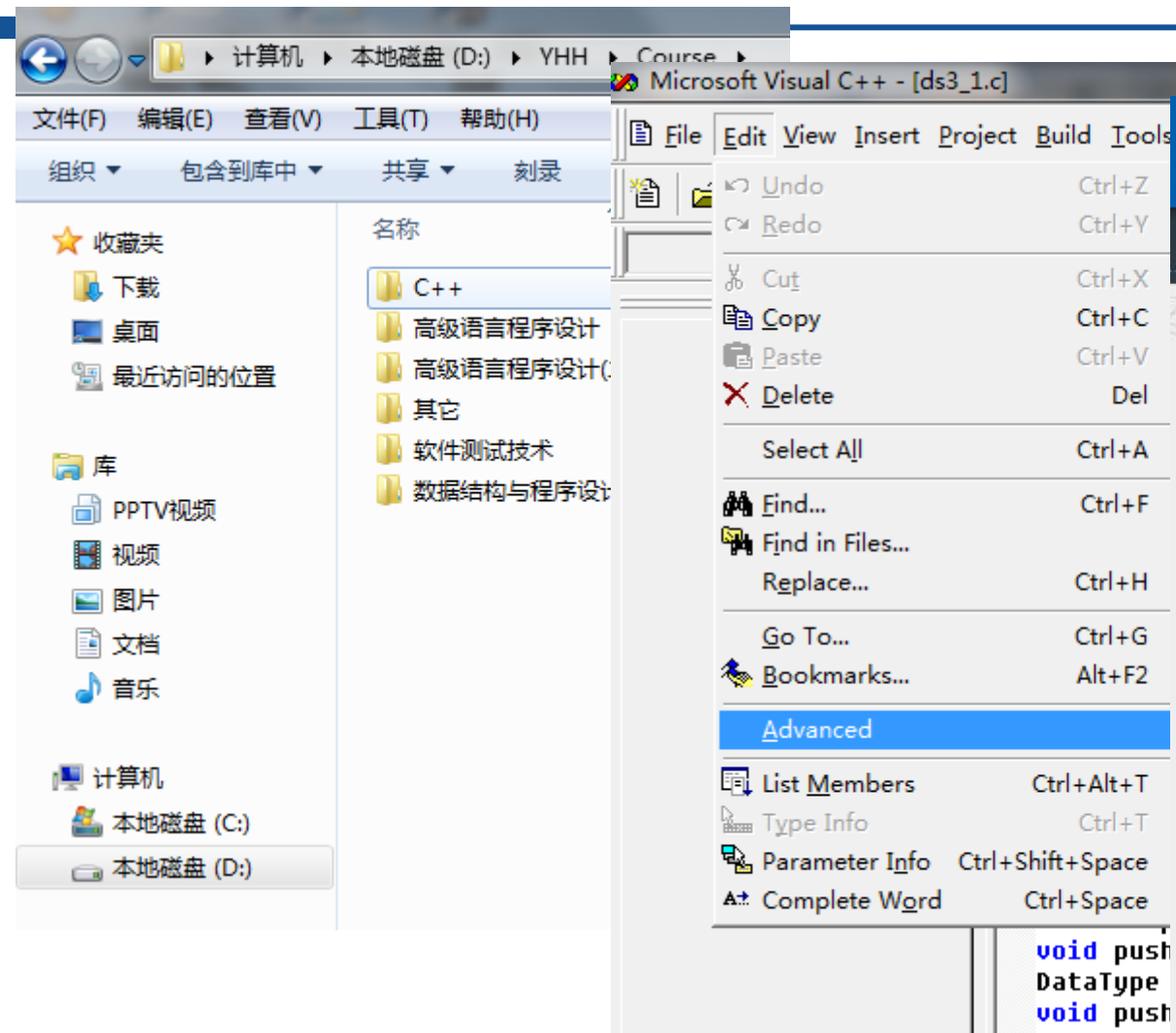


提纲：树和二叉树

- ◆ 4.1 树的基本概念
- ◆ 4.2 树的存储结构
- ◆ 4.3 二叉树
- ◆ 4.4 二叉树的存储结构
- ◆ 4.5 二叉树的遍历
- ◆ *4.6 线索二叉树
- ◆ 4.7 二叉查找树
- ◆ *4.8 堆和表达式树
- ◆ 4.9 哈夫曼树



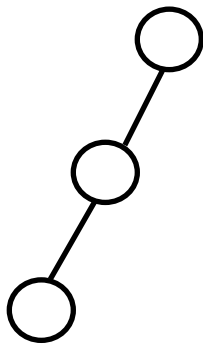
树是计算机中常见的数据组织方式





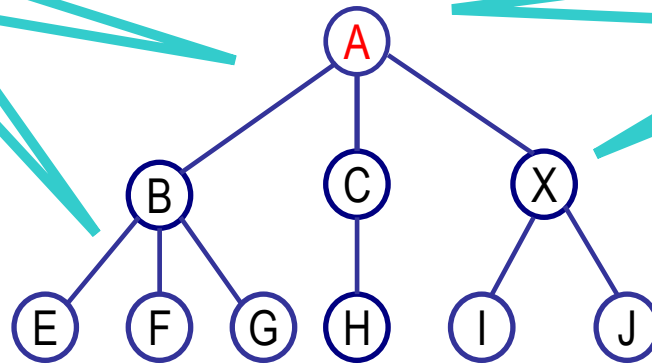
线性结构与树结构

$A = (a_1, a_2, a_3, \dots, a_n)$



关系

数据元素



线性结构	树结构
第一个数据元素（无前驱）	根节点（无前驱）
最后一个数据元素（无后继）	叶子节点（无后继）
其他数据元素（一个前驱、一个后继）	树中其他节点（一个前驱、多个后继）



1. 树

- ◆ 树是由 $n \geq 0$ 个结点组成的有穷集合（用符号 D 表示）以及结点之间关系组成的集合构成的结构，记为 T
 - ✓ 当 $n=0$ 时，称 T 为空树
 - ✓ 在任何一棵非空的树中，有一个特殊的结点 $t \in D$ ，称之为该树的根结点；其余结点 $D - \{t\}$ 被分割成 $m > 0$ 个不相交的子集 $D_1, D_2, \dots, D_m$ ，其中，每一个子集 D_i 分别构成一棵树，称之为 t 的子树

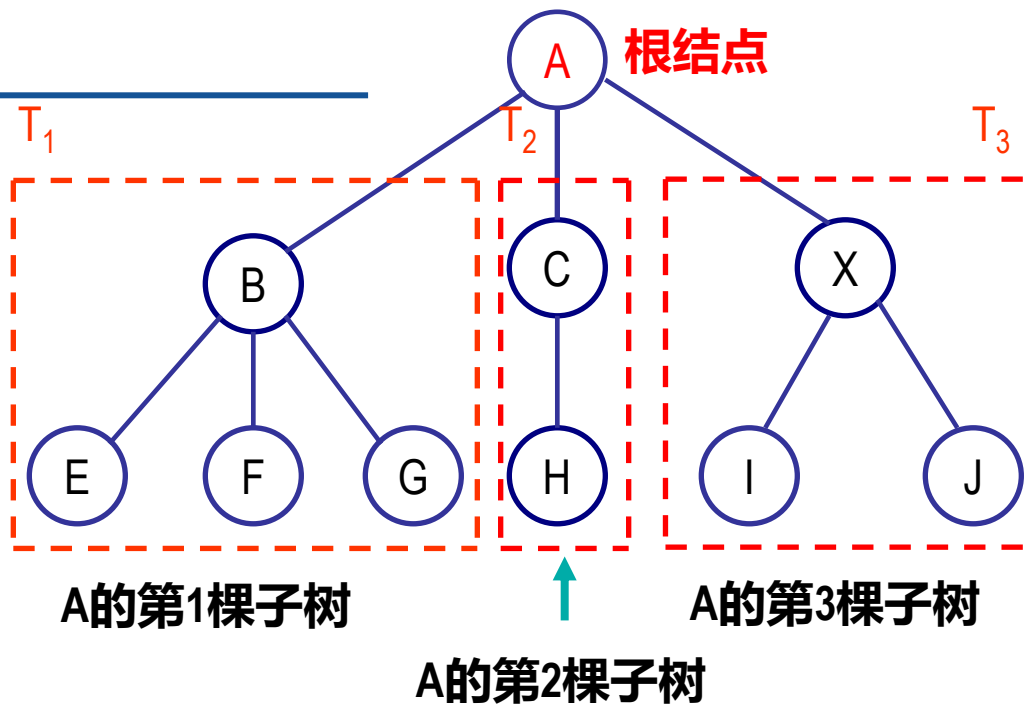
递归定义



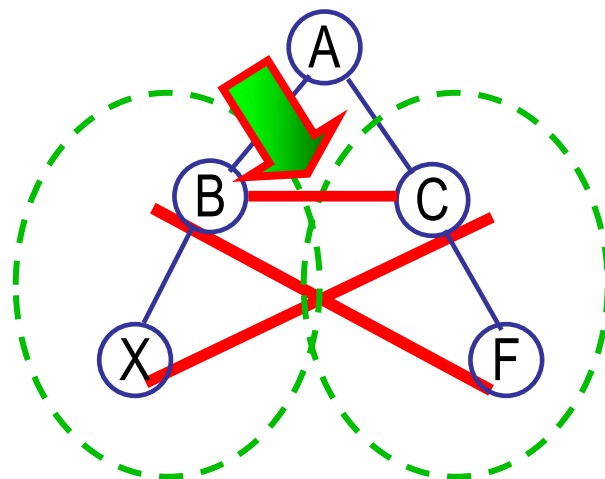
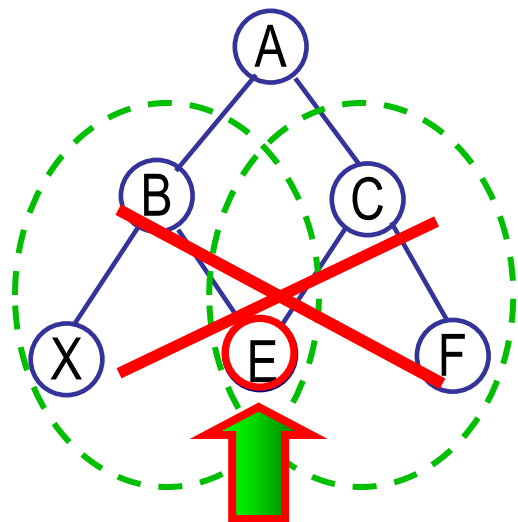
树结构

结点集合

$D = \{ A, B, C, E, F, G, H, I, J, X \}$



不相交的子集

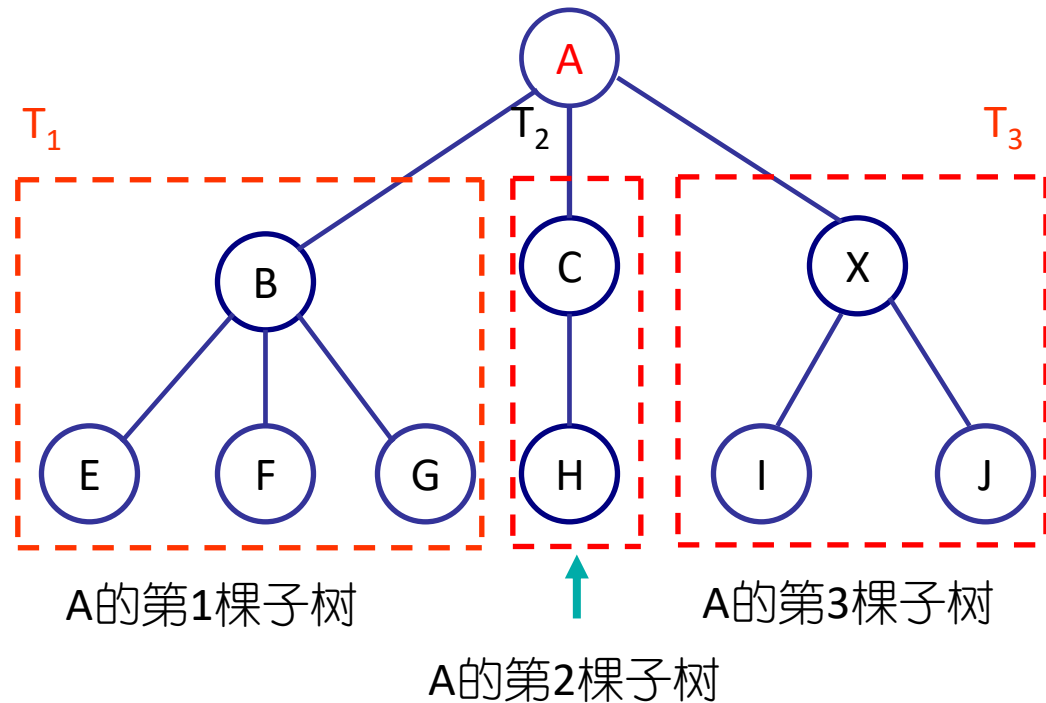




1.2 树的特点

◆树（逻辑上）的特点

- ✓1. 非空树中，**有且仅有一个**结点没有前驱结点,该结点为树的**根结点**
- ✓2. 除了根结点外,每个结点**有且仅有一个直接前驱**结点
- ✓3. 包括根结点在内，每个结点**可以有多个后继**结点





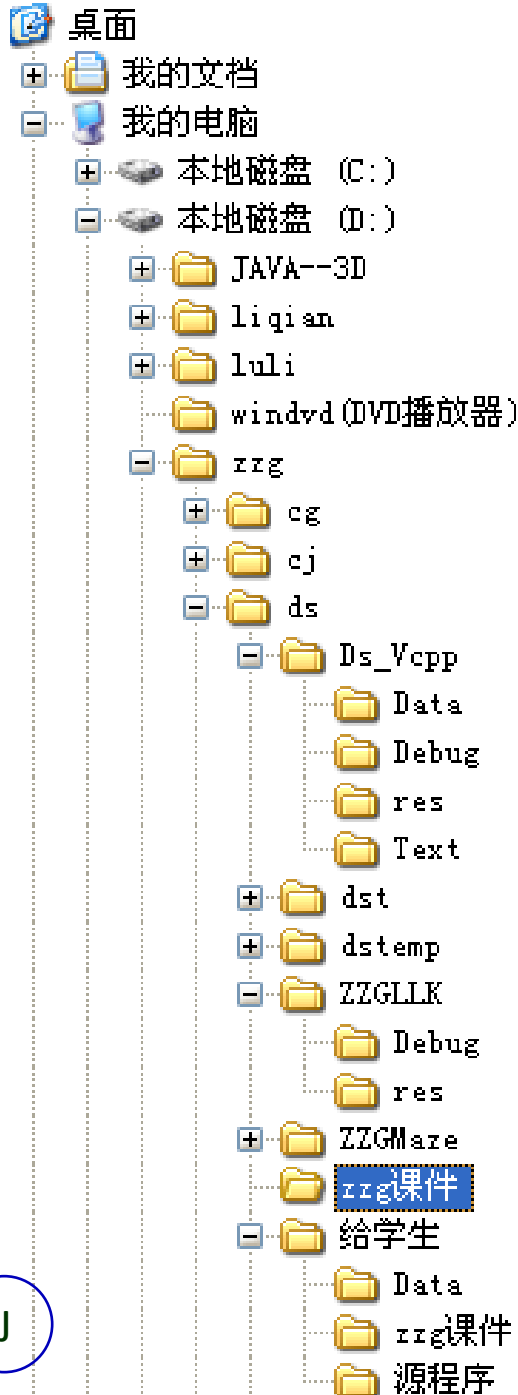
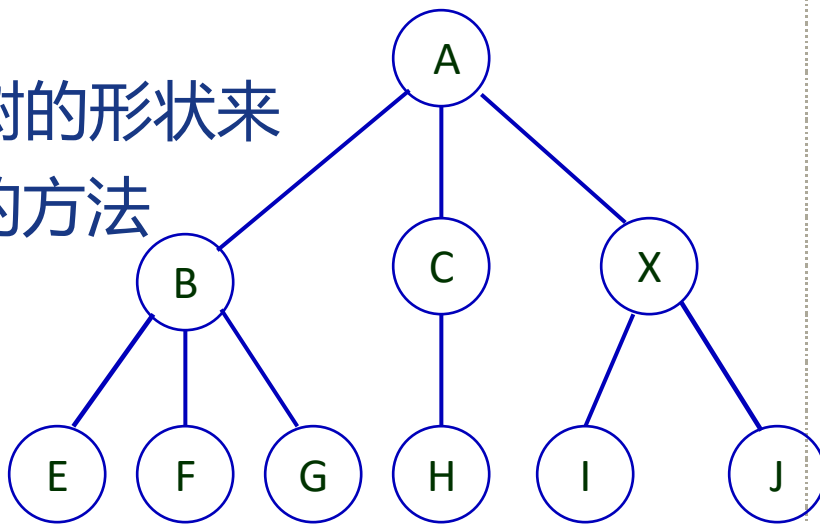
1.3 树的逻辑表示方法

- ◆ 1*. 文氏图表示法
- ◆ 2*. 凹入表示法
- ◆ 3*. 嵌套括号法(广义表表示法)

$A(\underbrace{B(E, F, G)}_{\text{第1棵子树}}, \underbrace{C(H)}_{\text{第2棵子树}}, \underbrace{X(I, J)}_{\text{第3棵子树}})$

◆ 4. 树形表示法

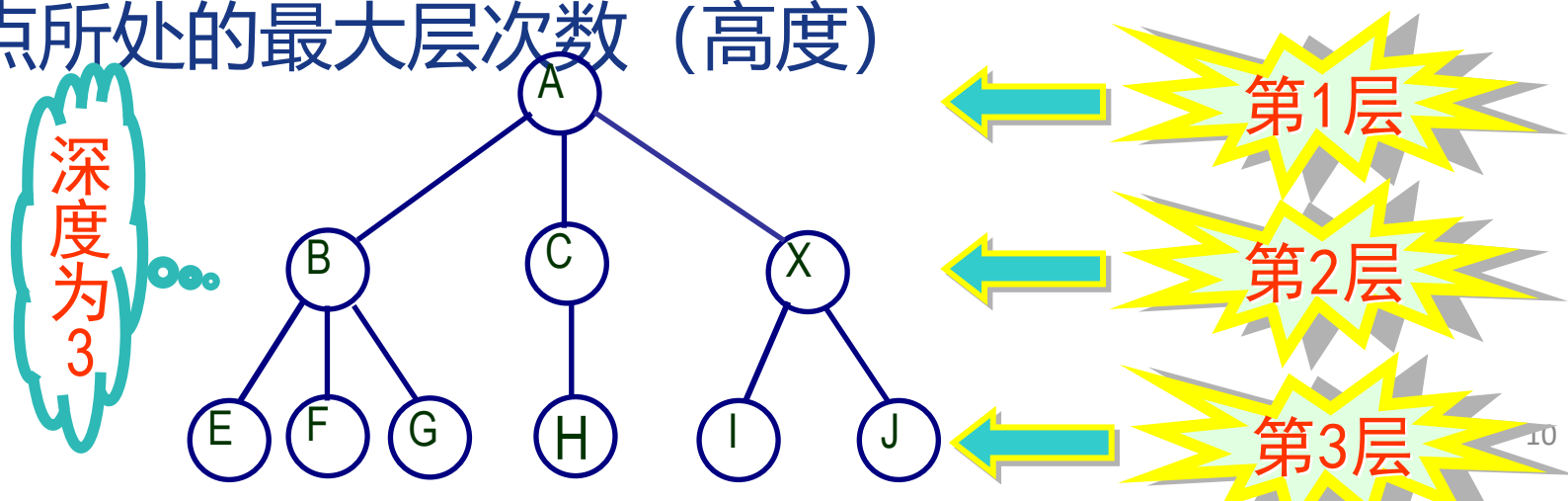
✓ 借助于自然界中一棵倒置的树的形状来表示数据元素之间层次关系的方法





1.4 基本名词术语

- ◆ 1. **结点的度**: 该结点拥有的子树数目
- ◆ 2. **树的度**: 树中结点度的最大值
- ◆ 3. **叶结点**: 度为0的结点 (终端结点)
- ◆ 4. **分支结点**: 度非0的结点 (非终端结点)
- ◆ 5. **树的层次**: 根结点为第1层, 若某结点在第 i 层, 则其孩子结点 (若存在) 为第 $i+1$ 层
- ◆ 6. **树的深度**: 树中结点所处的最大层次数 (高度)

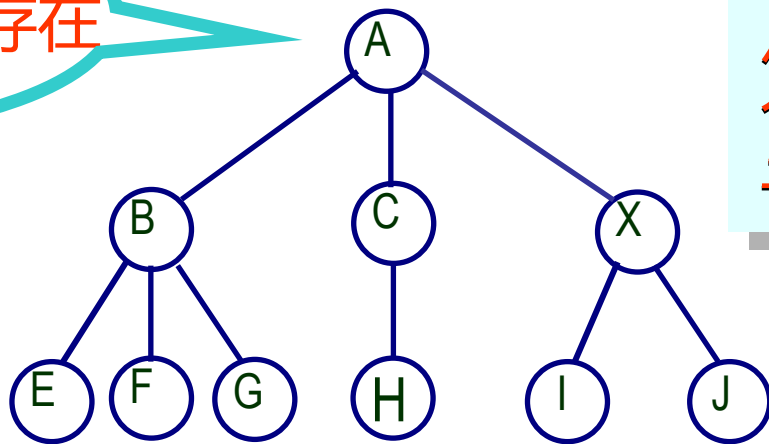




基本名词术语 (续)

- ◆ 7. **路径**: 对于树中任意结点 d_j , 若在树中存在一个结点序列 $d_1, d_2, \dots, d_i, \dots, d_j$, 使得 d_i 是 d_{i+1} 的双亲($1 \leq i < j$), 则称该结点序列是从 d_1 到 d_j 的一条路径, 路径的长度为 $j-1$
- ◆ 8. **祖先与子孙**: 若树中结点 d 到 d_s 存在一条路径, 则称 d 是 d_s 的祖先, d_s 是 d 的子孙

从根结点到树中
其余结点均分别存在
一条唯一路径

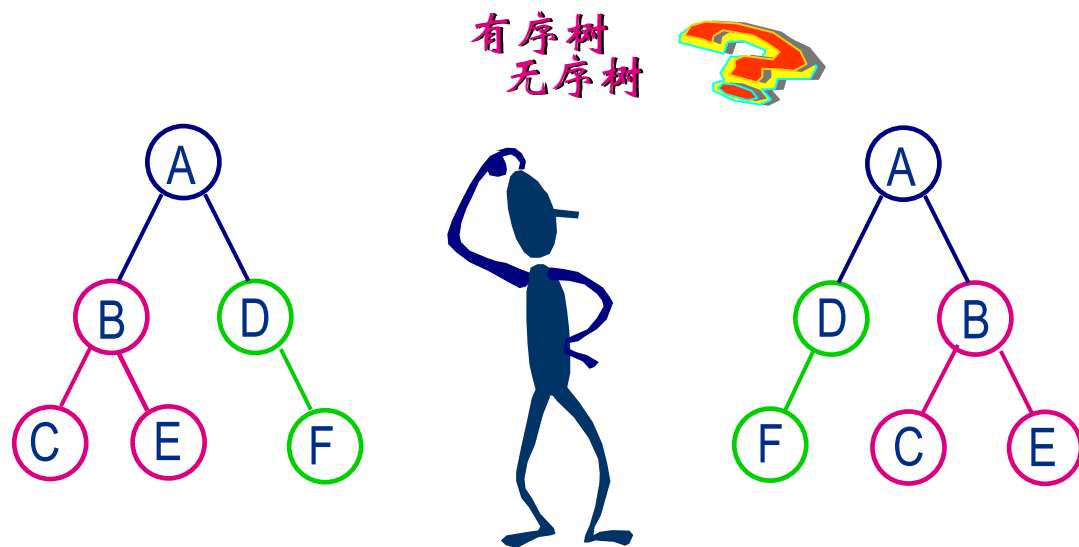


一个结点的祖先是根结点到该结点路径上所经过的所有结点; 而一个结点的子孙则是以该结点为根的子树上的所有其他结点



基本名词术语 (续)

- ◆ 9. **树林** (森林) : $m \geq 0$ 棵不相交的树组成的树的集合
- ◆ 10. **树的有序性**: 若树中结点的子树的相对位置不能随意改变, 则称该树为**有序树**, 否则称该树为**无序树**



结点间关系: 结点的子树的根称为该结点的**孩子** (child), 相应地, 该结点称为孩子结点的**父结点** (或双亲, parent)。同一个双亲的孩子之间互称**兄弟**



提纲：树和二叉树

- ◆ 4.1 树的基本概念
- ◆ 4.2 树的存储结构
- ◆ 4.3 二叉树
- ◆ 4.4 二叉树的存储结构
- ◆ 4.5 二叉树的遍历
- ◆ *4.6 线索二叉树
- ◆ 4.7 二叉查找树
- ◆ *4.8 堆和表达式树
- ◆ 4.9 哈夫曼树



2. 树的存储结构

◆两种存储结构

- ✓顺序存储结构

- ✓链式存储结构（常用）

◆无论采用何种存储结构，需要存储的信息包括

- ✓结点本身的数据信息

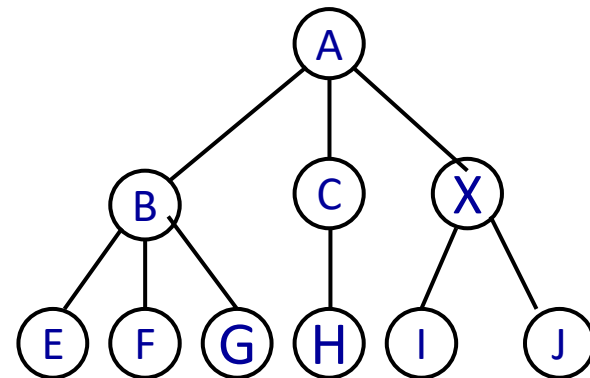
- ✓结点之间存在的关系（分支）



2.1 多重链表结构

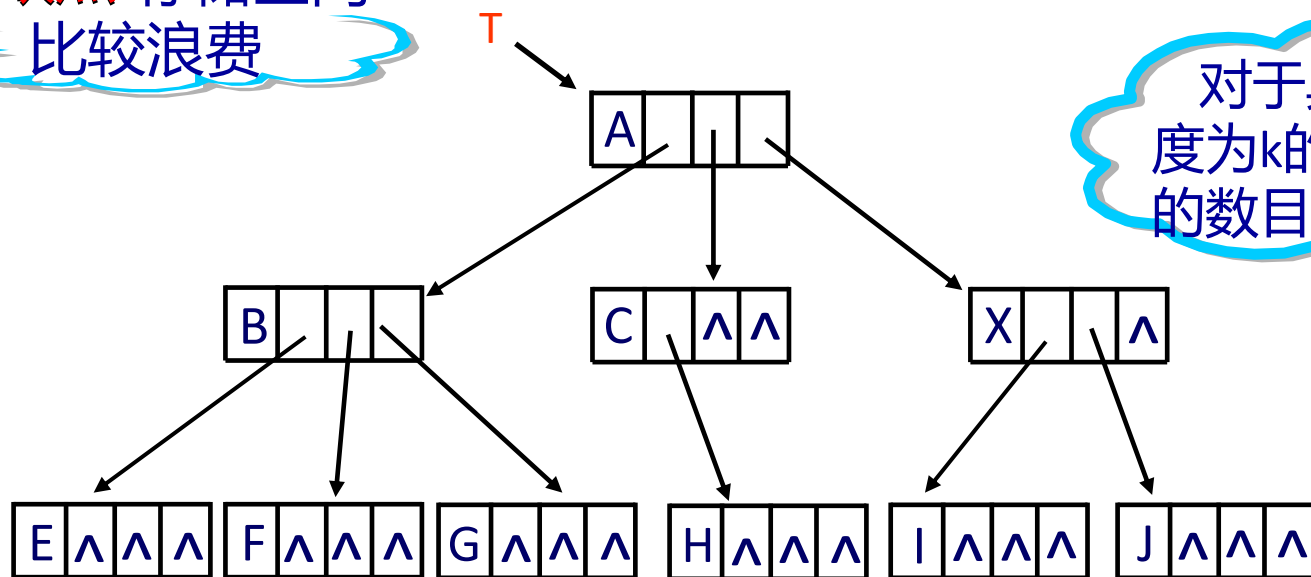
◆ 1. 定长结点的多重链表结构

链结点的构造



k 为树的度

缺点:存储空间
比较浪费



对于具有 n 个结点且
度为 k 的树,空指针域
的数目是多少?

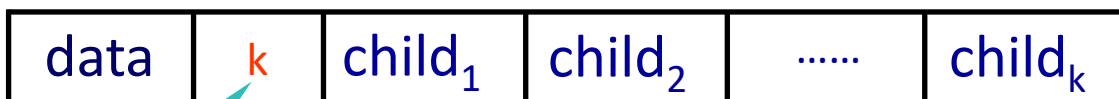
$$n(k-1) + 1$$



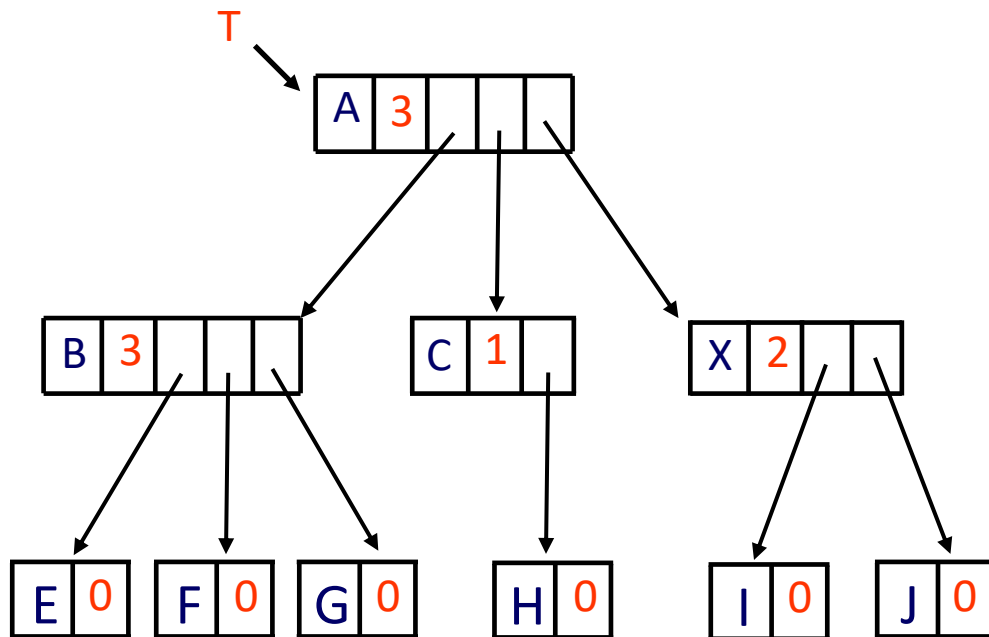
2.1 多重链表结构 (续)

◆ 2. 不定长结点的多重链表结构

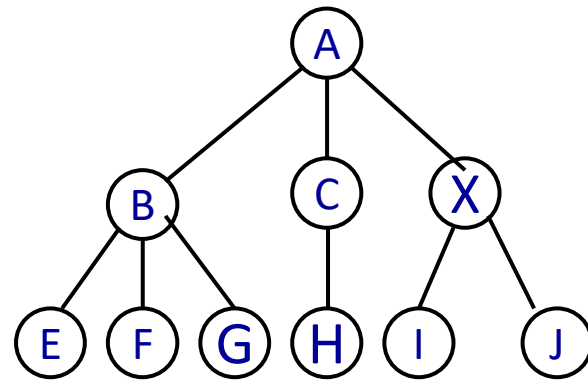
链结点的构造



结点的度



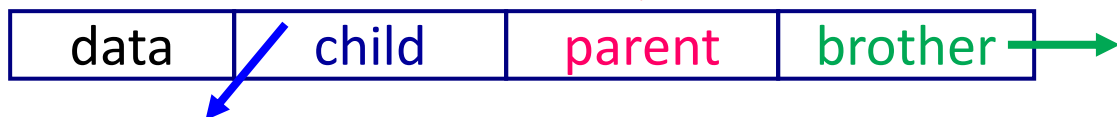
缺点:对树的操作不方便





2.2 三重链表结构

链结点的构造

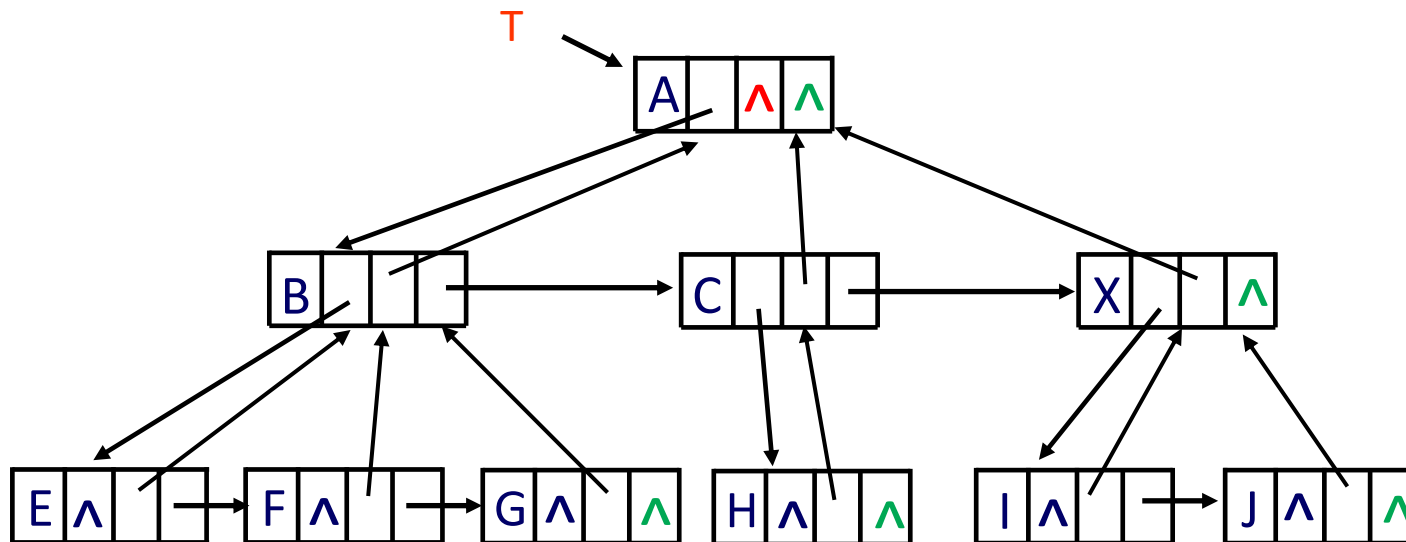
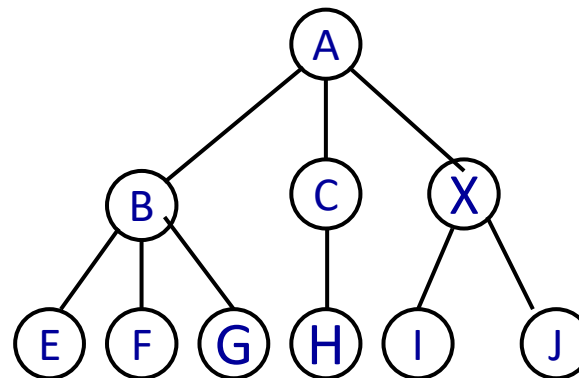


其中,data 为数据域;

child 为指针域,指向该结点的**第1个孩子结点**;

parent 为指针域,指向该结点的**双亲结点**;

brother 为指针域,指向右边第一个**兄弟结点**。





提纲：树和二叉树

- ◆4.1 树的基本概念
- ◆4.2 树的存储结构
- ◆4.3 二叉树
- ◆4.4 二叉树的存储结构
- ◆4.5 二叉树的遍历
- ◆*4.6 线索二叉树
- ◆4.7 二叉查找树
- ◆*4.8 堆和表达式树
- ◆4.9 哈夫曼树



3. 二叉树

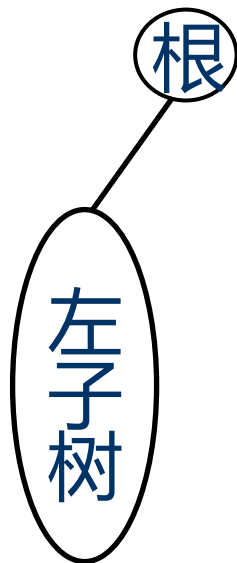
- ◆ **二叉树**是 $n \geq 0$ 个结点的有穷集合D与D上关系的集合R构成的结构，记为T
 - ✓ 当 $n=0$ 时，称T为空二叉树
 - ✓ 否则，它为包含了一个根结点以及**两棵**不相交的、分别称之为**左子树与右子树**的二叉树
- ◆ 二叉树是有序树，**区分左右子树**



二叉树的5种基本形态

(空)

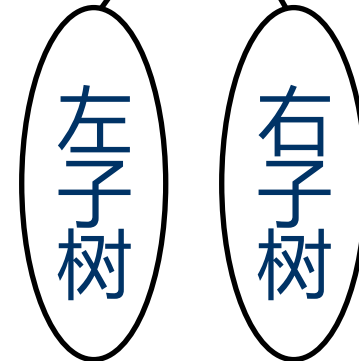
根



根



根

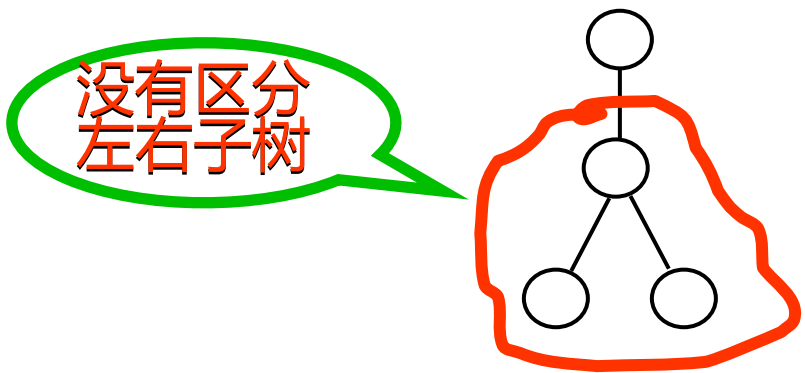




思考：下面的说法正确与否

✗ 1. 度为2的 树 是二叉树。

✗ 2. 度为2的有序树是二叉树。

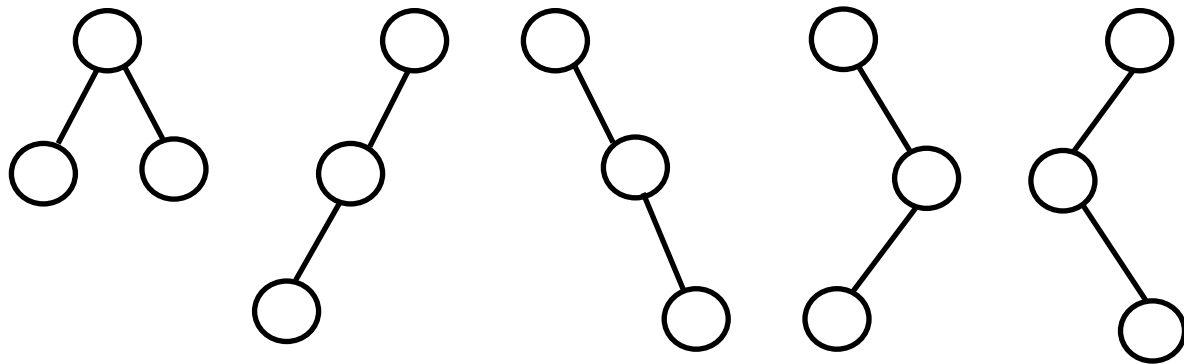


结论：子树有严格的左、右之分且度 ≤ 2 的树是二叉树。



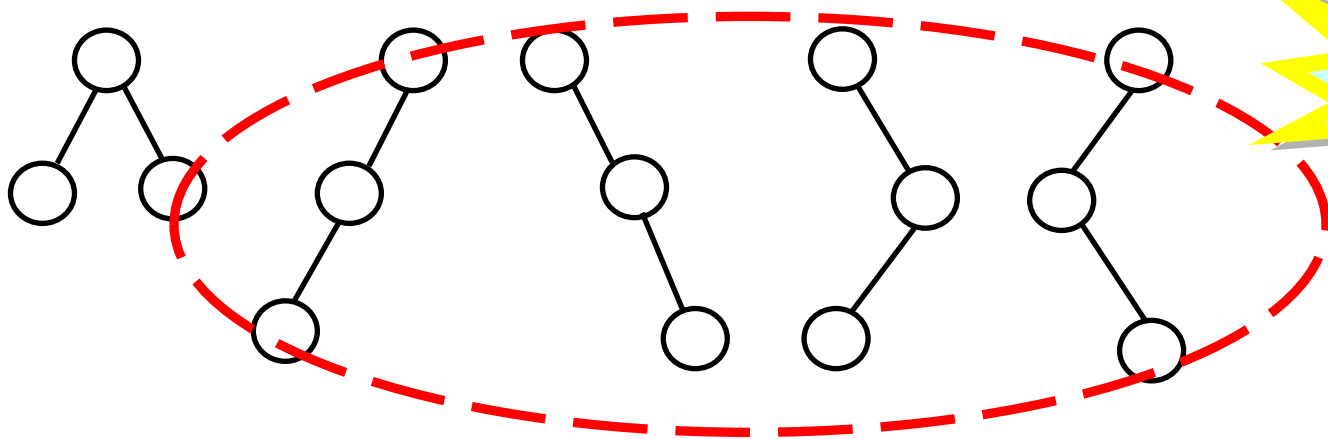
思考：下面的说法正确与否（续）

3. 具有三个结点的二叉树可以有多少种形态？



5种

✗ 4. 具有三个结点的树可以有 5 种形态



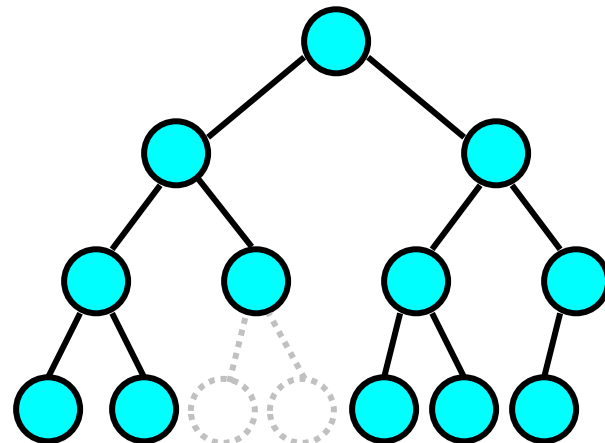
2种



3.2 两种特殊形态的二叉树

1. 满二叉树

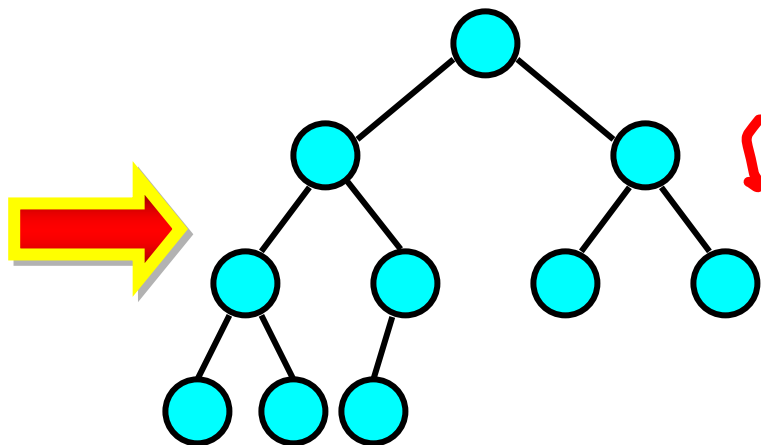
若一棵二叉树中的结点, 或者为叶结点, 或者具有两棵非空子树, 并且叶结点都集中在二叉树的最下面一层. 这样的二叉树为满二叉树



不是完全二叉树

2. 完全二叉树

若一棵二叉树中只有最下面两层的结点的度可以小于2, 并且最下面一层的结点(叶结点)都依次排列在该层从左至右的位置上. 这样的二叉树为完全二叉树



是完全二叉树

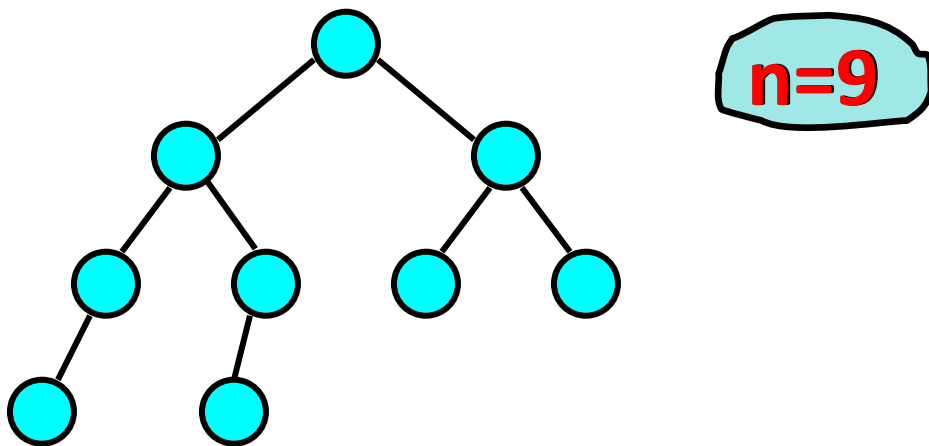


3.3 二叉树的性质

1. 具有 n 个结点的非空二叉树共有 **$n-1$** 个分支（边）。

证明:

除了根结点以外，每个结点有且仅有一个双亲结点，即每个结点与其双亲结点之间仅有一个分支存在，因此，具有 n 个结点的非空二叉树的分支总数为 $n-1$





二叉树的性质（续）

2. 非空二叉树的第 i 层最多有 2^{i-1} 个结点($i \geq 1$)。

证明(采用归纳法)

(1) 当 $i=1$ 时,结论显然正确。非空二叉树的第1层有且仅有一个结点,即树的根结点.

(2) 假设对于第 j 层($1 \leq j \leq i-1$)结论也正确,即第 j 层最多有 2^{j-1} 个结点.

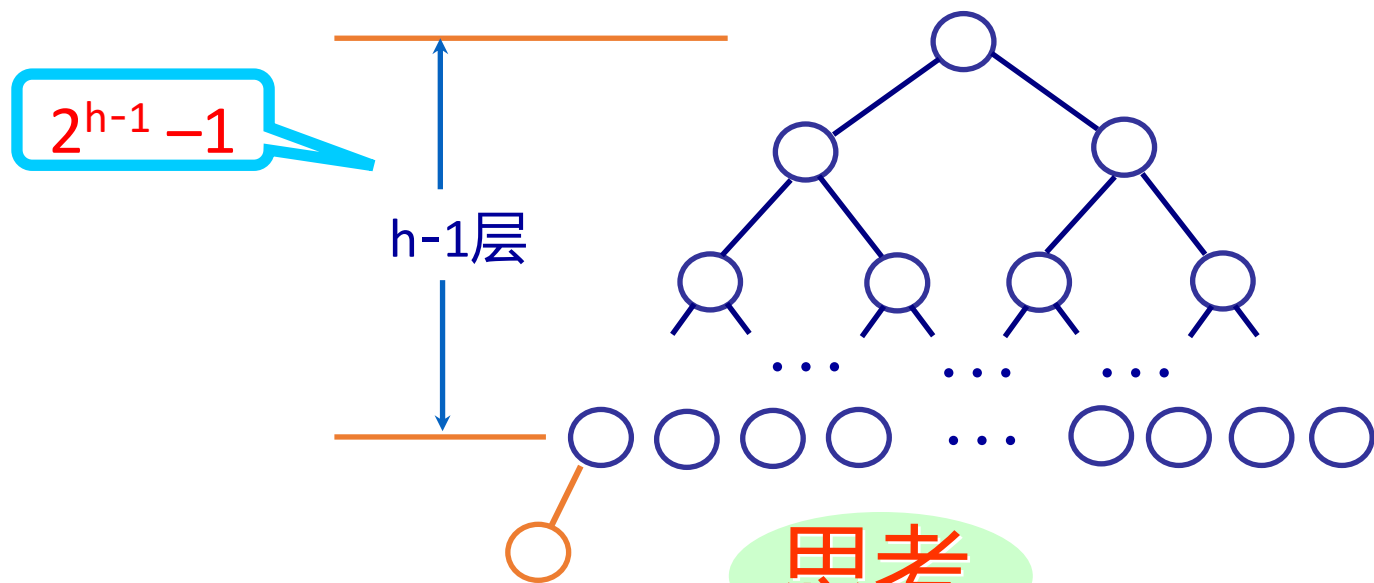
(3) 由定义可知, 二叉树中每个结点最多只能有两个孩子结点。若第 $i-1$ 层的每个结点都有两棵非空子树,则第 i 层的结点数目达到最大.而第 $i-1$ 层最多有 2^{i-2} 个结点已由假设证明,于是,应有

$$2 \times 2^{i-2} = 2^{i-1}$$



二叉树的性质（续）

3. 深度为 h 的非空二叉树最多有 $2^h - 1$ 个结点。



思考

深度为 h 的完全二叉树至少有多少个结点



结论

2^{h-1}



二叉树的性质（续）

4. 若非空二叉树有 n_0 个叶结点,有 n_2 个度为2的结点,则 $n_0=n_2+1$

证明:

设该二叉树有 n_1 个度为1的结点,结点总数为 n ,有
 $n=n_0+n_1+n_2$ ----- (1)

设二叉树的分支数目为 B , 根据性质1, 有 $B=n-1$ ----- (2)

这些分支来自度数为1的结点与度为2结点,即
 $B=n_1+2n_2$ ----- (3)

联列关系(1),(2)与(3),可得到 $n_0=n_2+1$

推论

$n_0=n_2+2n_3+3n_4+ \dots +(m-1)n_m+1$ (P207, 习题7-4 2)



二叉树的性质（续）

5. 具有 n 个结点的非空完全二叉树的深度为 $h = \lfloor \log_2 n \rfloor + 1$.

证明:

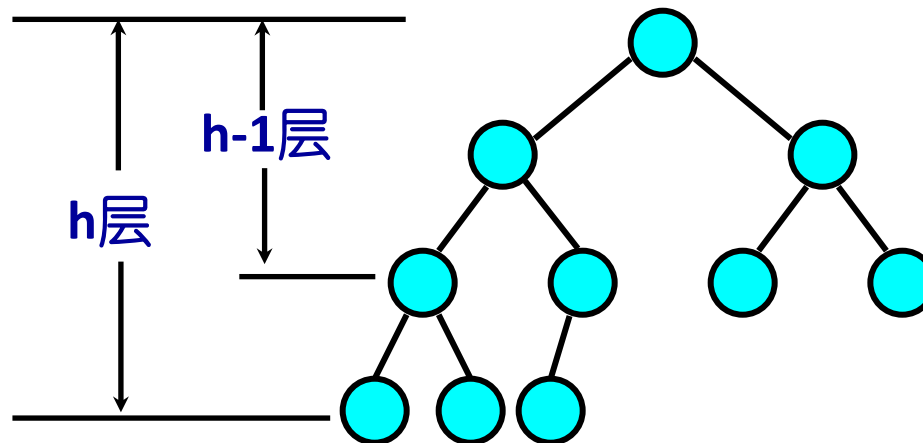
设深度为 h ,

则: $2^{h-1} - 1 < n \leq 2^h - 1$

即: $2^{h-1} \leq n < 2^h$

即: $h-1 \leq \log_2 n < h$

因此: $h = \lfloor \log_2 n \rfloor + 1$





二叉树的性质（续）

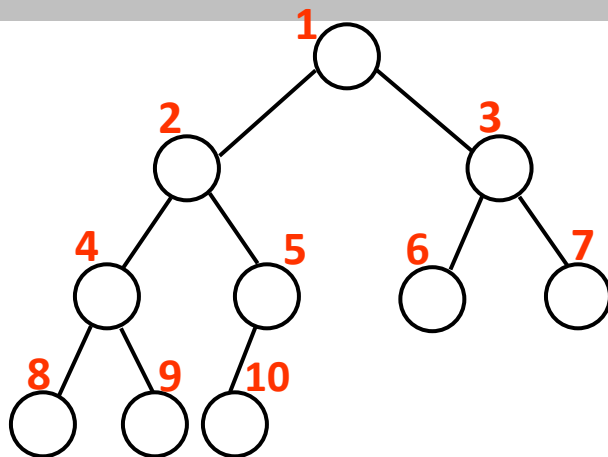
6. 若对具有 n 个结点的完全二叉树按照层次从上到下,每层从左到右的顺序进行编号, 则编号为 i 的结点具有以下性质:

(1) 当 $i=1$,则编号为 i 的结点为二叉树的根结点; 若 $i>1$,则编号为 i 的结点的双亲的编号为 $\lfloor i/2 \rfloor$;

(2) 若 $2i>n$,则编号为 i 的结点无左子树; 若 $2i\leq n$,则编号为 i 的结点的左孩子的编号为 $2i$;

(3) 若 $2i+1>n$,则编号为 i 的结点无右子树; 若 $2i+1\leq n$,则编号为 i 的结点的右孩子的编号为 $2i+1$ 。

$n=10$





3.4 二叉树的基本操作

1. **INITIAL(T)** 初始(创建)一棵二叉树。
2. **ROOT(T)**或**ROOT(x)** 求二叉树T的根结点, 或求结点x所在二叉树的根结点。
3. **PARENT(T,x)** 求二叉树T中结点x的双亲结点。
4. **LCHILD(T,x)**或**RCHILD(T,x)** 分别求二叉树T中结点x的左孩子结点或右孩子结点。
5. **LDELETE(T,x)**或**RDELETE(T,x)** 分别删除二叉树T中以结点x为根的左子树或右子树。
6. **TRAVERSE(T)** 按照某种次序(或原则)依次访问二叉树T中各个结点, 得到由该二叉树的所有结点组成的序列。
7. **LAYER(T,x)** 求二叉树中结点x所处的层次。
8. **DEPTH(T)** 求二叉树T的深度。
9. **DESTROY(T)** 销毁一棵二叉树。

.....



重点



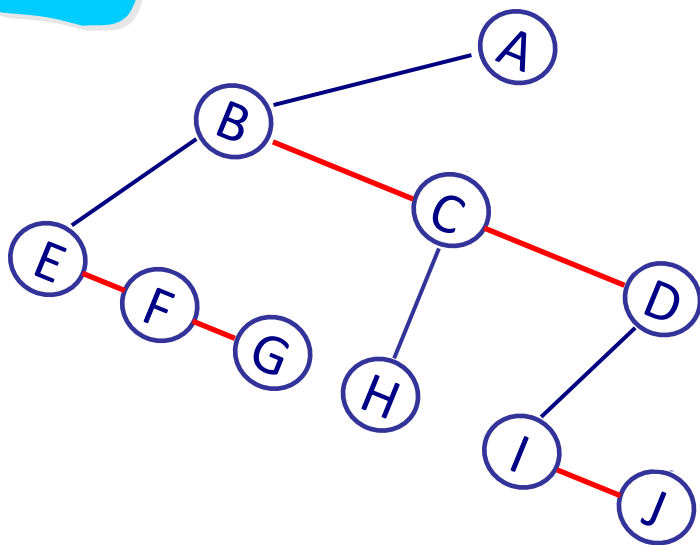
3.5 二叉树与树、树林之间的转换*

1. 树与二叉树的转换

步骤

- (1) 在所有相邻的兄弟结点之间分别加一条连线;
- (2) 对于每一个分支结点, 除了其最左孩子外, 删除该结点与其他孩子结点之间的连线;
- (3) 以根结点为轴心, 顺时针旋转 45° 。

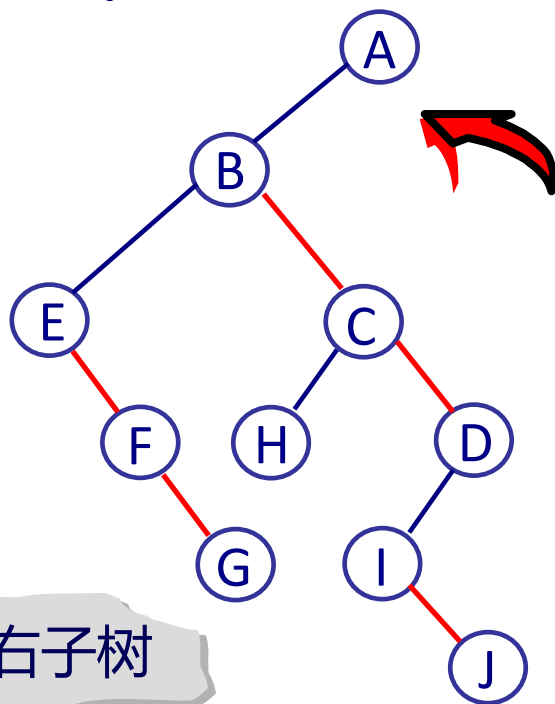
例



顺时针
旋转
 45°

特点

根结点无右子树

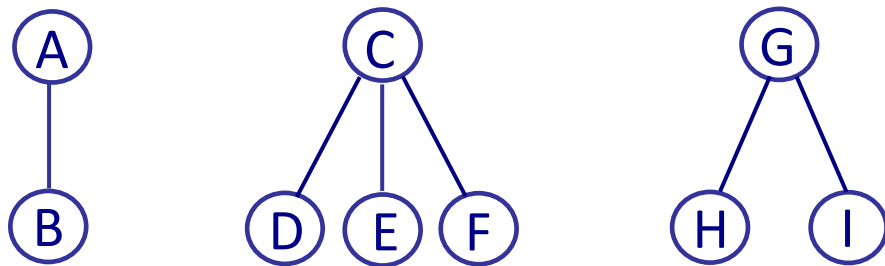




树林与二叉树的转换

2. 树林与二叉树的转换

例

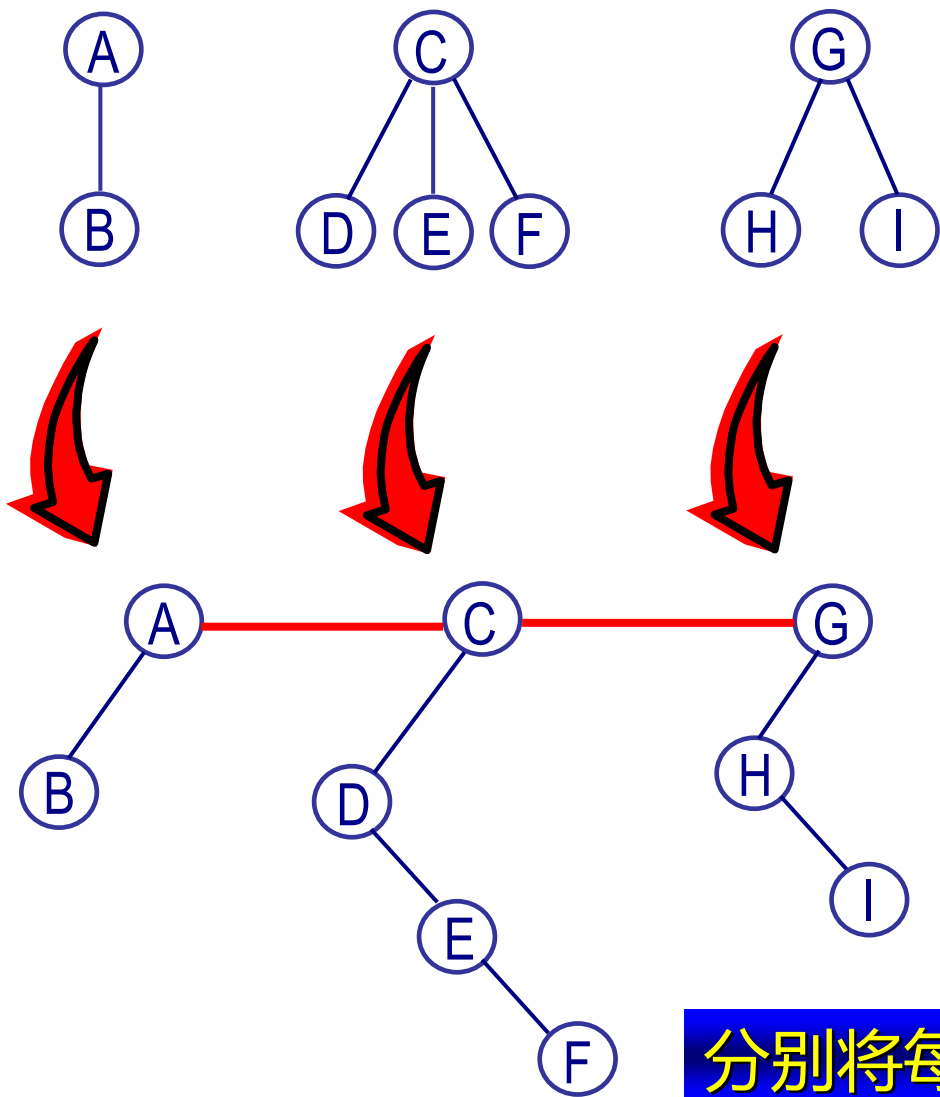


步骤

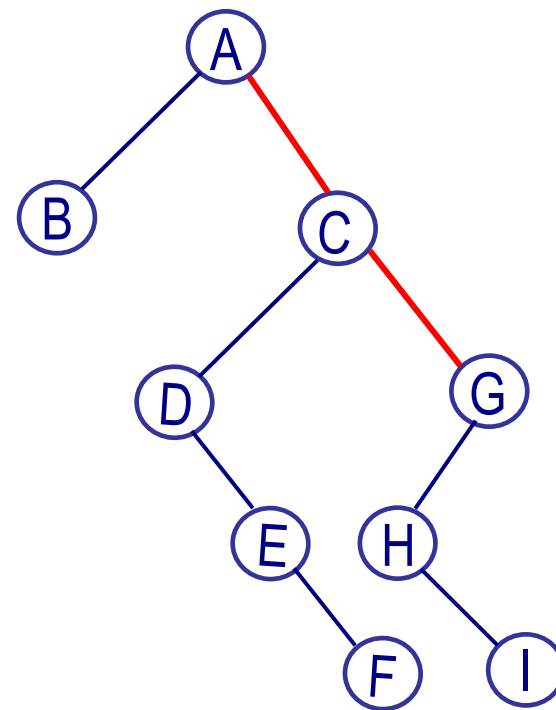
- (1) 分别将树林中每一棵树转换为一棵二叉树;
- (2) 从最后那一棵二叉树开始, 依次将后一棵二叉树的根结点作为前一棵二叉树的根结点的右孩子, 直到所有二叉树都这样处理。这样得到的二叉树的根结点是树林中第一棵二叉树的根结点。



示例：树林转换为二叉树



分别将每一棵树转换为二叉树



转换后的二叉树



二叉树还原为树

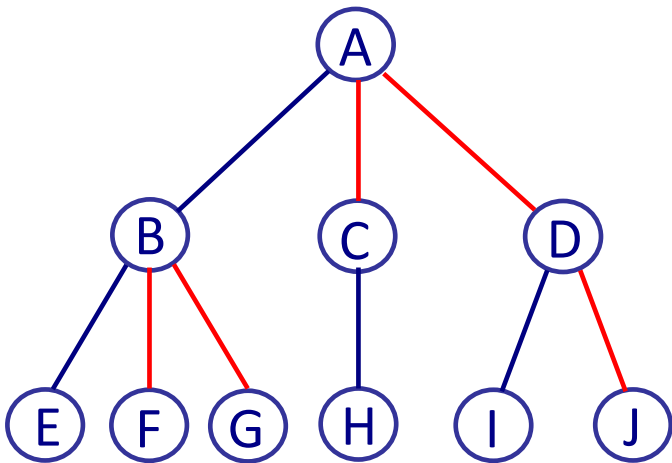
3. 二叉树还原为树

前提

由一棵树转换而来

步骤

1. 若某结点是其双亲结点的左孩子，则将该结点的右孩子以及当且仅当连续地沿此右孩子的右子树方向的所有结点都分别与该结点的双亲结点用一根虚线连接；
2. 去掉二叉树中所有双亲结点与其右孩子的连线；
3. 规整图形(即使各结点按照层次排列),并将虚线改成实线。





提纲：树和二叉树

- ◆ 4.1 树的基本概念
- ◆ 4.2 树的存储结构
- ◆ 4.3 二叉树
- ◆ 4.4 二叉树的存储结构
- ◆ 4.5 二叉树的遍历
- ◆ *4.6 线索二叉树
- ◆ 4.7 二叉查找树
- ◆ *4.8 堆和表达式树
- ◆ 4.9 哈夫曼树

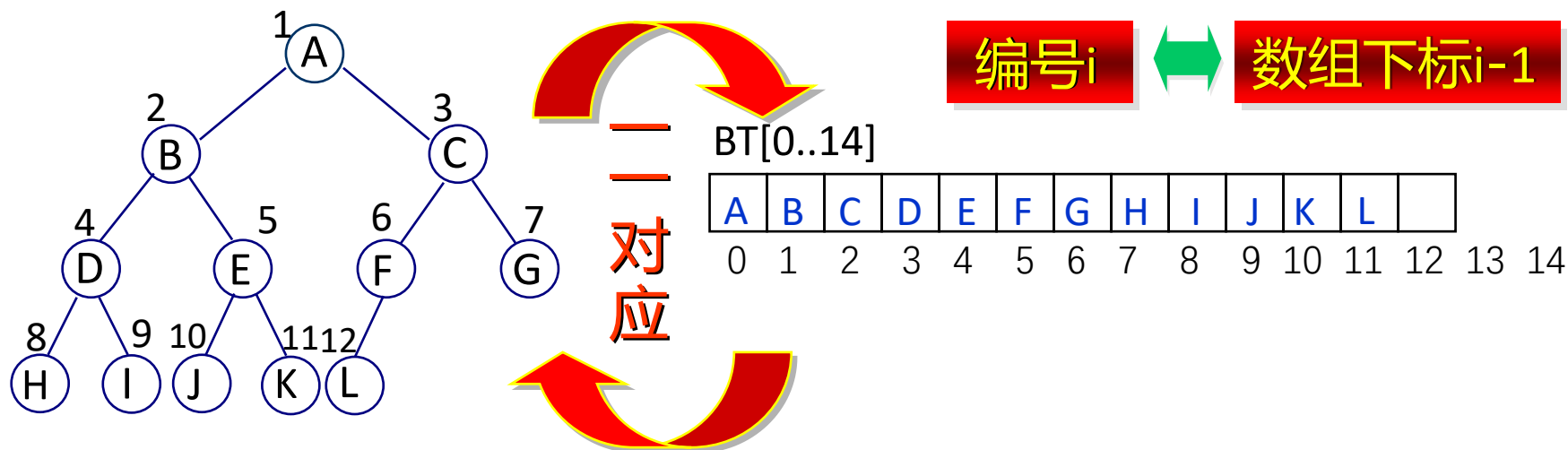


4. 二叉树的存储结构

4.1. 二叉树的顺序存储结构

1. 完全二叉树的顺序存储结构

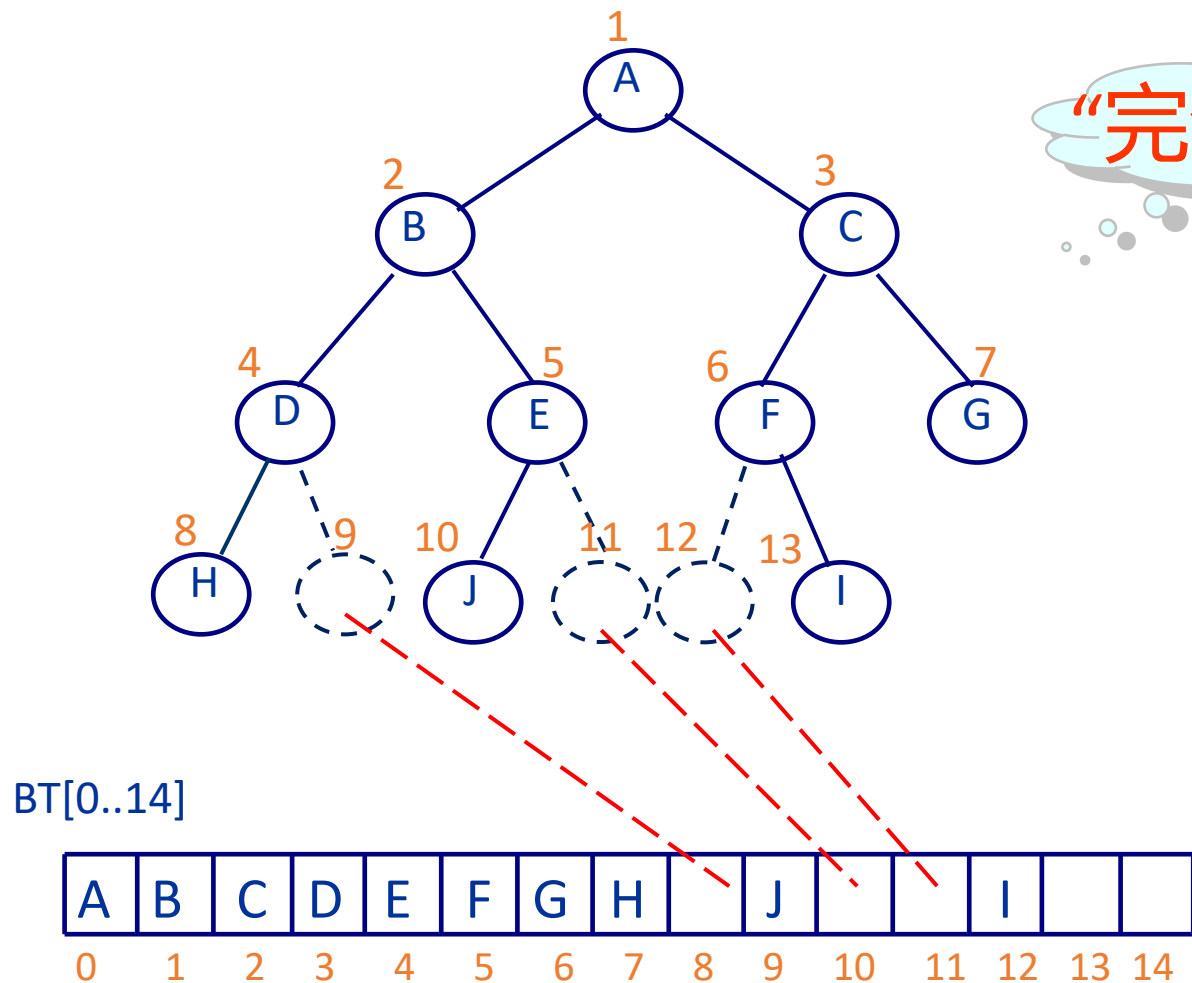
根据完全二叉树的性质6, 对于深度为 h 的完全二叉树, 将树中所有结点的数据信息按照编号的顺序依次存储到一维数组 $BT[0..2^h-2]$ 中, 由于编号与数组下标一一对应, 该数组就是该完全二叉树的顺序存储结构。





2. 一般二叉树的顺序存储

“完全二叉树”



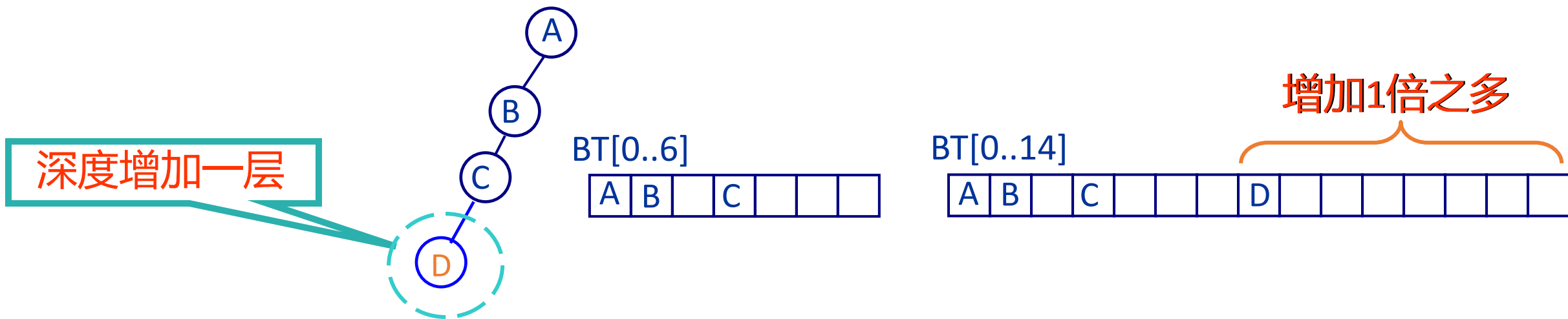
对于一般二叉树, 只须在二叉树中“添加”一些实际上二叉树中并不存在的“**虚结点**”(可以认为这些结点的数据信息为空), 使其在形式上成为一棵“**完全二叉树**”, 然后按照完全二叉树的顺序存储结构的构造方法将所有结点的数据信息依次存放于数组BT[0.. 2^h-2]中。



二叉树的顺序存储结构

◆二叉树的顺序存储结构特点

- ✓1.顺序存储结构比较适合满二叉树，或者接近于满二叉树的完全二叉树，对于一些称为“退化二叉树”的二叉树,顺序存储结构的时空开销浪费的缺点表现比较突出
- ✓2.顺序存储结构便于结点的检索（由双亲查子、由子查双亲）
- ✓3.顺序存储结构需要事先分配存储空间，对于动态数据容易溢出





二叉树的顺序存储

- ◆ 对于一般二叉树, 只须在二叉树中 “添加” 一些实际上二叉树中并不存在的 “**虚结点**” (可以认为这些结点的数据信息为空), 使其在形式上成为一棵 “**完全二叉树**”, 然后按照完全二叉树的顺序存储结构的构造方法将所有结点的数据信息依次存放于数组 $BT[0..2^h - 2]$ 中



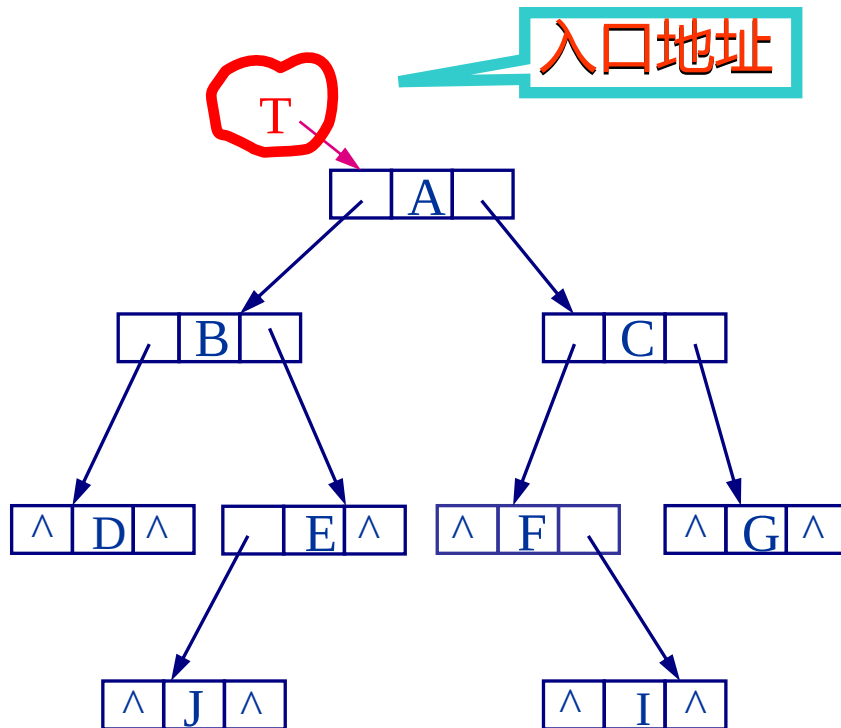
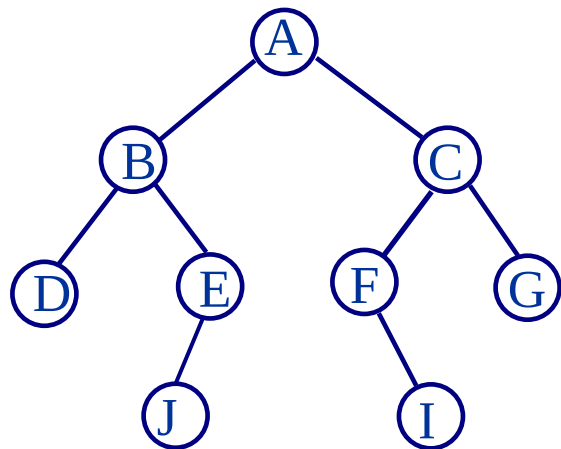
4.2 二叉树的链式存储结构

1. 二叉链表

链结点的构造为



其中, data 为数据域; lchild 与 rchild 分别为指向左、右子树的指针域





二叉链表的C语言实现

二叉链表的结点类型定义为

```
typedef struct _Node
{
    ElemType data;
    struct _Node *lchild, *rchild;
} BTreeNode, *BTreeNodeptr;
```

BTreeNodeptr T, p, q;



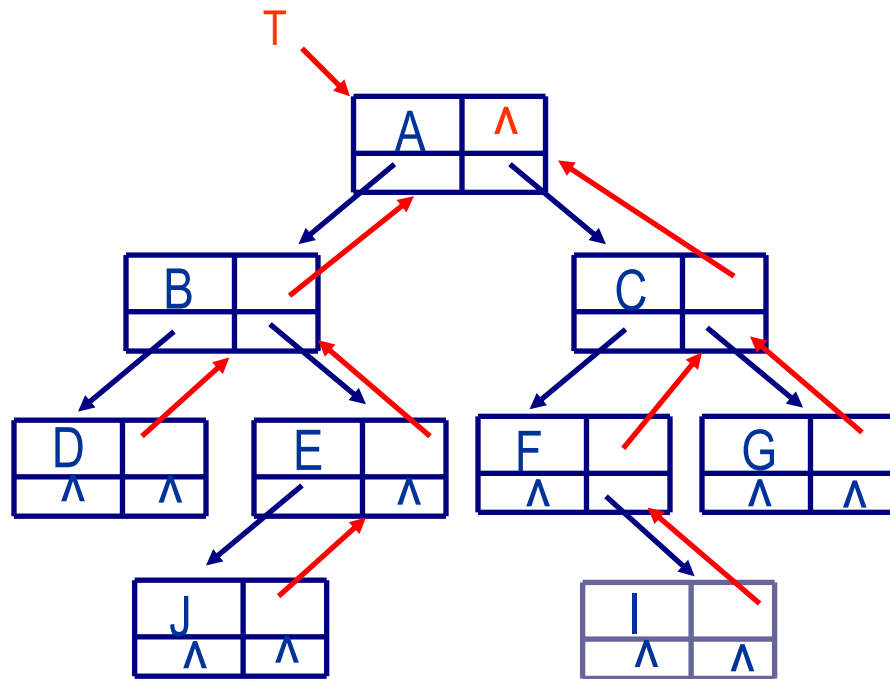
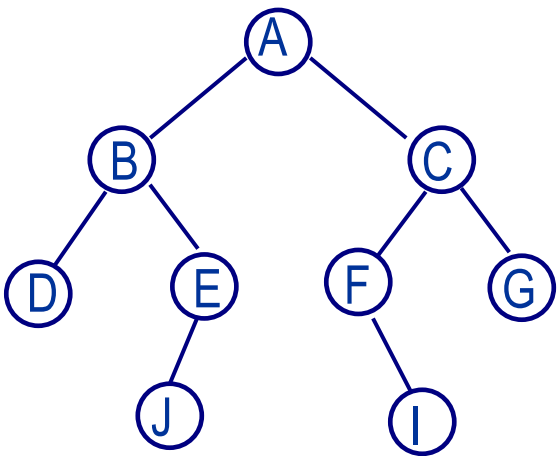
2. 三叉链表

2.三叉链表

链结点的构造为

data	parent
lchild	rchild

其中, data 为数据域, parent为指向**双亲结点的指针**;
lchild 与rchild 分别为指向**左、右孩子结点的指针**。





问题4.1-查家谱*

同姓氏中国人见面常说的一句话是“我们五百年前可能是一家”。从当前目录下的文件in.txt中读入一家谱，从标准输入读入两个人的名字（两人的名字肯定会在家谱中出现），编程查找判断这两个人相差几辈，若同辈，还要查找两个人共同的最近祖先以及与他（她）们的关系远近。假设输入的家谱中**每人最多有两个孩子**，例如下图是根据输入形成的一个简单家谱。

若要查找的两个人是wangqinian和wangguoan，从家谱中可以看出两人相差两辈；若要查找的两个人是wangping和wanglong，可以看出两人共同的最近祖先是wangguoan，和两人相差两辈。

【输入示例】

假设家谱文件中内容为：

6

wangliang wangguoping wangguoan

wangguoping wangtian wangguang

wangguoan wangxiang wangsong

wangtian wangqinian NULL

wangxiang wangping NULL

wangsong wanglong NULL

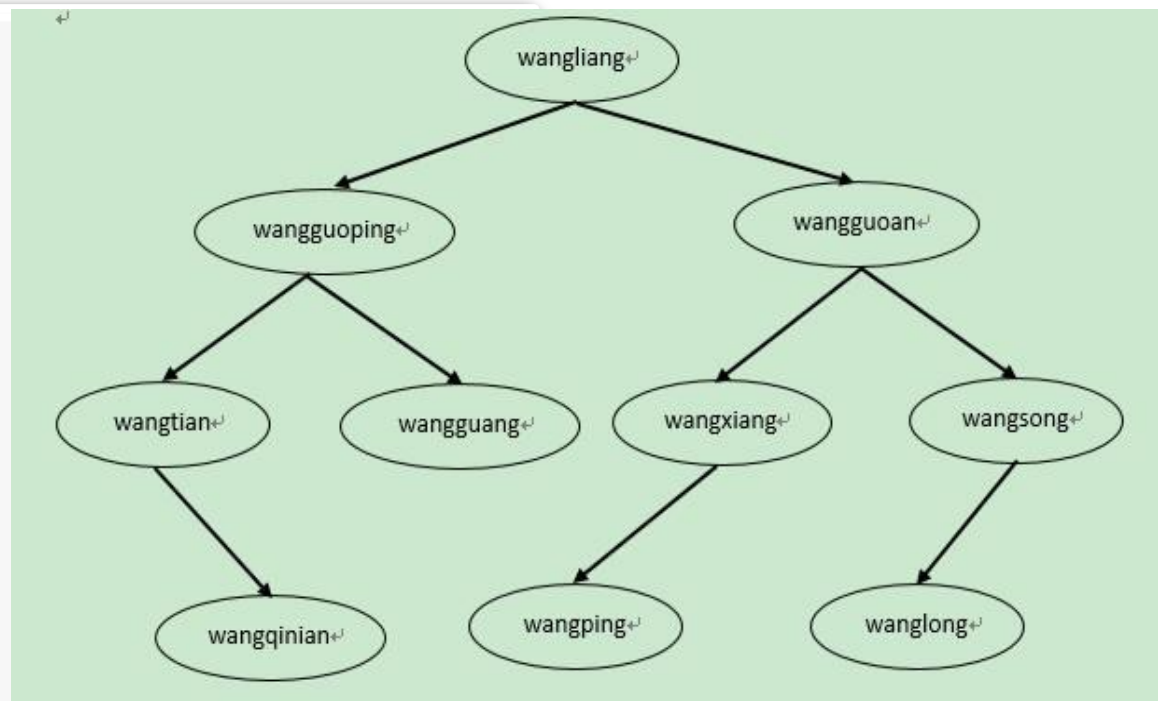
从标准输入读取： wangping wanglong

【输出示例】 wangguoan wangping 2

wangguoan wanglong 2

【说明】

wangping和wanglong共同的最近祖先是wangguoan，该祖先与两人相差两辈。





问题4.1*：问题分析与设计

- ◆查家谱：实际上就是查找相应节点。如果能得到节点至根的路径信息，就很容易计算出两个节点关系（如是否同辈、相差几辈、共同的祖先等）
- ◆如何在查找一个结点时得到其（从根结点至该结点的）路径信息：
 - ✓在前序查找过程中设置一个栈来保存路径信息
 - ✓一个简单的方法：为每个结点增加一个指向父结点的指针信息，这样在找到结点的同时，也就获得了相应的路径



提纲：树和二叉树

- ◆ 4.1 树的基本概念
- ◆ 4.2 树的存储结构
- ◆ 4.3 二叉树
- ◆ 4.4 二叉树的存储结构
- ◆ 4.5 二叉树的遍历
- ◆ *4.6 线索二叉树
- ◆ 4.7 二叉查找树
- ◆ *4.8 堆和表达式树
- ◆ 4.9 哈夫曼树

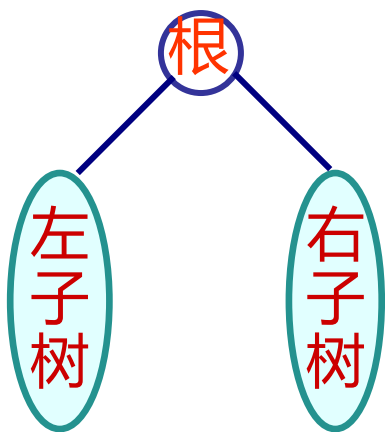


5. 二叉树的遍历

5.1 什么是二叉树的遍历

- ◆ 输出所有结点的信息;
- ◆ 找出或统计满足条件的结点;
- ◆ 求二叉树的深度;
- ◆ 求指定结点所在的层次;
- ◆

按照一定的顺序（原则）对二叉树中**每一个结点都访问一次（且仅访问一次）**，得到一个由该二叉树的所有结点组成的序列,这一过程称为二叉树的遍历



L表示遍历左子树； R表示遍历右子树；
D表示访问根结点；

常用的遍历方法:

- 前序遍历(DLR)
- 中序遍历(LDR)
- 后序遍历(LRD)
- 按层次遍历



5.2 前序遍历

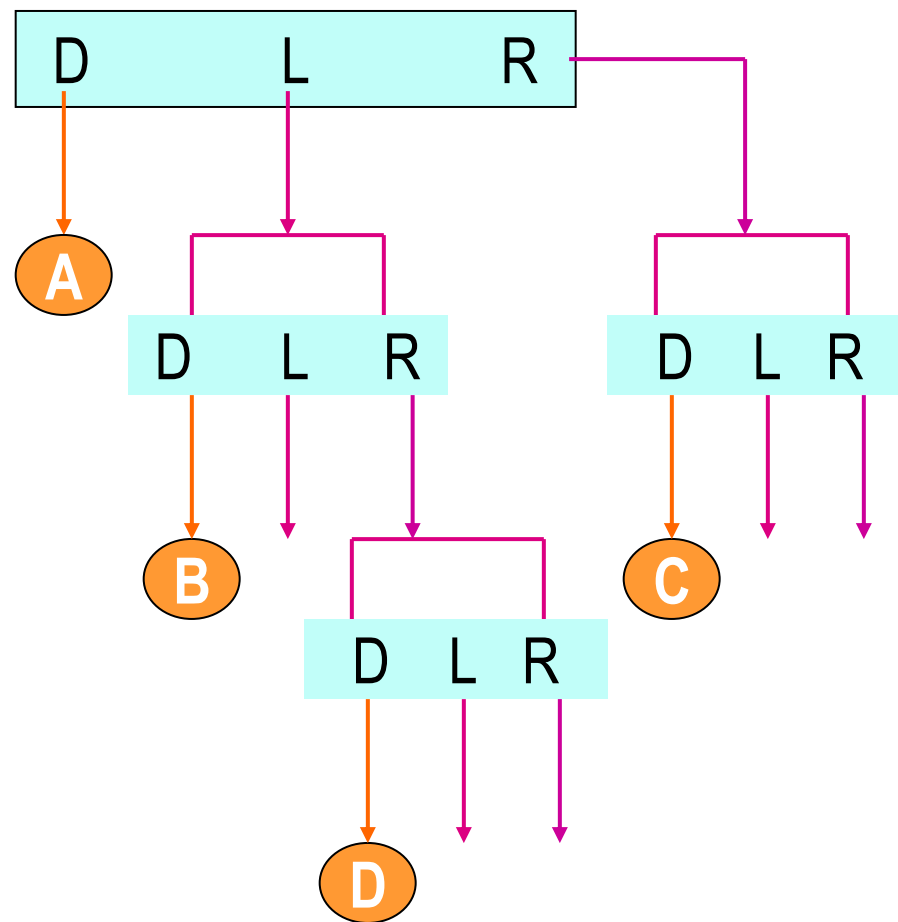
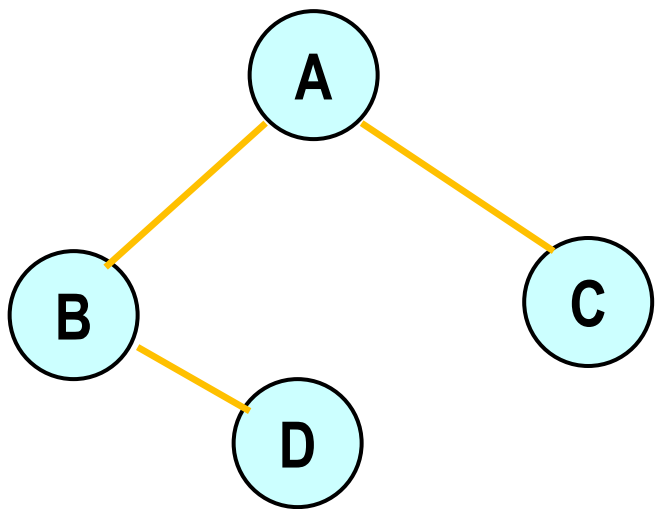
原则 若被遍历的二叉树非空, 则

递归

1. 访问根结点;

2. 以前序遍历原则遍历根结点的左子树;

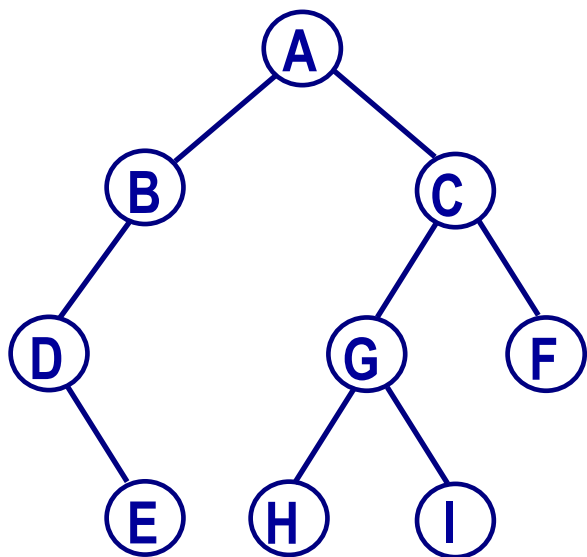
3. 以前序遍历原则遍历根结点的右子树。





前序遍历 (续)

递归算法



```
void preorder(BTNodeptr t)
{
    if (t != NULL)
    {
        VISIT(t); // 访问t结点
        preorder(t->lchild);
        preorder(t->rchild);
    }
}
```

前序序列: **ABDECGHIF**



前序遍历 (续)

```
void preorder(BTNodeptr t){  
    if (t != NULL)  
    {  
        VISIT(t); //访问t结点  
        preorder(t->lchild);  
        preorder(t->rchild);  
    }  
}
```

主程序

Pre(T)

前序序列: A B D C

t → A

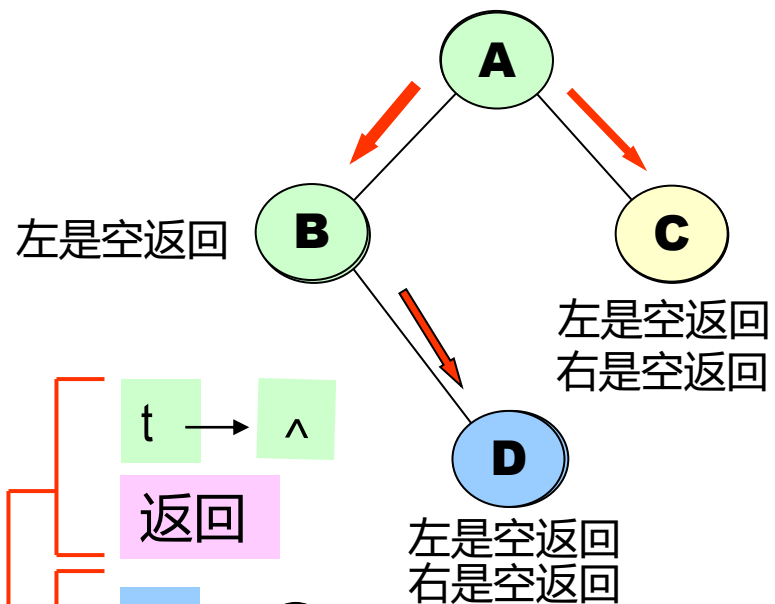
print(A);
pre(t → L);
pre(t → R);

t → B

print(B);
pre(t → L);
pre(t → R);

t → C

print(C);
pre(t → L);
Pre (t → R);



返回

t → D

print(D);
pre(t → L);
pre(t → R);

t → ^

返回

t → ^

返回

t → ^

返回

t → ^

返回

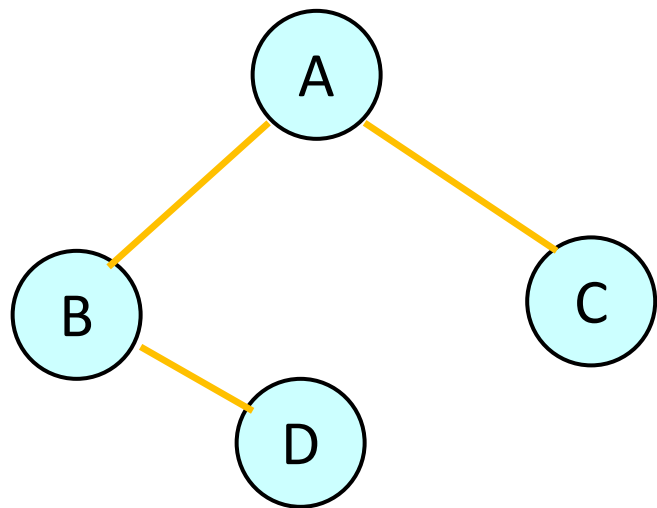


5.3 中序遍历

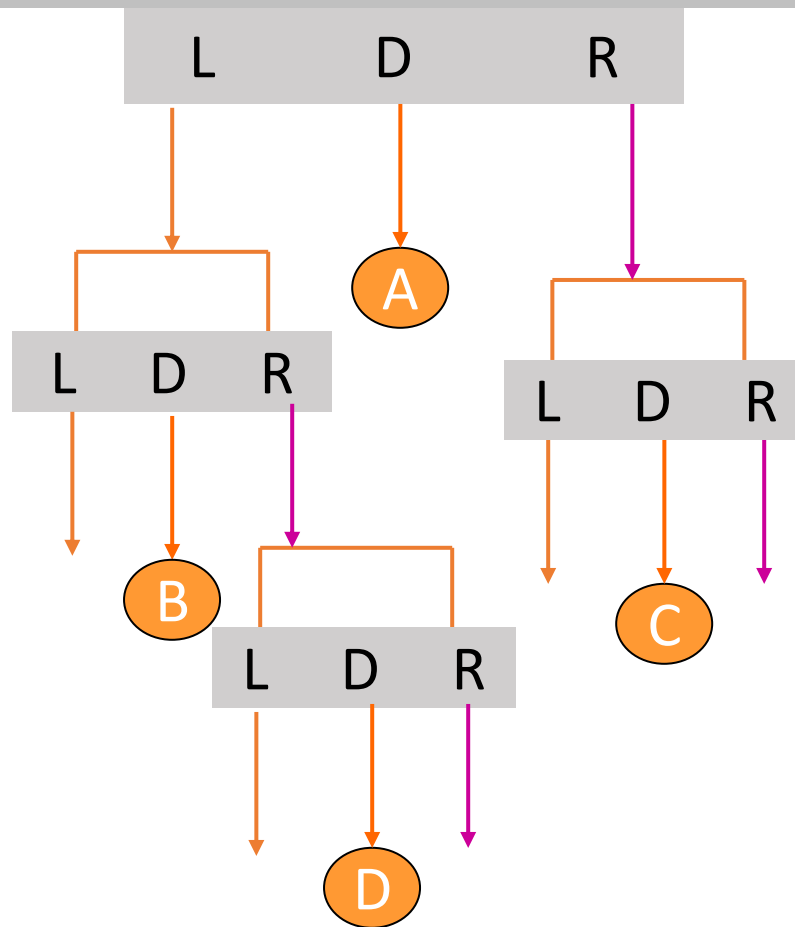
递归

原则 若被遍历的二叉树非空, 则

1. 以中序遍历原则遍历根结点的左子树;
2. 访问根结点;
3. 以中序遍历原则遍历根结点的右子树。



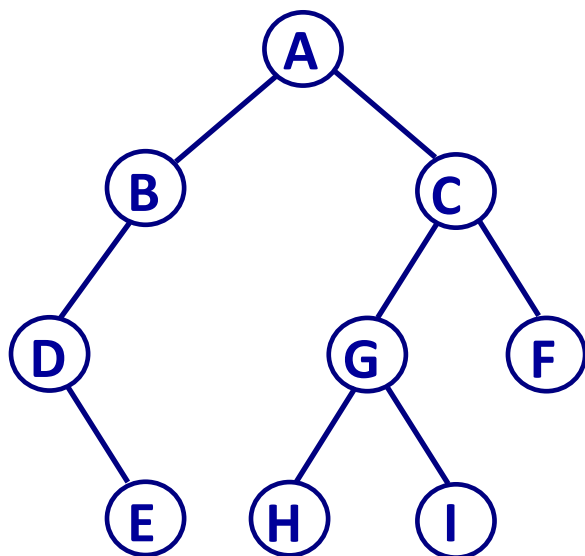
中序遍历序列: B D A C





中序遍历 (续)

递归算法



```
void inorder(BTNodeptr t)
{
    if (t != NULL)
    {
        inorder(t->lchild);
        VISIT(t); //访问t结点
        inorder(t->rchild);
    }
}
```

前序序列: **A B D E C G H I F**

中序序列: **D E B A H G I C F**



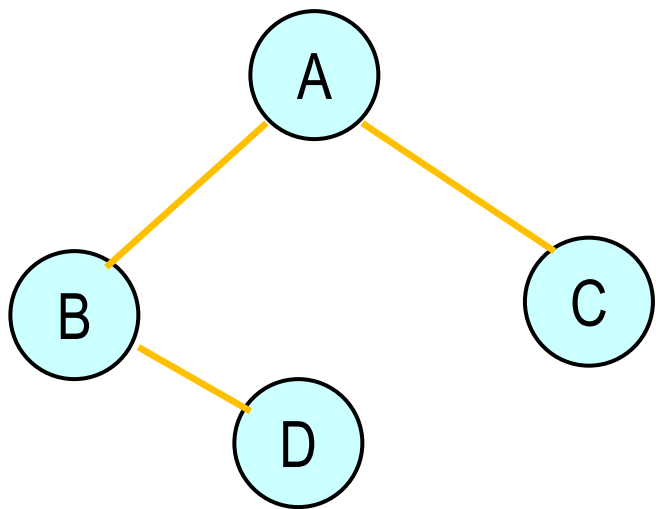
5.4 后序遍历

原则

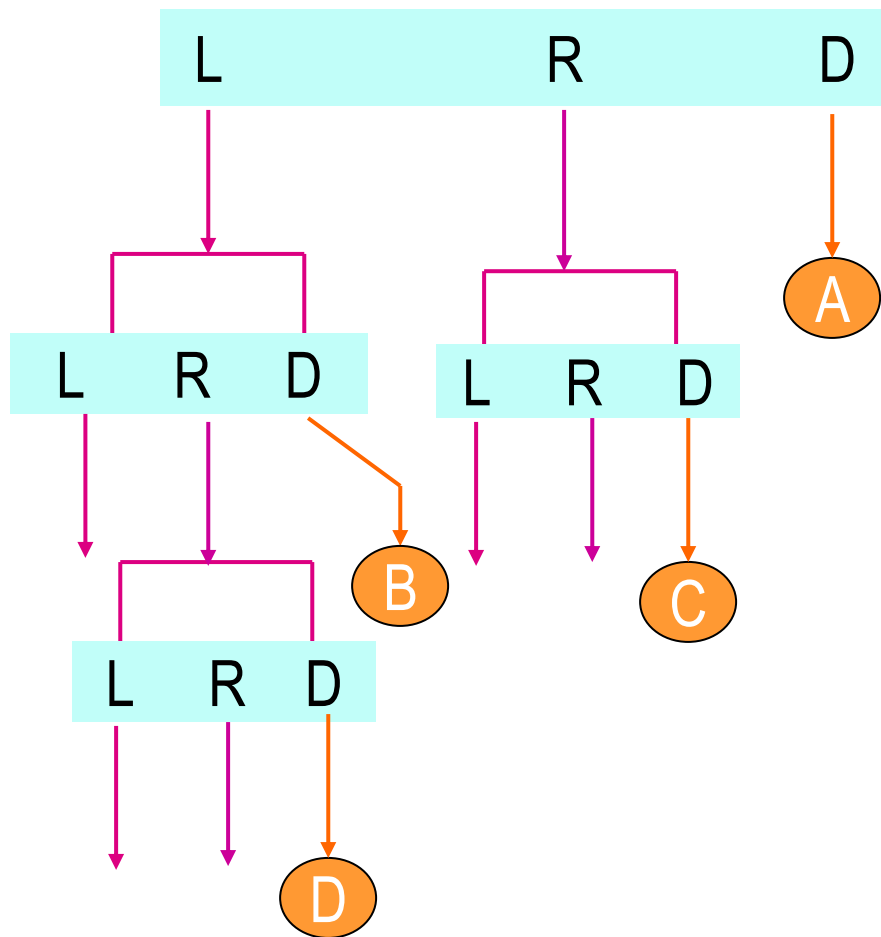
递归

若被遍历的二叉树非空, 则

1. 以后序遍历原则遍历根结点的左子树;
2. 以后序遍历原则遍历根结点的右子树;
3. 访问根结点。

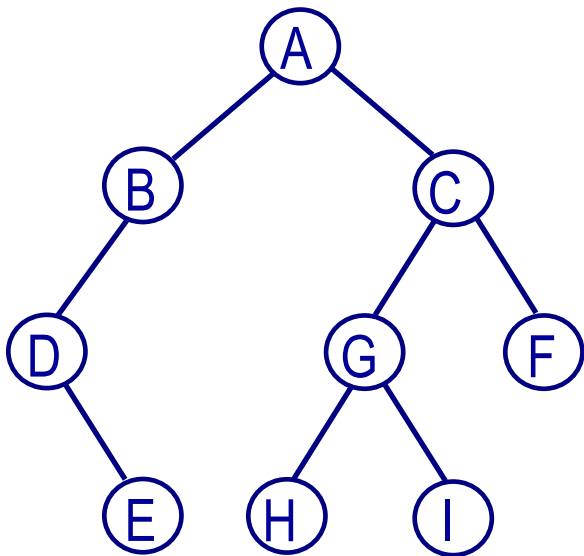


后序遍历序列: D B C A





后序遍历 (续)



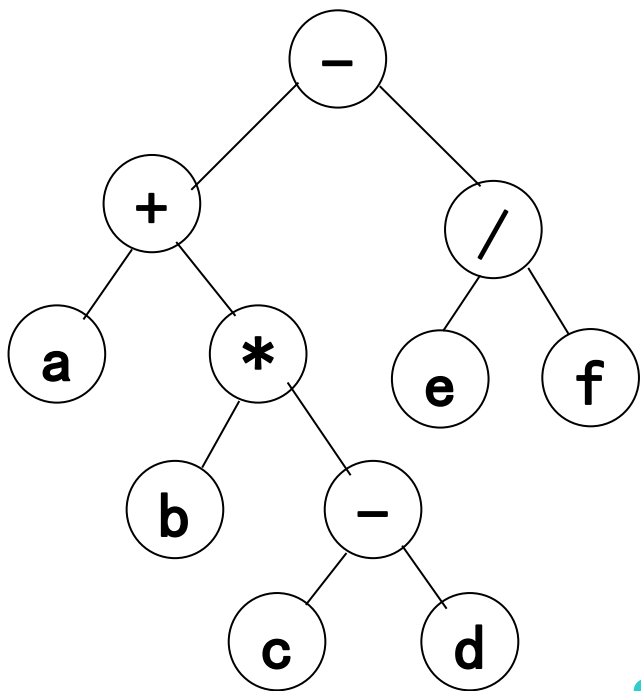
递归算法

```
void postorder(BTNodeptr t){  
    if (t != NULL)  
    {  
        postorder(t->lchild);  
        postorder(t->rchild);  
        VISIT(t); //访问t结点  
    }  
}
```

前序序列: ABDECGHIF
中序序列: DEBAHGICF
后序序列: EDBHIGFCA



练习：三种遍历方法



前序遍历: $- + a * b - c d / e f$

中序遍历: $a + b * c - d - e / f$

后序遍历: $a b c d - * + e f / -$

这是一个表达式树



二叉树的遍历算法应用

二叉树的拷贝

```
BTNodeptr copyTree(BTNodeptr src)
{
    BTNodeptr obj;
    if (src == NULL) obj = NULL;
    else
    {
        obj = (BTNodeptr)malloc(sizeof(BTNode));
        obj->data = src->data;
        obj->lchild = copyTree(src->lchild);
        obj->rchild = copyTree(src->rchild);
    }
    return obj;
} //二叉树拷贝
```

- 1) 树拷贝时先拷贝当前结点，再拷贝子结点（同前序遍历）；
- 2) 树删除时先删除子结点，再删除当前结点。（同后序遍历）

二叉树的删除

```
void destoryTree(BTNodeptr p){
    if (p != NULL)
    {
        destoryTree(p->lchild);
        destoryTree(p->rchild);
        free(p);
        p = NULL;
    }
} //二叉树删除
```

二叉树的高度

```
int max(x, y)
{if((x > y) return x; else return y;}
int heightTree(BTNodeptr p){
    if (p == NULL)
        return 0;
    else
        return 1 + max(heightTree(p->lchild),
                        heightTree(p->rchild));
} //计算二叉树树的高度
```



小结：二叉树的遍历（递归算法）

前序遍历

若被遍历的二叉树非空，则

1. 访问根结点;
2. 以前序遍历原则遍历根结点的左子树;
3. 以前序遍历原则遍历根结点的右子树。

中序遍历

若被遍历的二叉树非空，则

1. 以中序遍历原则遍历根结点的左子树;
2. 访问根结点;
3. 以中序遍历原则遍历根结点的右子树。

后序遍历

若被遍历的二叉树非空，则

1. 以后序遍历原则遍历根结点的左子树
2. 以后序遍历原则遍历根结点的右子树;
3. 访问根结点。



小结：二叉树的遍历（递归算法）

前序遍历

```
void preorder(BTNodeptr t){  
    if (t != NULL) {  
        VISIT(t); //访问t结点  
        preorder(t->lchild);  
        preorder(t->rchild);  
    }  
}
```

中序遍历

```
void inorder(BTNodeptr t){  
    if (t != NULL){  
        inorder(t->lchild);  
        VISIT(t); //访问t结点  
        inorder(t->rchild);  
    }  
}
```

后序遍历

```
void postorder(BTNodeptr t){  
    if (t != NULL){  
        postorder(t->lchild);  
        postorder(t->rchild);  
        VISIT(t); //访问t结点  
    }  
}
```



5.5 递归算法的非递归算法设计

◆递归算法的优点

- ✓ (1) 问题的数学模型本身就是递归的，采用递归算法来描述非常自然
- ✓ (2) 算法的描述直观，结构清晰，简洁
- ✓ (3) 算法的正确性证明比非递归算法容易

◆递归算法的不足

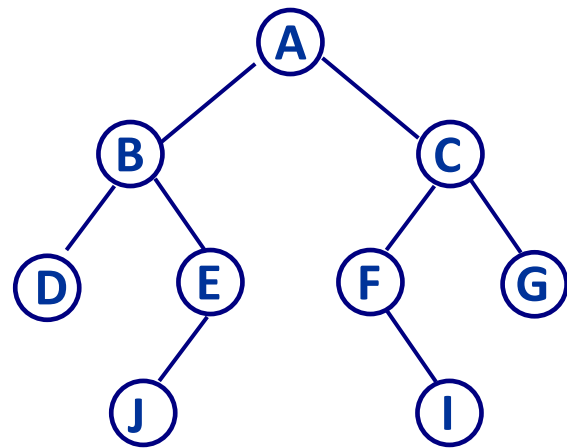
- ✓ (1) 算法的执行时间与空间开销往往比非递归算法大，当问题规模较大时尤为明显
- ✓ (2) 对算法进行优化比较困难
- ✓ (3) 分析和跟踪算法的执行过程比较麻烦
- ✓ (4) 描述算法的而语言不具有递归功能时，算法无法描述



二叉树遍历的非递归算法设计

递归算法 (中序)

```
void inorder(BTNodeptr t){  
    if (t != NULL)  
    {  
        inorder(t->lchild);  
        VISIT(t); //访问T指结点  
        inorder(t->rchild);  
    }  
}
```



如何设计非递归算法?



中序遍历的非递归算法实现

中序遍历非递归算法思路

1. 若p指向的结点非空，则将该结点地址进栈，然后，将p指向左子树的根，循环直到P为空;
 2. 若p指向的结点为空，则从堆栈中退出栈顶元素送p，访问该结点，然后，将p指向右子树的根;
- 重复上述过程,直到p为空,并且堆栈也为空。

$p = p \rightarrow lchild;$

$p = p \rightarrow rchild;$

STACK[0..M-1] -- 保存遍历过程中结点的地址;

top -- 栈顶指针，初始为-1;

p -- 为遍历过程中使用的指针变量，初始时指向根结点。



中序遍历的非递归算法

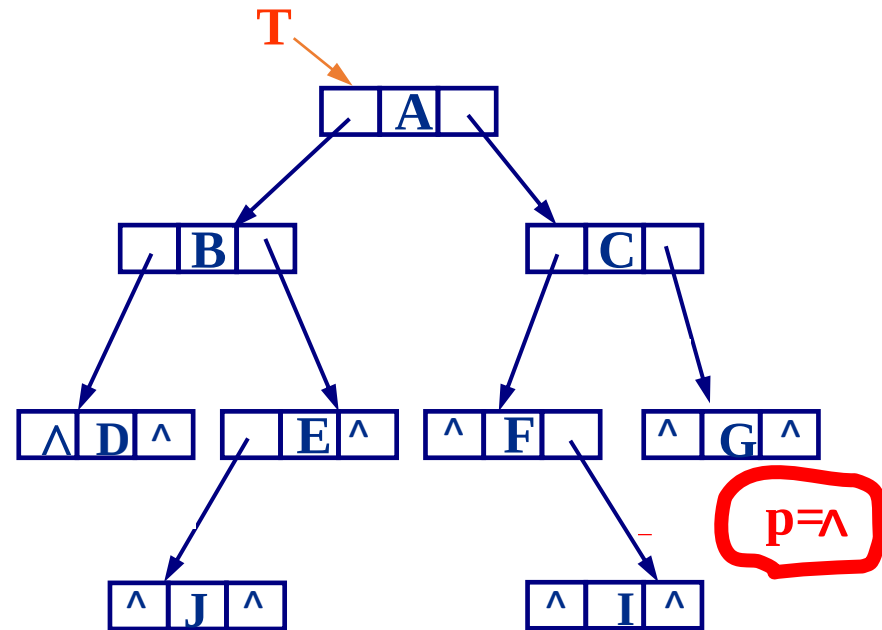
```
void inorder(BTNodeptr t){
    BTNodeptr stack[M], p = t;
    int top = -1;
    if (t != NULL)
    do{
        for (;p!=NULL;p=p->lchild)
        {
            stack[++top] = p;
        }
        p = stack[top--];
        VISIT(p);
        p = p->rchild;
    } while (p != NULL || top != -1);
}
```

前序遍历

堆栈

中序序列:

D, B, J, E, A, F, I, C, G



D B J E A F I C G



后序遍历的非递归算法

stack1[0..M-1] -- 保存遍历过程中结点的地址;

stack2[0..M-1] – 栈中结点是否可被访问标志: 0为不可访问, 1为可访问;

两个栈使用一个栈顶指针top;

p -- 为遍历过程中使用的指针变量, 初始时指向根结点。

后序遍历算法基本思路

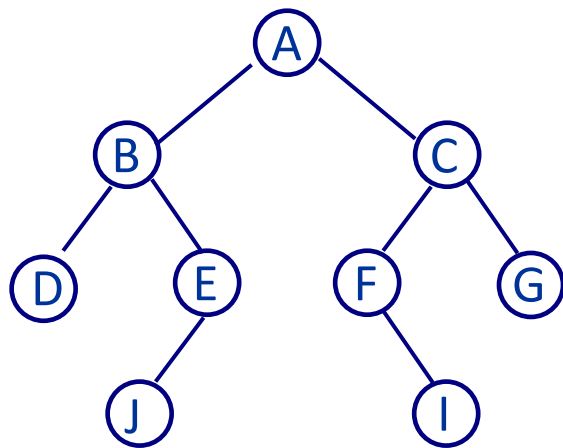
1. 若p所指结点非空, 则将该结点的地址及不可访问的标志进栈, 然后将p指向其左子树的根, 循环直到P为空;
2. 栈顶结点的地址退栈送至P, 相应标志位退栈送至flag;
3. 若flag为不可访问标志, 则将p所指结点地址及该结点可访问标志重新进栈, 移动p指向其右孩子结点; 否则访问P所指结点, 并置P为空 (表明以P为根的子树已经访问完成)。

重复上述过程,直到p为空,并且堆栈也为空。



后序遍历的非递归算法

```
void postorder(BTNodeptr t){
    BTNodeptr stack1[M], p = t;
    int stack2[M], top = -1, flag;
    if (t != NULL)
        do{
            while (p != NULL){
                stack1[++top] = p; //当前P所指结点的地址进栈
                stack2[top] = 0; //当前P所指结点不可访问标志进栈
                p = p->lchild;
            } // P指向其左孩子结点
            p = stack1[top];
            flag = stack2[top--]; //退栈
            if (flag == 0){
                stack1[++top] = p; //当前P所指结点再次进栈
                stack2[top] = 1; // 当前P所指结点可访问标志进栈
                p = p->rchild;
            }
            else{VISIT(p);
                p = NULL; //表明以P结点为根的树访问完成
            }
        }while (p != NULL || top != -1);
}
```



后序序列: D, J, E, B, I, F, G, C, A



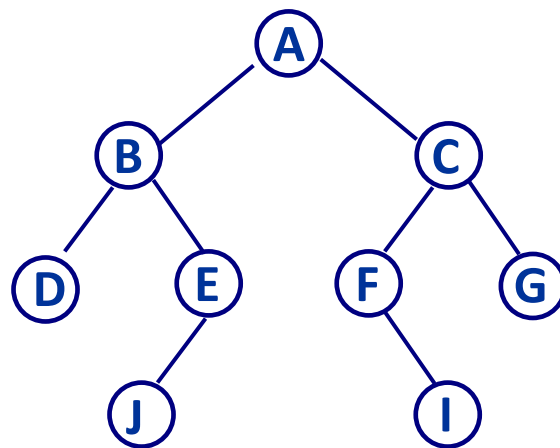
5.6 按层次遍历

◆按层次遍历

✓若被遍历二叉树非空，则按照层次从上到下，每一层从左到右依次访问结点

按层次遍历序列:

A B C D E F G J I



前序、中序及后序遍历实质为深度优先算法 (DFS)
层次遍历为一种广度优先算法 (BFS)



层次遍历算法实现

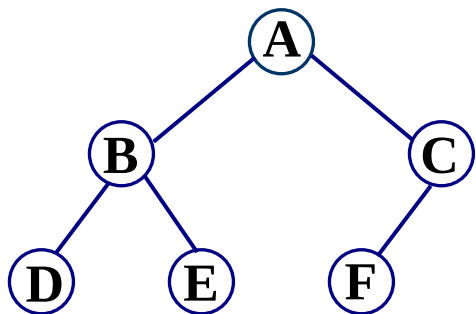
```
#define NodeNum 100
void layerorder(BTNodeptr t){
    BTNodeptr queue[NodeNum], p;
    int front, rear;
    if (t != NULL) {
        queue[0] = t;
        front = -1;
        rear = 0;
        while (front < rear){ // 若队列不空
            p = queue[++front];
            VISIT(p);          // 访问p指结点
            if (p->lchild != NULL) // 若左孩子非空
                queue[++rear] = p->lchild;
            if (p->rchild != NULL) // 若右孩子非空
                queue[++rear] = p->rchild;
        }
    }
}
```



5.7 二叉树顺序存储的遍历算法

- ◆ 已知具有 n 个结点的**完全二叉树**采用顺序存储结构，结点数据信息依次存放于一维数组 $BT[0..n-1]$ 中，写出中序遍历二叉树的非递归算法

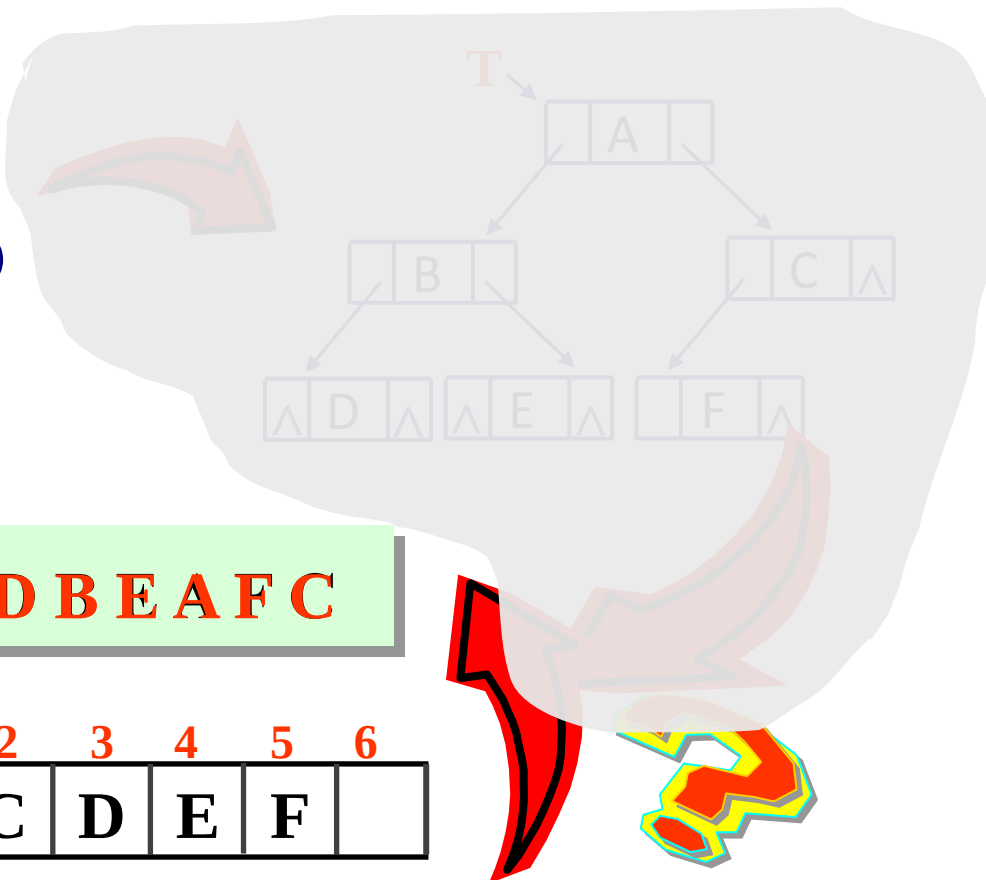
完全二叉树



中序序列: **D B E A F C**

BT[0..6]

0	1	2	3	4	5	6
A	B	C	D	E	F	





遍历算法的核心思想

设置一个 **栈** 保存遍历过程中结点的**位置**;
设置一个**变量**, 初始时给出根结点的**位置**;

反复执行下列过程:

1. 若变量所指位置上的结点存在, 则将该变量 所指**位置**进栈,
然后将该变量移到左孩子;
2. 若变量所指位置上的结点不存在, 则从栈中退出栈顶元素送变
量, 访问该变量位置上的结点, 然后将该变量移到右孩子;

直到变量所指位置上结点不存在, 同时堆栈也为空。



算法设计

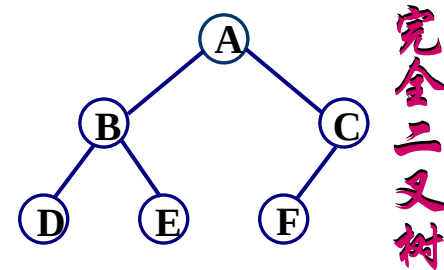
STACK[0..M-1] -- 保存遍历过程中结点的 **位置(下标)**;
 top -- 栈顶指针, 初始为-1;
 i -- 为遍历过程中使用的 **位置** 变量, 初始指向根结点。

i=0, BT[0]

1. 若 **i** 指向的结点非空 则将 **i** 进栈, 然后将 **i** 指向左子树的根; **i=2*i+1;**
2. 若 **i** 指向的结点为空, 则从堆栈中退出栈顶元素送 **i**, 访问该结点, 然后, 将 **i** 指向右子树的根; **i=2*i+2;**

重复上述过程, 直到 **i** 指结点不存在, 并且栈空。

顺序存储结构



	0	1	2	3	4	5	6
BT[0..6]	A	B	C	D	E	F	



算法实现

```
#define MaxSize 100
void inorder(Datatype bt[], int n){
    int stack[MaxSize], i, top = -1;
    i = 0;
    if (n >= 0){
        do {
            while (i < n)
            {
                stack[++top] = i; /* bt[i]的位置i进栈*/
                i = i * 2 + 1;    /* 找到i的左孩子的位置 */
            }
            i = STACK[top--]; /* 退栈*/
            VISIT(bt[i]);    /* 访问结点bt[i] */
            i = i * 2 + 2;    /* 找到i的右孩子的位置*/
        } while (!(i > n - 1 && top == -1));
    }
}
```



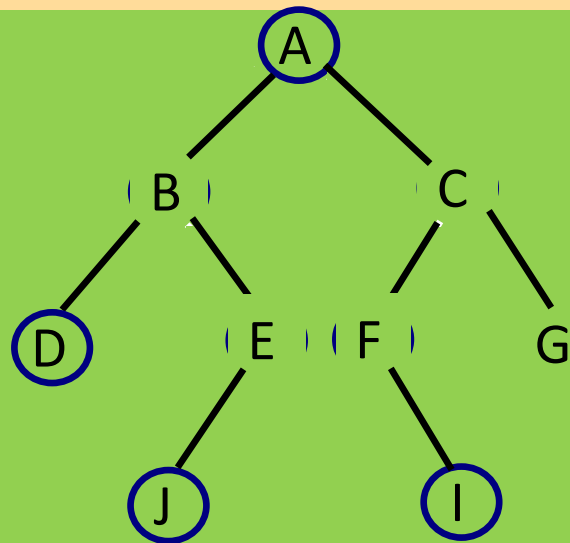
5.8 由遍历序列恢复二叉树

前序序列:

A, B, D, E, J, C, F, I, G

中序序列:

D, B, J, E, A, F, I, C, G



利用前序序列和中序序列恢复二叉树 ✓

利用中序序列和后序序列恢复二叉树 ✓

利用前序序列和后序序列恢复二叉树 ✗

后序序列:

D, J, E, B, I, F, G, C, A



由遍历序列恢复二叉树

已知前序序列和中序序列，恢复二叉树
在前序序列中确定根；
到中序序列中分左右。

规律

已知中序序列和后序序列，恢复二叉树
在后序序列中确定根；
到中序序列中分左右。



练习

前序序列:

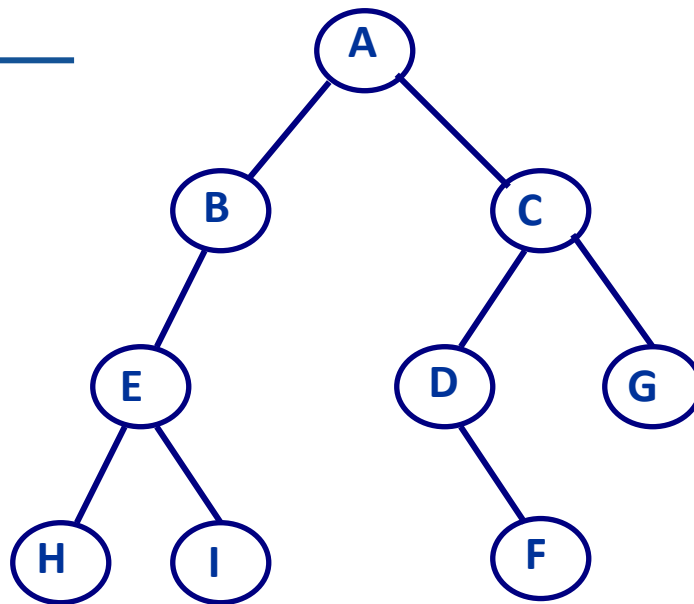
ABEHICDFG

中序序列:

HEIBADFCG

后序序列:

HIEBFDGCA





5.9 树的遍历

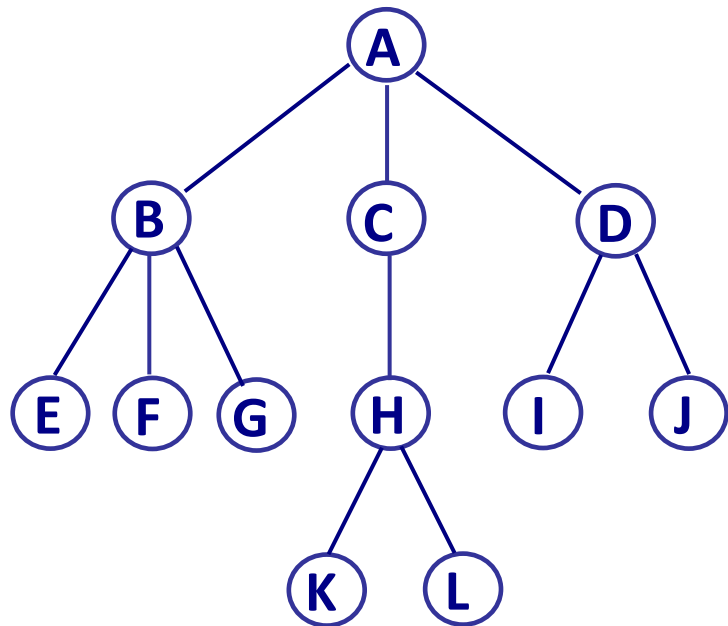
1. 树的遍历(深度优先)

(1) 前序遍历

若被遍历的树非空,则

- ① 访问根结点;
- ② 依次按前序遍历方式遍历根结点每一棵子树。

类似转换后的二
叉树的前序遍历



前序遍历序列: **A B E F G C H K L D I J**

(2) 后序遍历

若被遍历的树非空,则

- ① 依次按后序遍历方式遍历根结点每一棵子树;
- ② 访问根结点。

类似转换后的二
叉树的中序遍历

后序遍历序列: **E F G B K L H C I J D A**



树的遍历算法

深度优先遍历算法

```
void DFStree(TNodeptr t){
    int i;
    if (t != NULL) {
        VISIT(t); /* 访问t指向结点 */
        for (i = 0; i < MAXD; i++)
            if (t->next[i] != NULL)
                DFStree(t->next[i]);
    }
}
```

```
#define MAXD 3 //树的度
struct node{
    DataType data;
    struct node *next[MAXD];
};
typedef struct node TNode;
typedef struct node *TNodeptr;
```

广度优先遍历算法

```
void BFStree(TNodeptr t){
    TNodeptr p; int i;
    if (t != NULL) {
        enqueue(t);
        while (!isEmpty()){ //若队列不空
            p = dequeue();
            VISIT(p);
            for (i = 0; i < MAXD; i++) //依次访问p指向的子结点
                if (p->next[i] != NULL) enqueue(p->next[i]);
        }
    }
}
```



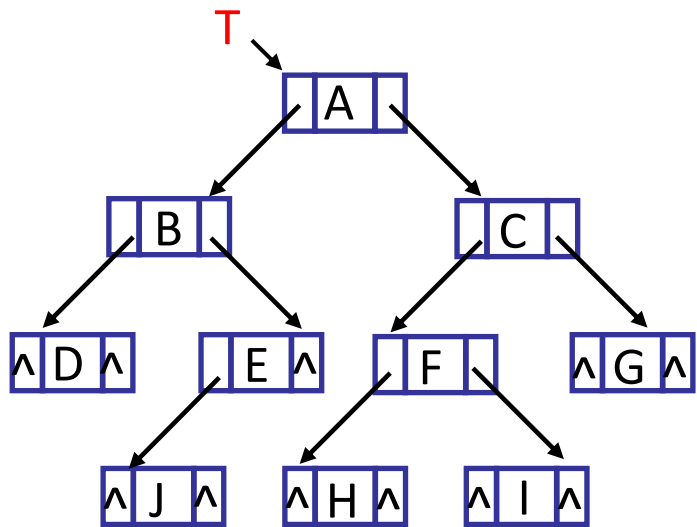
提纲：树和二叉树

- ◆ 4.1 树的基本概念
- ◆ 4.2 树的存储结构
- ◆ 4.3 二叉树
- ◆ 4.4 二叉树的存储结构
- ◆ 4.5 二叉树的遍历
- ◆ *4.6 线索二叉树
- ◆ 4.7 二叉查找树
- ◆ *4.8 堆和表达式树
- ◆ 4.9 哈夫曼树



6. 线索二叉树*

6.1 基本概念



中序序列:

D, B, J, E, A, H, F, I, C, G

如何在存储结构中方便地知道结点的直接前驱或后继

对于具有 n 个结点的二叉树,二叉链表中空的指针域数目为

$n+1$

$$2n - (n-1) = n+1$$

指针域总数

已用指针域数



线索二叉树

◆ 线索二叉树

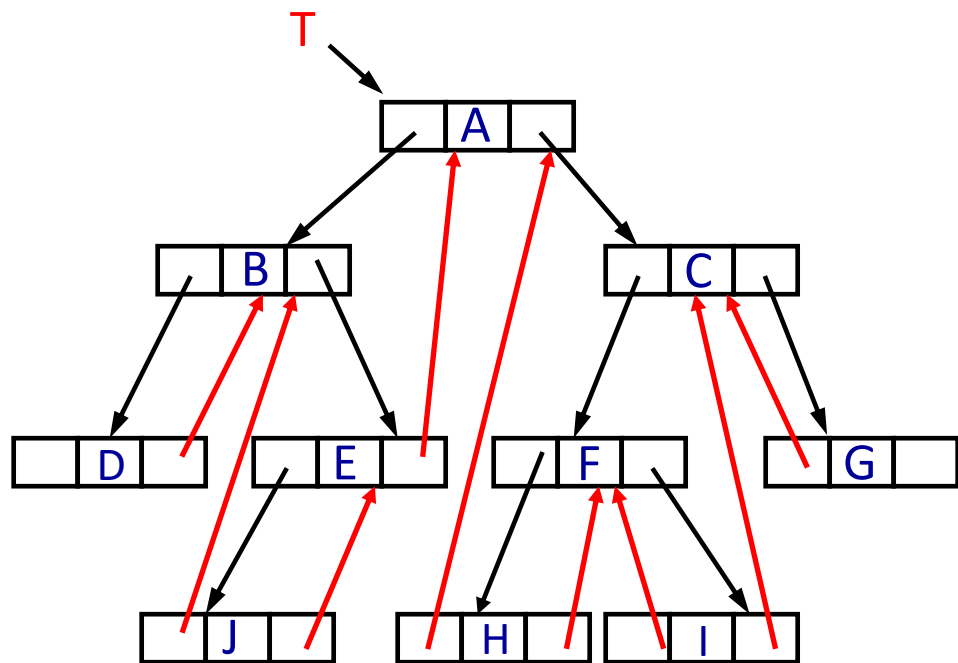
✓ 利用二叉链表中空的指针域指出结点在某种遍历序列中的直接前驱或直接后继，指向前驱和后继的指针称为线索，加了线索的二叉树称为线索二叉树

◆ 构造线索二叉树

✓ 利用链结点的空的左指针域存放该结点的直接前驱的地址，空的右指针域存放该结点直接后继的地址；而非空的指针域仍然存放结点的左孩子或右孩子的地址



中序线索二叉树



中序序列:

D, B, J, E, A, H, F, I, C, G

lbit	lchild	data	rchild	rbit
------	--------	------	--------	------

$p \rightarrow lbit = \begin{cases} 0 & \text{表示 } p \rightarrow lchild \text{ 为指向直接前驱的线索} \\ 1 & \text{表示 } p \rightarrow lchild \text{ 为指向左孩子的指针} \end{cases}$

$p \rightarrow rbit = \begin{cases} 0 & \text{表示 } p \rightarrow rchild \text{ 为指向直接后继的线索} \\ 1 & \text{表示 } p \rightarrow rchild \text{ 为指向右孩子的指针} \end{cases}$

如何区分
指针和线索





线索二叉树链节点类型定义

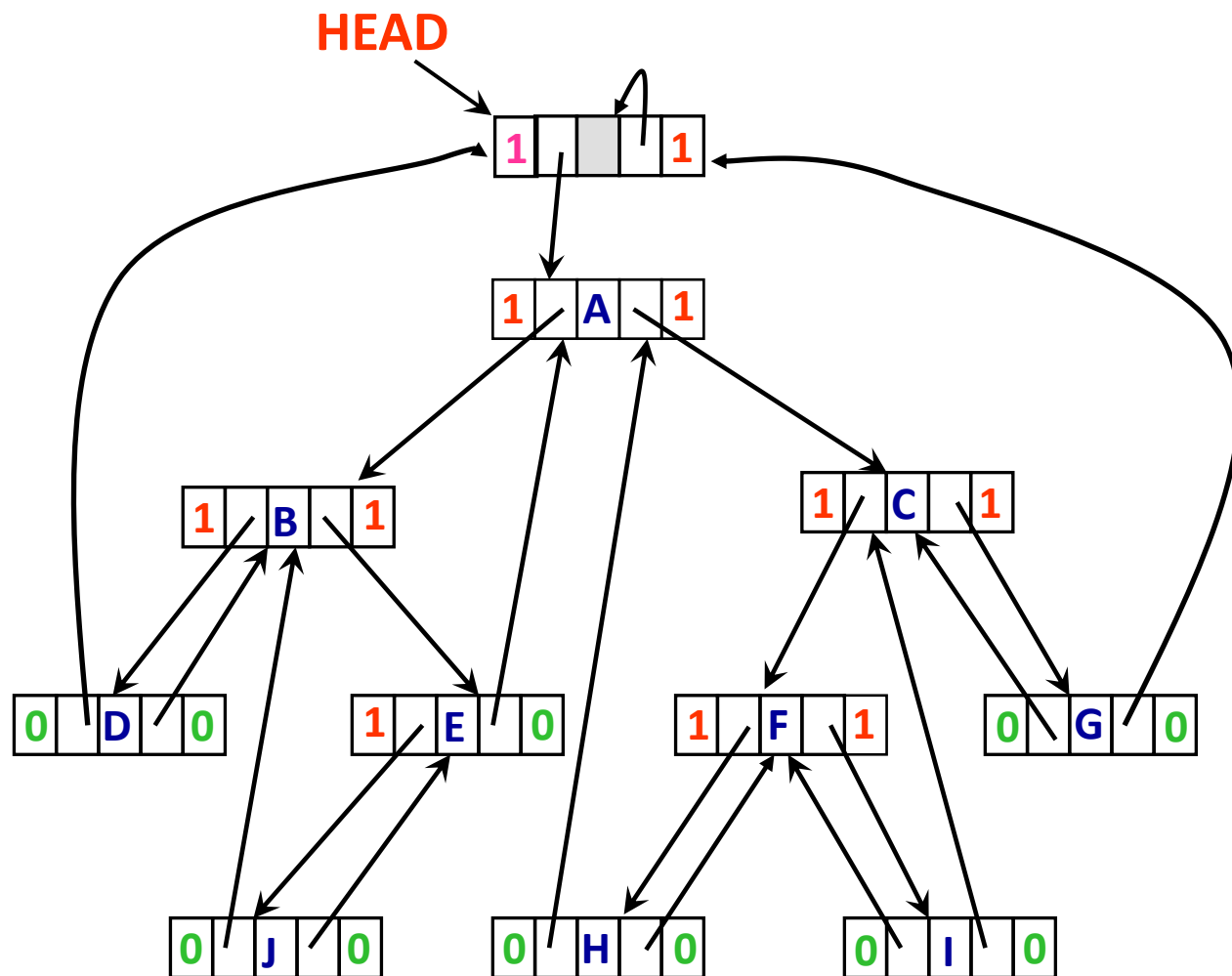
```
struct node
{
    DataType data;
    struct node *lchild, *rchild;
    char lbit, rbit;
};
typedef struct node TBTNode;
typedef struct node *TBTNodeptr;
```





增加头结点后完整的中序线索二叉树

中序序列 D, B, J, E, A, H, F, I, C, G





二叉树的线索化

◆通过遍历可以实现二叉树的线索化

- ✓线索过程请参阅教材和PPT相关内容

◆线索化二叉树等于将一棵二叉树转变成了一个双向链表，这为二叉树结点的插入、删除和查找带来了方便

- ✓在实际问题中，如果所用的二叉树需要经常遍历或查找结点时需要访问结点的前驱和后继，则采用线索二叉树结构是一个很好的选择

- ✓将二叉树线索化可以实现不用栈的树深度优先遍历算法



提纲：树和二叉树

- ◆4.1 树的基本概念
- ◆4.2 树的存储结构
- ◆4.3 二叉树
- ◆4.4 二叉树的存储结构
- ◆4.5 二叉树的遍历
- ◆*4.6 线索二叉树
- ◆4.7 二叉查找树
- ◆*4.8 堆和表达式树
- ◆4.9 哈夫曼树



词频统计再讨论

◆线性表中两种词频统计实现方法各有优缺点

◆基于顺序表（数组）

✓优点：

- 单词查找效率高（可用折半查找）

✓缺点：

- 1. 单词表长度需要事先定义，容易造成空间浪费或不足
- 2. 插入操作效率低（需要移动大量数据）

◆基于链表

✓优点：

- 空间动态管理（随问题规模动态改变），程序具有可扩展性
- 单词插入不需要移动数据，效率高

✓缺点：

- 单词查找效率低（每次要从头开始查找）

有没有一种方法能兼顾两者的优点？

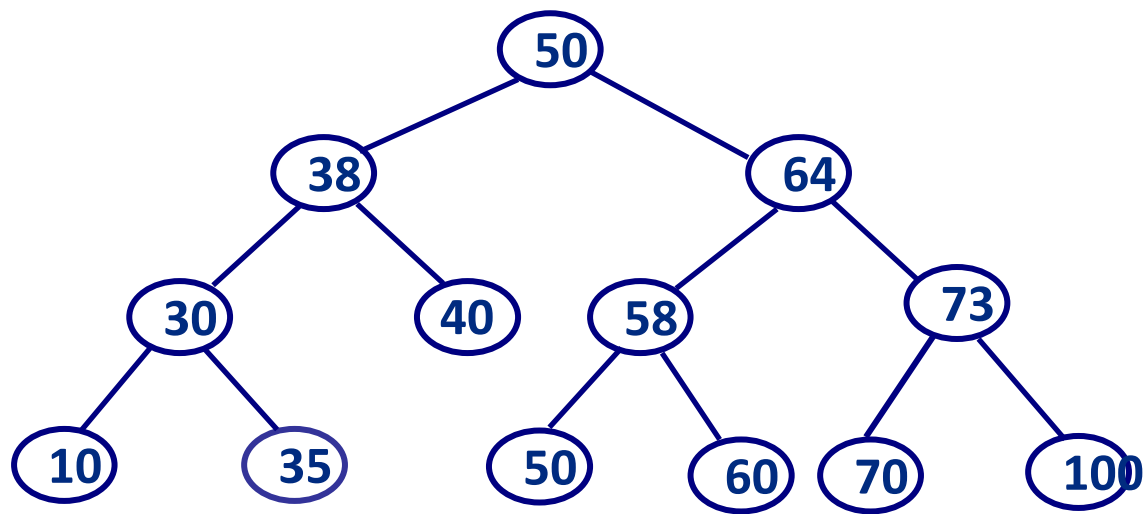


7. 二叉查找树（二叉搜索树、二叉排序树）

- ◆ 二叉查找树(Binary Search Tree, BST): 二叉查找树或者为空二叉树, 或者为具有以下性质的二叉树:
 - ✓ 若根结点的左子树不空, 则左子树上所有结点的值都小于根结点的值
 - ✓ 若根结点的右子树不空, 则右子树上所有结点的值都大于或等于根结点的值
 - ✓ 每一棵子树分别也是二叉查找树 (或称为二叉排序树)



一颗二叉查找树



中序序列

10, 30, 35, 38, 40, 50, 50, 58, 60, 64, 70, 73, 100



二叉查找树的建立（逐点插入法）

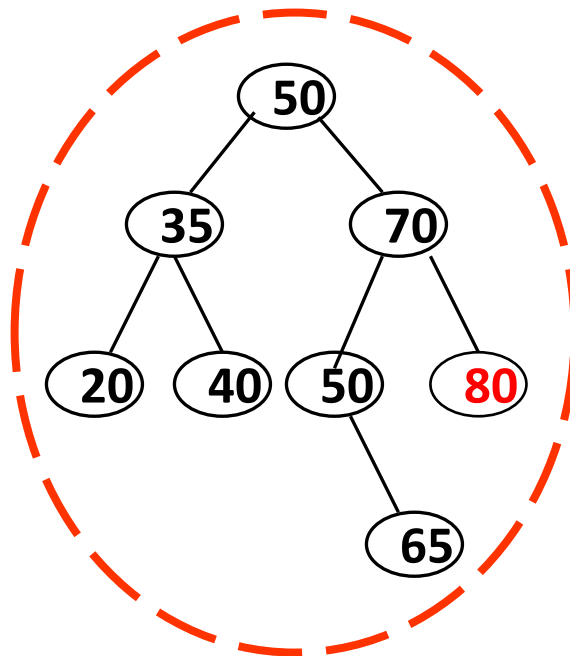
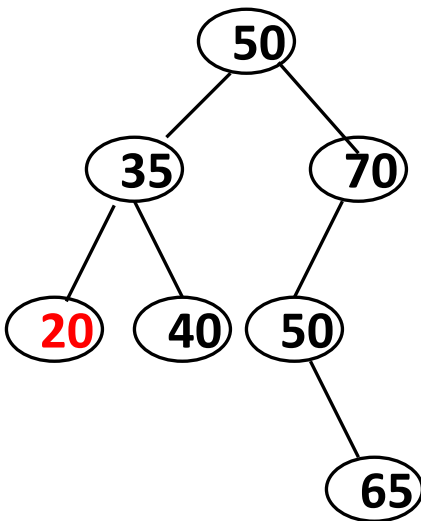
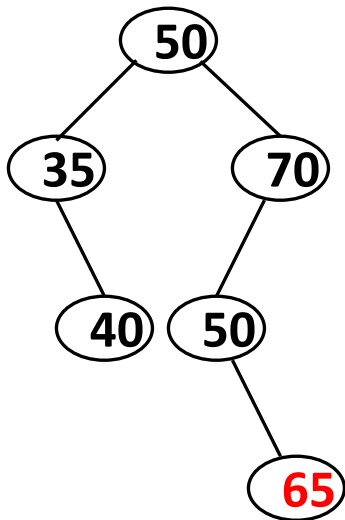
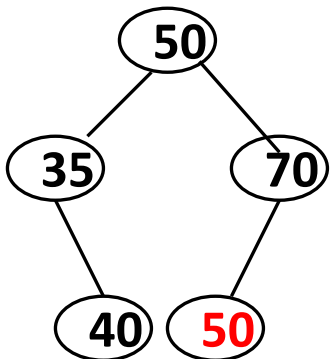
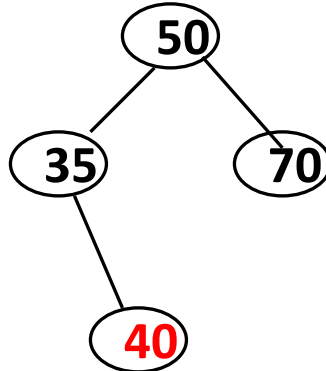
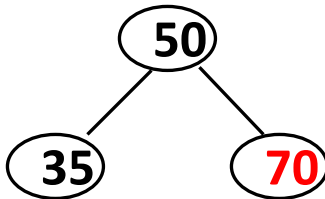
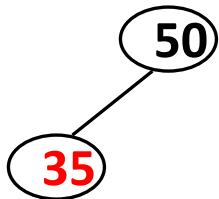
- ◆ 设 $K = (k_1, k_2, k_3, \dots, k_n)$ 为具有 n 个数据元素的序列，从序列的第一个元素开始，依次取序列中的元素，每取一个元素 k_i ，按照下述原则将 k_i 插入到二叉树中
 - ✓ 1. 若二叉树为空，则 k_i 作为该二叉树的根结点
 - ✓ 2. 若二叉树非空，则将 k_i 与该二叉树的根结点的值进行比较，若 k_i 小于根结点的值，则将 k_i 插入到根结点的左子树中；否则，将 k_i 插入到根结点的右子树中
 - ✓ 3. 将 k_i 插入到左子树或者右子树中仍然遵循上述原则（递归）



示例

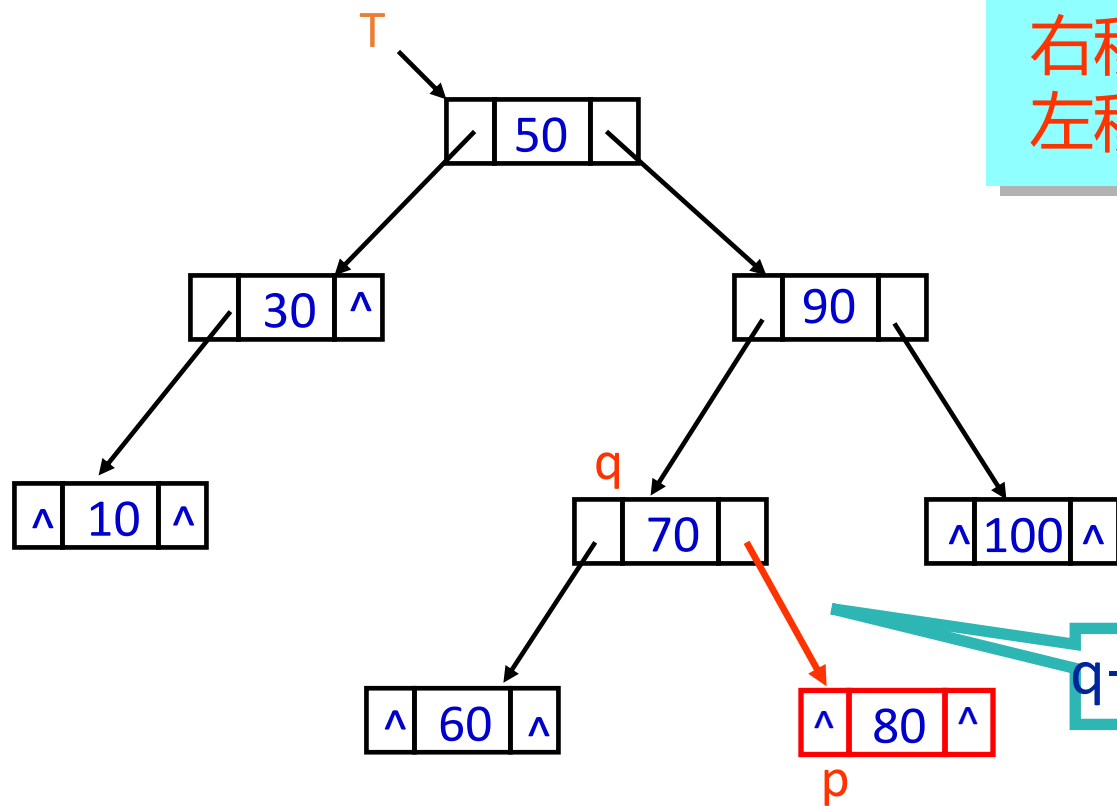
$K = (\underline{50}, \underline{35}, \underline{70}, \underline{40}, \underline{50}, \underline{65}, \underline{20}, \underline{80})$

空





二叉查找树中如何插入一个元素



右移 $q=q \rightarrow rchild;$
左移 $q=q \rightarrow lchild;$

插入

item=80

$q \rightarrow rchild = p;$

二叉树采用二叉链式存储结构



将一个数据元素item插入到根指针为root的二叉排序树中

```
BTNodeptr insertBST(BTNodeptr p, Datatype item)
{
    if (p == NULL)
    {
        p = (BTNodeptr)malloc(sizeof(BTNode));
        p->data = item;
        p->lchild = p->rchild = NULL;
    }
    else if (item < p->data)
        p->lchild = insertBST(p->lchild, item);
    else if (item > p->data)
        p->rchild = insertBST(p->rchild, item);
    else
        do-something; //树中存在该元素
    return p;
}
```

```
#include <stdio.h>
typedef int Datatype;
struct node{
    Datatype data;
    struct node *lchild, *rchild;
};
typedef struct node BTNode, *BTNodeptr;
BTNodeptr insertBST(BTNodeptr p, Datatype item);
int main(){
    int i, item;
    BTNodeptr root = NULL;
    for (i = 0; i < 10; i++)
    { //构造一个有10个元素的BST树
        scanf("%d", &item);
        root = insertBST(root, item);
    }
    return 0;
}
```



算法实现：非递归算法

```
BTNodeptr Root = NULL; // Root是一个全局变量
void insertBST(Typedata item){
    BTNodeptr p, q;
    p = (BTNodeptr)malloc(sizeof(BTNode));
    p->data = item;
    p->lchild = NULL;
    p->rchild = NULL;
    if (Root == NULL)
        Root = p;
    else
    {
        q = Root;
```

```
while (1){
    // 比较值的大小
    // 小于向左，大于向右
    if (item < q->data) {
        if (q->lchild == NULL) {
            q->lchild = p;
            break;
        }
        else q = q->lchild;
    }
    else if (item > q->data) {
        if (q->rchild == NULL) {
            q->rchild = p;
            break;
        }
        else
            q = q->rchild;
    }
    else
    { // do-something
    }
}
}
```



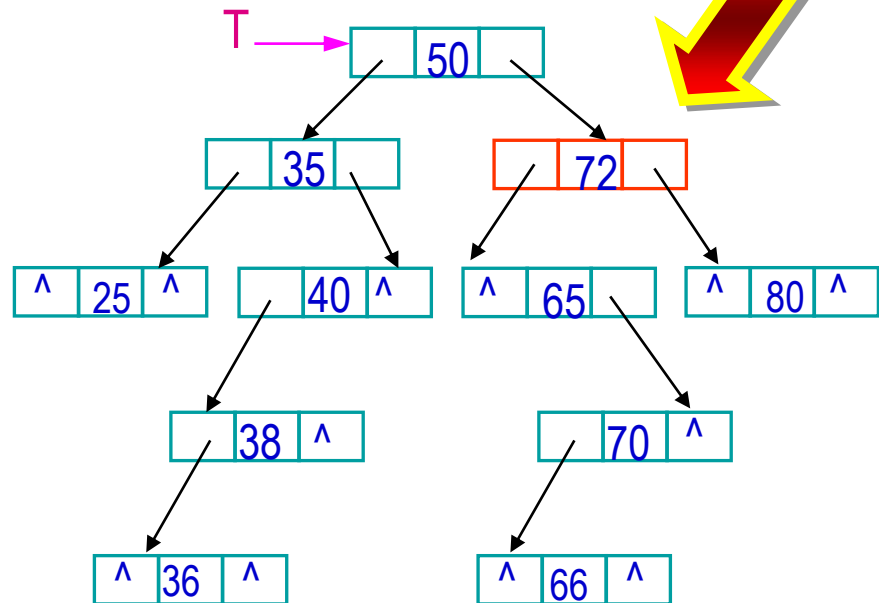
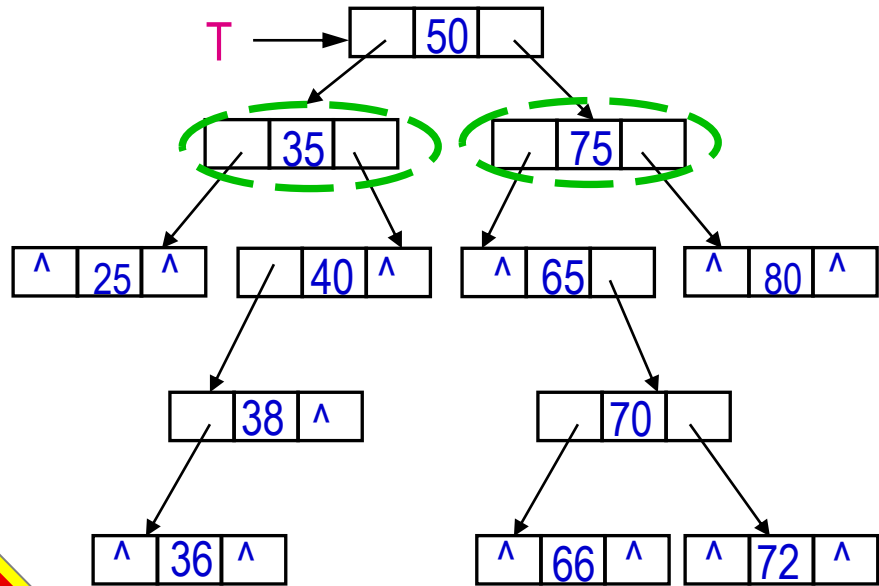
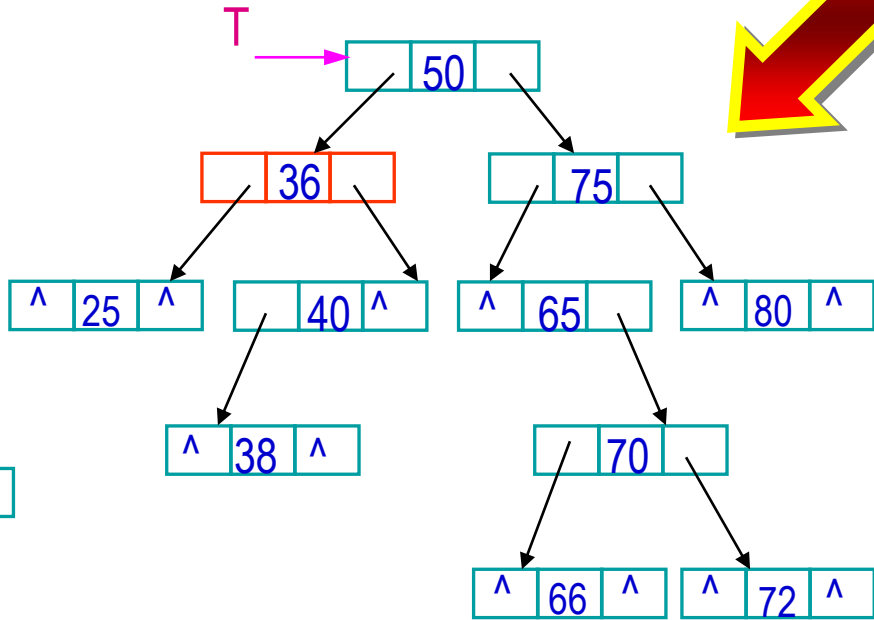
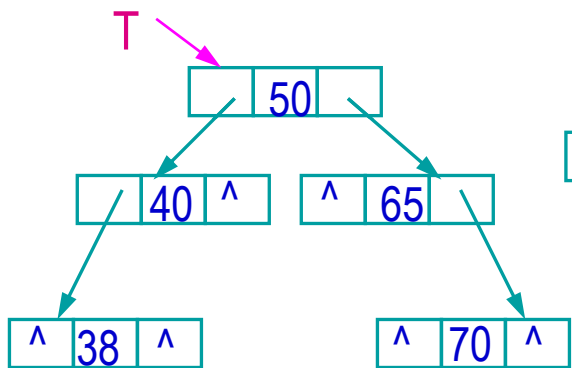
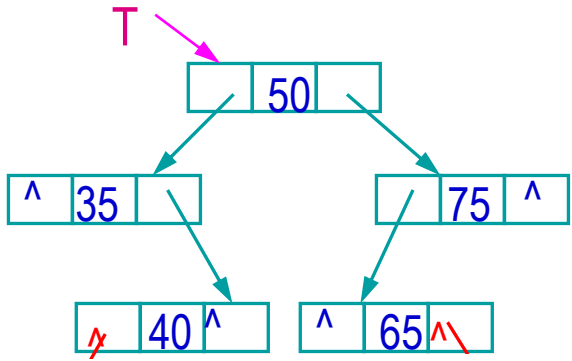
7.3 二叉查找树的删除

◆原则

- ✓1. 被删除结点为叶结点，则直接删除
- ✓2. 被删除结点无左子树，则用右子树的根结点取代被删除结点
- ✓3. 被删除结点无右子树，则用左子树的根结点取代被删除结点
- ✓4. 被删除结点的左、右子树都存在，则用被删除结点的右子树中值最小的结点（或被删除结点的左子树中值最大的结点）取代被删除结点



示例





7.4 二叉查找树的查找

◆查找过程

- ✓若二叉查找树为空，则查找失败，查找结束
- ✓若二叉查找树非空，则将被查找元素与二叉查找树的根结点的值进行比较
(递归)
 - 若等于根结点的值，则查找成功，结束
 - 若小于根结点的值,则到根结点的左子树中重复上述查找过程
 - 若大于根结点的值,则到根结点的右子树中重复上述查找过程
- ✓直到查找成功或者失败



算法实现：非递归算法

```
BTNodeptr searchBST(BTNodeptr t, Datatype key){
    BTNodeptr p = t;
    while (p != NULL)
    {
        if (key == p->data)
            return p; // 查找成功
        if (key > p->data)
            p = p->rchild; // 将p移到右子树的根结点
        else
            p = p->lchild; // 将p移到左子树的根结点
    }
    return NULL; // 查找失败
}
```



算法实现：递归算法

```
BTNodeptr searchBST(BTNodeptr t, Datatype key)
{
    if (t != NULL)
    {
        if (key == t->data)
            return t;          // 查找成功
        if (key > t->data) // 查找T的右子树
            return searchBST(t->rchild, key);
        else
            return searchBST(t->lchild, key);
    } // 查找T的左子树
    else
        return NULL; // 查找失败
}
```



平均查找长度

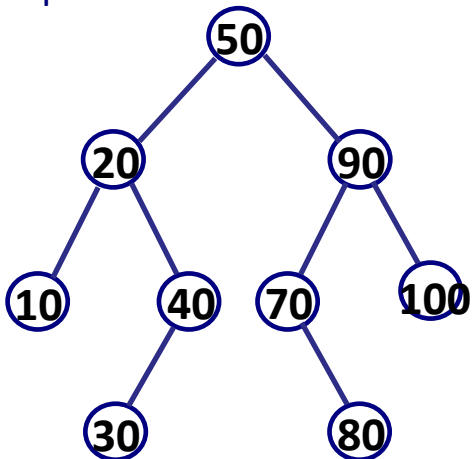
- ◆ 平均查找长度 (ASL) 确定一个元素在树中位置所需要进行的元素间的比较次数的期望值 (平均值)

$$ASL = \sum_{i=1}^n p_i c_i$$

n 二叉树中结点的总数

p_i 表示查找第 i 个元素的概率;

c_i 表示查找第 i 个元素需要进行的元素之间的比较次数。



$$\begin{aligned} ASL &= (1 \times 1 + 2 \times 2 + 3 \times 4 + 4 \times 2) / 9 \\ &= (1 + 4 + 12 + 8) / 9 \\ &= 25 / 9 \end{aligned}$$



查找效率与树的深度相关

如果被插入的元素序列是随机序列，或者序列的长度较小，采用“逐点插入法”建立二叉查找树可以接受。如果建立的二叉查找树中出现结点子树的深度之差较大时（即产生不平衡），就有必要采用其他方法建立二叉查找树，即建立所谓 **平衡二叉树**。

查找时间

$O(\log_2 n)$

比较理想的情况

$O(n)$

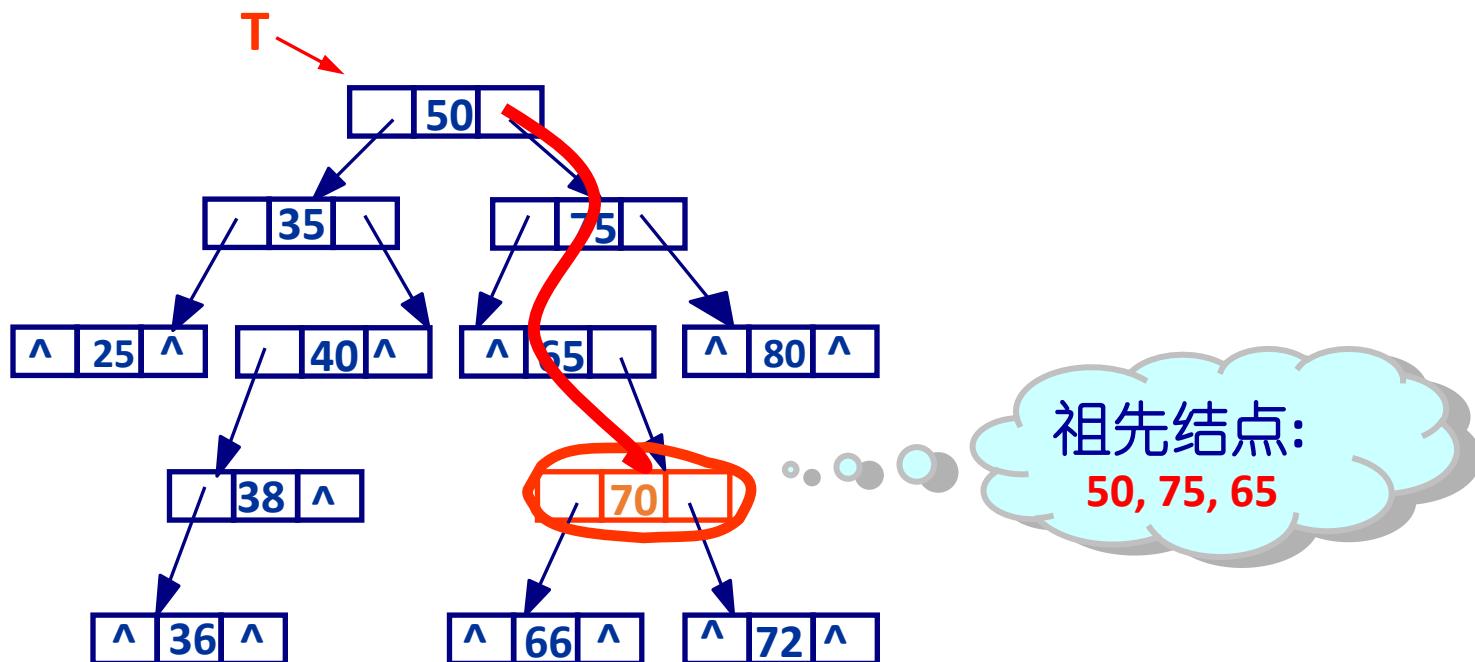
当出现“退化二叉树”时



二叉查找树的应用：祖先结点

◆ 已知二叉查找树采用二叉链表存储结构，根结点地址为T，请写一非递归算法，打印数据信息为item的结点的所有祖先结点（设该结点存在祖先结点）

✓ 祖先结点：从根结点到该结点的所有分支上经过的结点





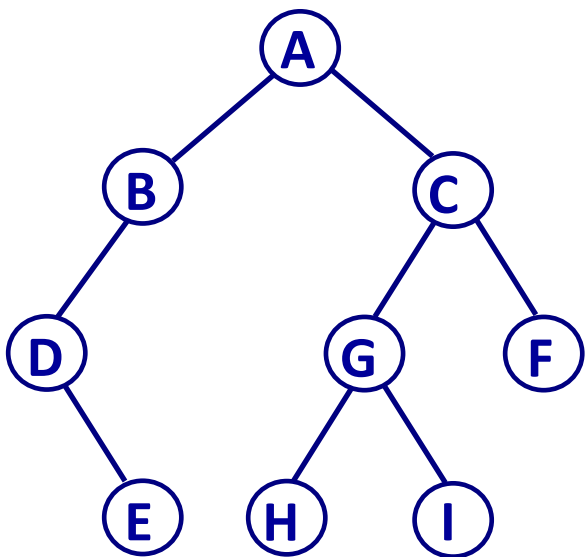
寻找祖先结点：非递归算法

```
void searchBST(BTNodeptr t, DataType item){
    BTNodeptr p = t;
    while (p != NULL) {
        if (item == p->data)
            break; // 查找结束
        if (item > p->data) {
            printf("%d ", p->data);
            p = p->rchild; // 将p移到右子树的根结点
        }
        else {
            printf("%d ", p->data);
            p = p->lchild; // 将p移到左子树的根结点
        }
    }
}
```



如何获得普通二叉树的祖先结点

前序遍历算法



前序序列: **A B D E C G H I F**

```
void preorder(BTNodeptr t, DataType item)
{
    if (t != NULL)
    {
        push(t);
        if (item == t->data)
            //弹出栈中所有元素;
        preorder(t->lchild);
        preorder(t->rchild);
        pop();
    }
}
```



关于二叉查找树

对于输入序列无序、且需要频繁进行查找、插入和删除操作的动态表（如词频统计中的单词表），如何选取数据结构和算法即能兼顾插入和删除效率，又能较高效率地实现查找？

二叉查找树是很好的选择

有序顺序表(数组)

- ✓ 有序顺序存储的数据能够采用如折半查找等算法，**查找效率高**
- ✓ 有序顺序存储数据由于据需要移动数据，**插入和删除操作效率低**
- ✓ 顺序存储需要事先分配空间，**空间利用率低，同时存在溢出风险**

有序链表

- ✓ 有序链表存储，数据**插入和删除操作效率高**
- ✓ 链式存储能够动态分配空间，**空间利用率高**
- ✓ 链表存储的数据查找必须从链头开始，数据**查找效率低**

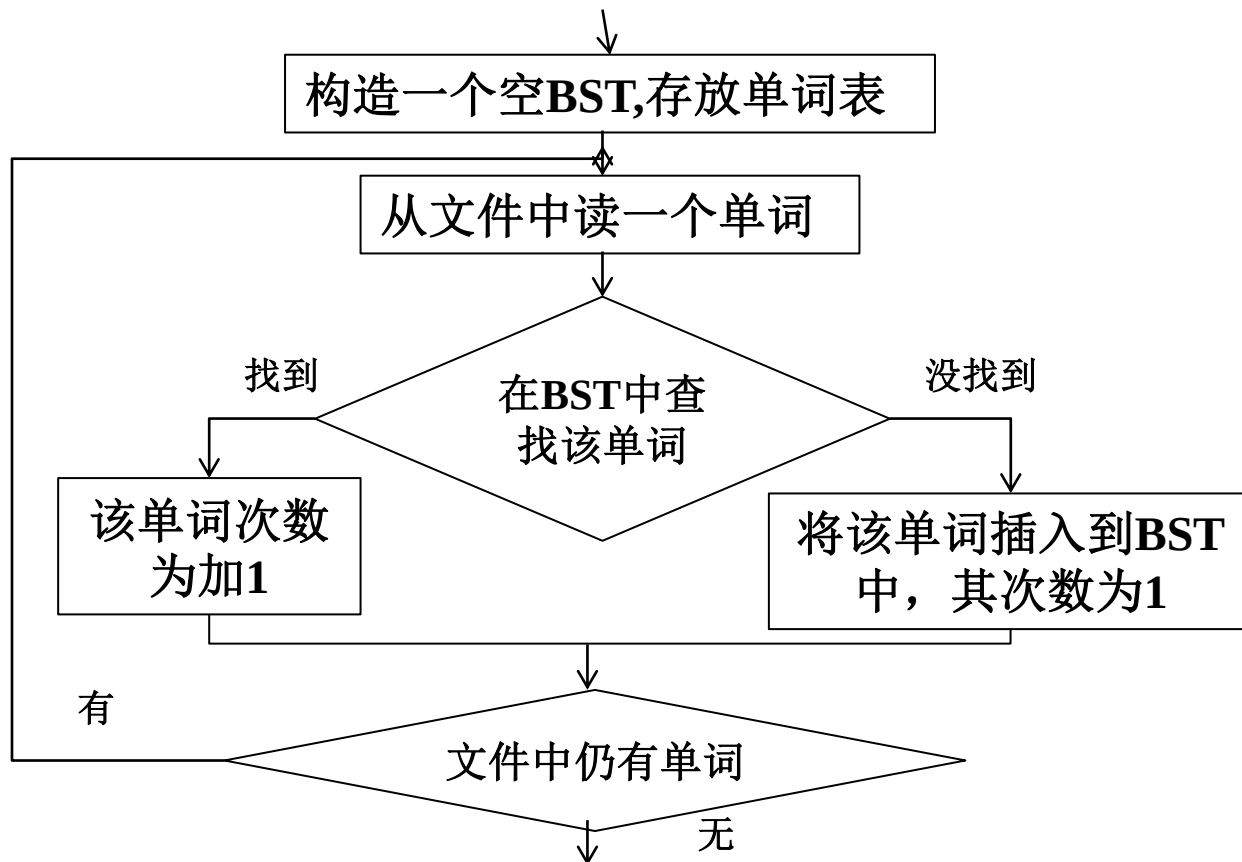
二叉查找树

- ✓ 数据**插入和删除操作效率高**
- ✓ 能够动态分配空间，**空间利用率高**
- ✓ 数据**查找效率较高**
(理想情况下查找效率为 $O(\log_2 n)$)



问题：词频统计 - 二叉查找树

- ◆问题：编写程序统计一个文件中每个单词的出现次数（词频统计），并按字典序输出每个单词及出现次数
- ◆算法分析：本问题算法很简单，基本上只有查找和插入操作





词频统计-二叉查找树的代码实现

```
#include <stdio.h>
#include <string.h>
#define MAXWORD 32
struct tnode
{
    char word[MAXWORD];
    int count;
    struct tnode *left, *right;
}; // BST, 单词树结构
//从文件中, 读入一个单词
int getWord(FILE *bfp, char *w);
//查找单词, 如果不存在, 则插入, 存在则增加计数
struct tnode *treeWords(struct tnode *p,
                        char *w);

//打印BST中的单词
void treePrint(struct tnode *p);
```

```
int main()
{
    char filename[32], word[MAXWORD];
    FILE *bfp;
    //BST树根节点指针
    struct tnode *root = NULL;

    scanf("%s", filename);
    if ((bfp = fopen(filename, "r")) == NULL){
        //打开一个文件
        printf("%s can't open!\n", filename);
        return -1;
    }
    while (getWord(bfp, word) != EOF)
        //从文件中读入一个单词
        root = treeWords(root, word);
    treePrint(root); //遍历输出单词树
    return 0;
}
```



词频统计-二叉查找树的代码实现 (续)

//在P结点为根的树中插入w

```
struct tnode *treeWords(struct tnode *p, char *w){
    int cond;
    if (p == NULL){//p为null, 新建结点作为当前子树的根
        p = (struct tnode *)malloc(sizeof(struct tnode));
        strcpy(p->word, w); p->count = 1;
        p->left = p->right = NULL;    }
    else if ((cond = strcmp(w, p->word)) == 0)
        p->count++;//存在相同的单词, 计数加1
    else if (cond < 0) //单词比根节点小, 插入左子树
        p->left = treeWords(p->left, w);
    else //单词比根节点大, 插入右子树
        p->right = treeWords(p->right, w);
    return p;
}
```

```
void treePrint(struct tnode *p){//中序遍历打印树中的结点
    if (p != NULL)    {
        treePrint(p->left);
        printf("%s  %4d\n", p->word, p->count);
        treePrint(p->right);
    }
}
```

在本程序中:
在理想情况下 (构成的是平衡二叉树), 查找算法复杂度为 $O(\log_2 n)$
插入算法复杂度为 $O(1)$

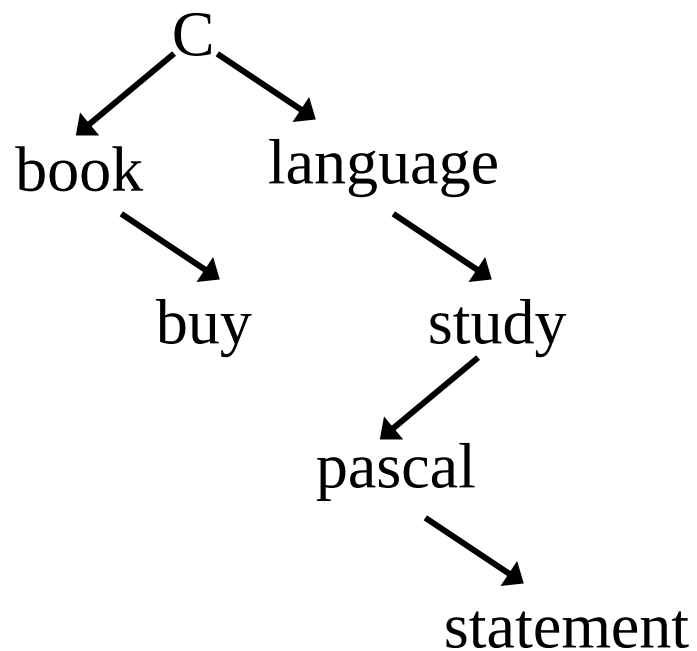


问题4.2：测试

若输入：

C language book study pascal statement buy

则生成如下树：





关于二叉查找树*

◆关于二叉查找树

- ✓从二叉查找树的定义与构造方式不难看到，其查找某个元素的效率要比采用顺序比较（如链表）的效率要高得多
- ✓二叉查找树特别适合于数据量大、且无序的数据，如单词词频统计（单词索引）等

◆二叉查找树是重要的一种数据结构，在实际应用中经常使用，请同学们自学下面内容：

- ✓二叉查找树的结点的插入、删除操作
- ✓平衡二叉树（AVL）



平衡二叉树 (AVL)

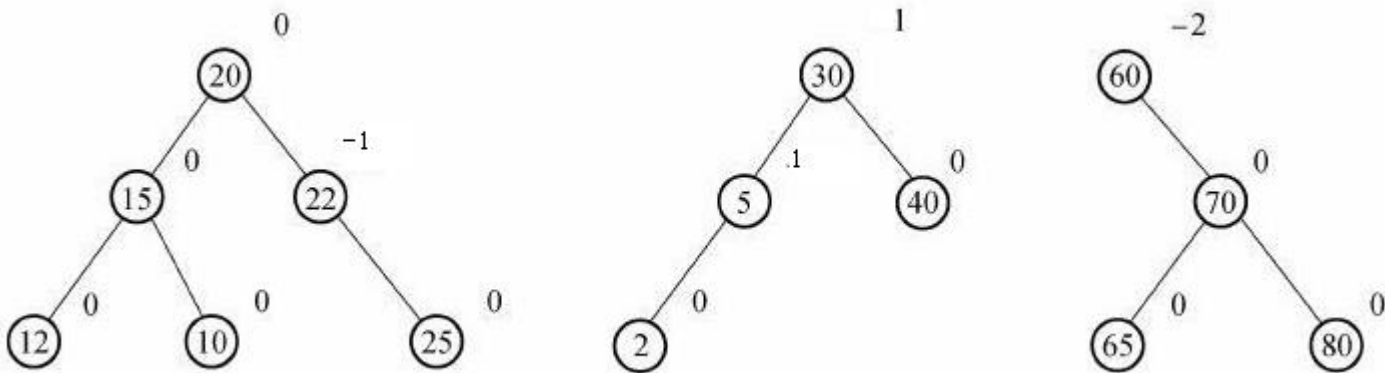
4, 5, 6, 7, 8

7, 5, 4, 6, 8

二叉查找树的缺陷—树的形态无法预料、随意性大。得到的可能是一个不平衡的树，即树的深度差很大。丧失了利用二叉树组织数据带来的好处。

平衡二叉树又称AVL树。它或者是一棵空树,或者是具有下列性质的二叉树:

它的左子树和右子树都是平衡二叉树, 且左子树和右子树的深度之差的绝对值不超过1。若将二叉树的平衡因子定义为该结点左子树深度减去右子树深度, 则平衡二叉树上所有结点的平衡因子只可能是-1、0和1





提纲：树和二叉树

- ◆4.1 树的基本概念
- ◆4.2 树的存储结构
- ◆4.3 二叉树
- ◆4.4 二叉树的存储结构
- ◆4.5 二叉树的遍历
- ◆*4.6 线索二叉树
- ◆4.7 二叉查找树
- ◆*4.8 堆和表达式树
- ◆4.9 哈夫曼树



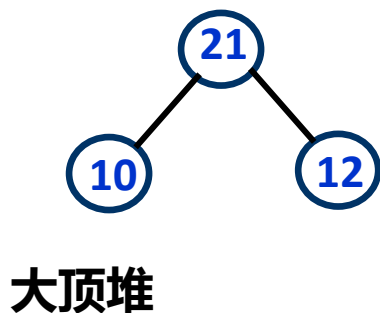
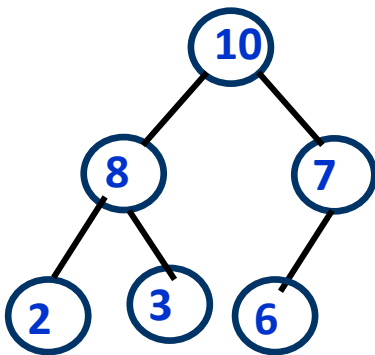
8. 堆 (heap)*

- ◆堆是一种特殊类型的二叉树，具有以下两个性质：
 - ✓ (1) 每个节点的值大于（或小于）等于其每个子节点的值；
 - ✓ (2) 该树完全平衡，其最后一层的叶子都处于最左侧的位置。
- ◆满足上面两个性质定义的是**大顶堆** (max heap) （或**小顶堆** min heap)
 - ✓这意味着大顶堆的根节点包含了最大的元素，小顶堆的根节点包含了最小的元素



堆结构

- ◆堆结构的最大好处是元素查找、插入和删除效率高 ($O(\log_2(n))$)
- ◆堆的主要应用：
 - ✓ (1) 可用来实现优先队列 (Priority Queue)
 - ✓ (2) 用来实现一种高效排序算法-堆排序 (Heap Sort) , 在排序一讲中详细介绍



由于堆是一个完全平衡树，其可以通过数组来实现：

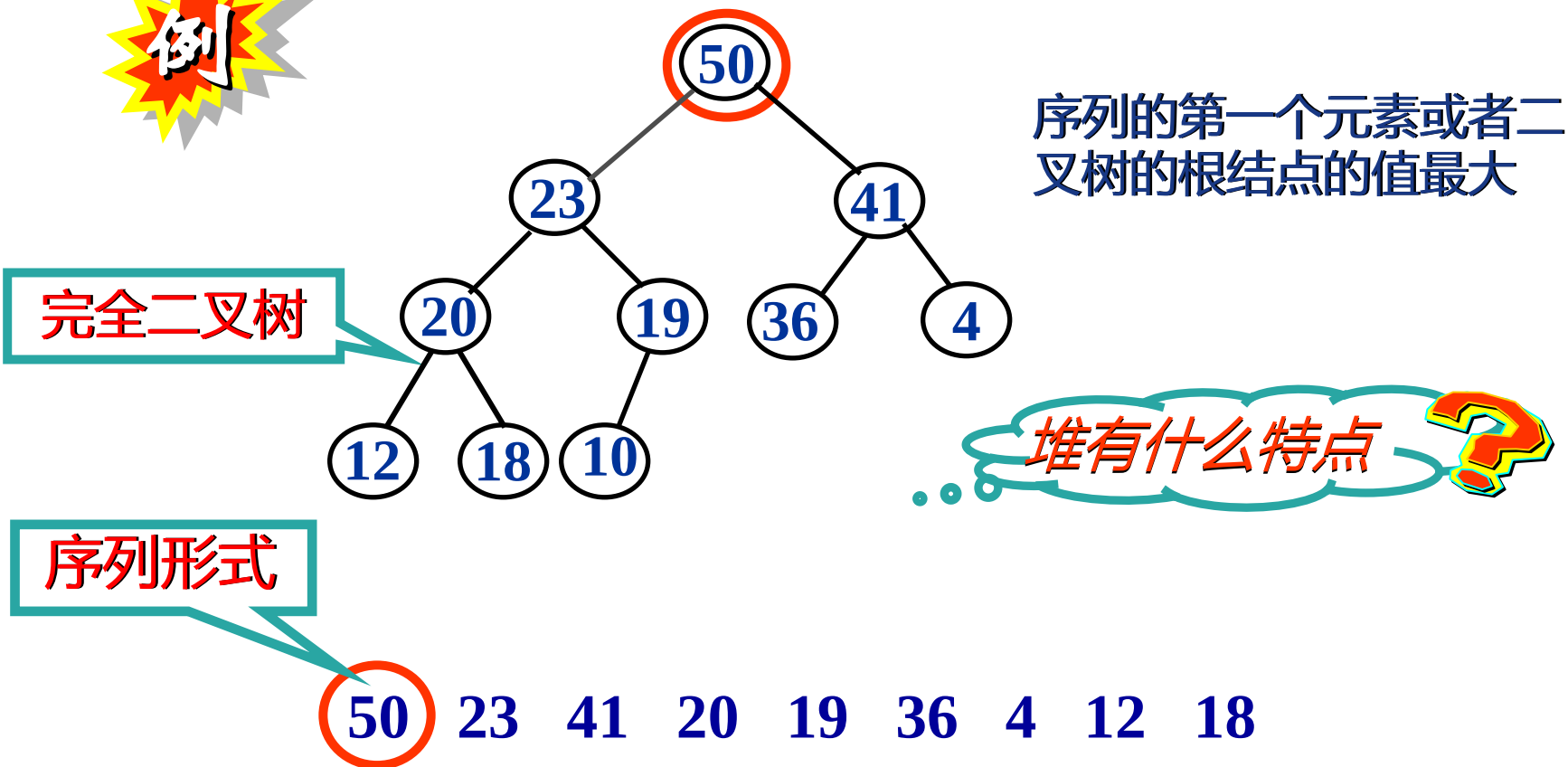
$$\text{heap}[i] \geq \text{heap}[2*i]$$

$$\text{heap}[i] \geq \text{heap}[2*i+1]$$



堆结构的特点

- ◆堆是一棵完全二叉树，二叉树中任何一个分支结点的值都大于或者等于它的孩子结点的值，并且每一棵子树也满足堆的特性





8.2 堆的基本操作*

堆的基本操作: insert(插入)和delete(删除)

插入算法

heapInsert(e)

将e放在堆的末尾;

while e 不是根 && $e > parent(e)$

e 与其父节点交换

删除算法

heapDelete() //取堆顶 (树根) 元素

从根节点提取元素;

将最后一个叶节点中的元素放到要删除的元素位置;

删除最后一个叶节点;

//根的两个子树都是堆

p = 根节点;

while p 不是叶节点 && $p <$ 它的任何子节点

p 与其较大的子节点交换

删除指获取堆顶元素, 并从堆中删除。



8.3 堆的构造*

堆的构造有两种方法：自顶向下(John Williams提出)和自底向上(Robert Floyd提出)

自顶向下

从空堆开始，按顺序向堆中添加（用`headinsert`函数）元素

自底向上

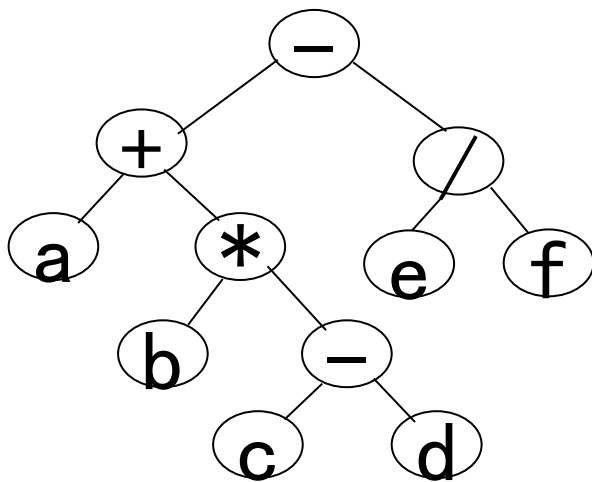
首先从底层开始构造较小的堆，然后再重复构造较大的堆
(算法将在堆排序一节中介绍)



8a 表达式树 (expression tree) *

◆ 表达式树是一种特殊类型的树，其叶结点是操作数 (operand)，而其它结点为操作符 (operator)

- ✓ (1) 由于操作符一般都是双目的，通常情况下该树是一棵二叉树
- ✓ (2) 对于单目操作符 (如++)，其只有一个子结点



对于表达式: $a + b * (c - d) - e / f$ ，其对应的表达树如左所示

中序遍历: $a + b * c - d - e / f$ (中缀表达式)

后序遍历: $a b c d - * + e f / -$ (后缀表达式)

表达式树的最大好处是表达式没有括号，计算时也不用考虑运算符优先级。

主要应用: (1) 编译器用来处理程序中的表达式



表达式树的构造*

- ◆ 表达式树是这样一种树，非叶节点为操作符，叶节点为操作数，对其进行遍历可计算表达式的值
- ◆ 由后缀表达式生成表达式树的方法如下：
 - ✓ 1. 从左至右从后缀表达式中读入一个符号：
 - 如果是操作数，则建立一个单节点树并将指向该节点的指针推入栈中；（栈中元素为树节点的指针）
 - 如果是运算符，就从栈中弹出指向两棵树T1和T2的指针（T1先弹出）并形成一棵新树，树根为该运算符，它的左、右子树分别指向T2和T1，然后将新树的指针压入栈中
 - ✓ 2. 重复步骤1，直到后缀表达式处理完



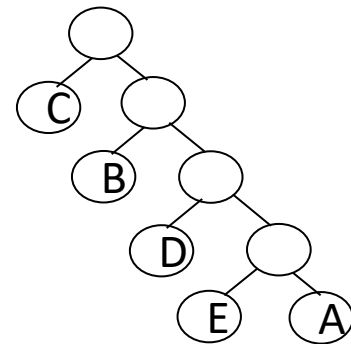
提纲：树和二叉树

- ◆4.1 树的基本概念
- ◆4.2 树的存储结构
- ◆4.3 二叉树
- ◆4.4 二叉树的存储结构
- ◆4.5 二叉树的遍历
- ◆*4.6 线索二叉树
- ◆4.7 二叉查找树
- ◆*4.8 堆和表达式树
- ◆4.9 哈夫曼树

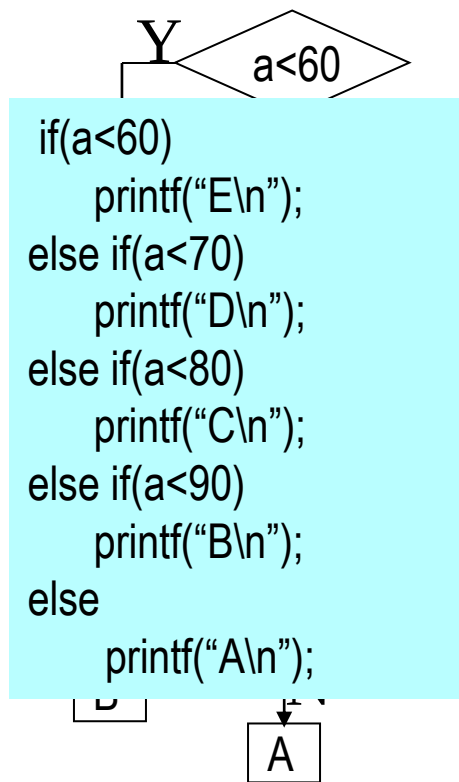


考察一个场景

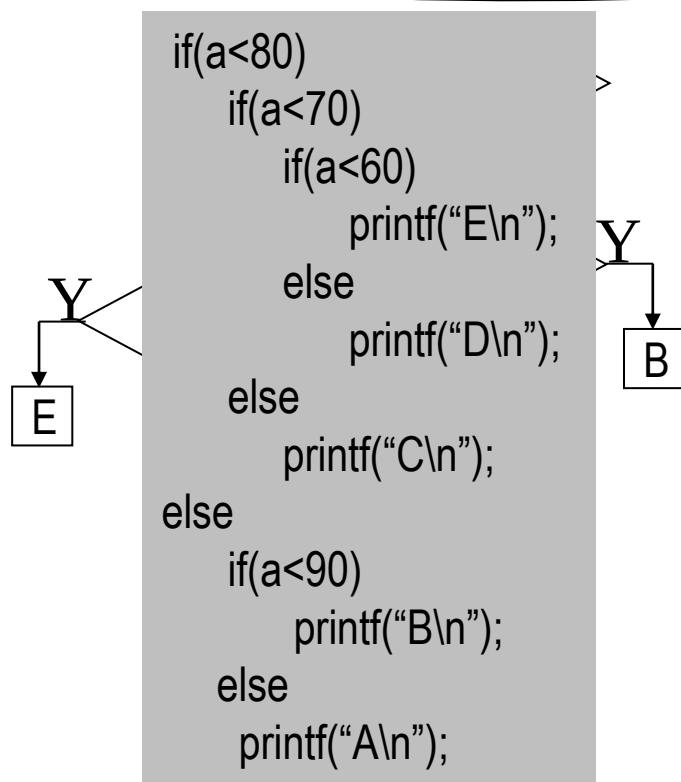
等级	E	D	C	B	A
分数段	0~59	60~69	70~79	80~89	90~100
比例	0.05	0.15	0.40	0.30	0.10



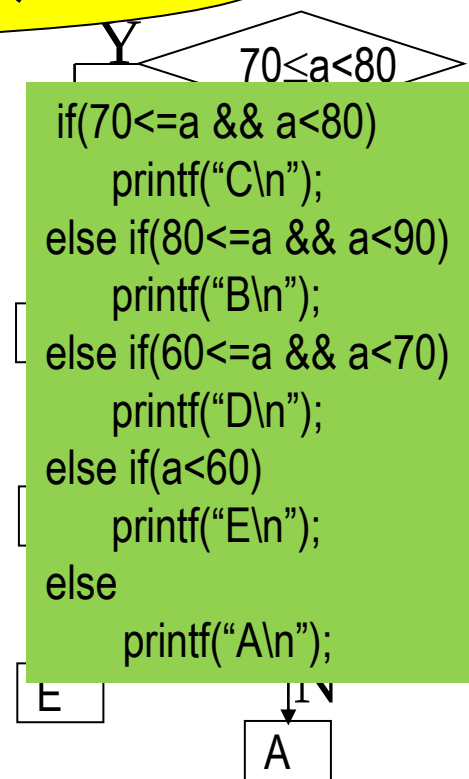
假设输入10000个同学的成绩



比较31500次



比较22000次



比较20500次



9. 哈夫曼树及其应用

1. 树的带权路径长度

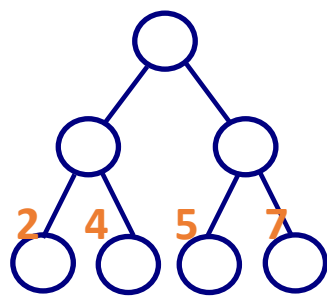
若给具有 m 个叶结点的二叉树的每个叶结点赋予一个权值，则该二叉树的带权路径长度定义为

$$WPL = \sum_{i=1}^m w_i l_i$$

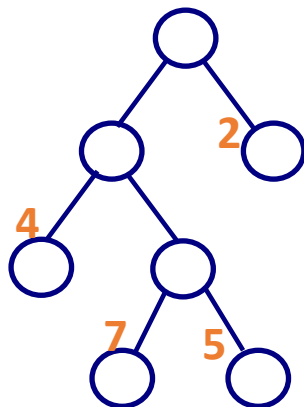
其中， w_i 为第 i 个叶结点被赋予的权值， l_i 为第 i 个叶结点的路径长度

例

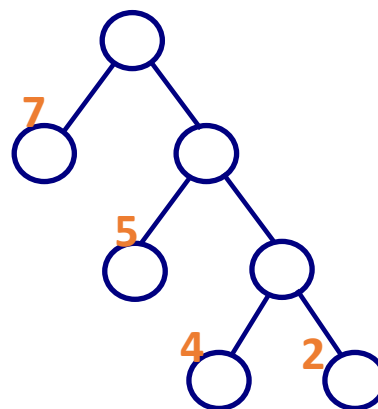
对于一组权值{ 7, 5, 2, 4 }，可以有如下多种不同的带权二叉树



WPL=36



WPL=46



WPL=35

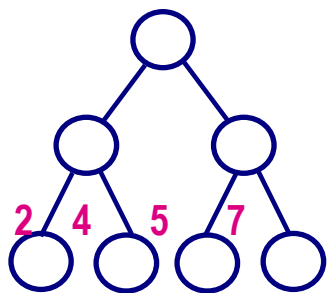


哈夫曼树的定义

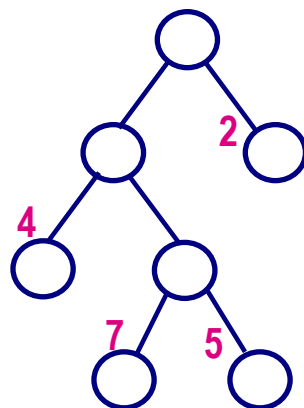
◆ 给定一组权值，构造出的具有**最小带权路径长度**的二叉树称为哈夫曼树

◆ 哈夫曼树特点

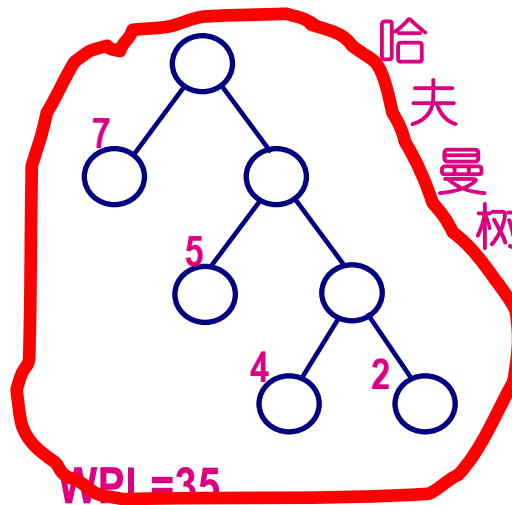
- ✓ 权值越大的叶节点离根结点越近，权值越小的叶结点离根结点越远
- ✓ 无度为1的结点
- ✓ 不唯一



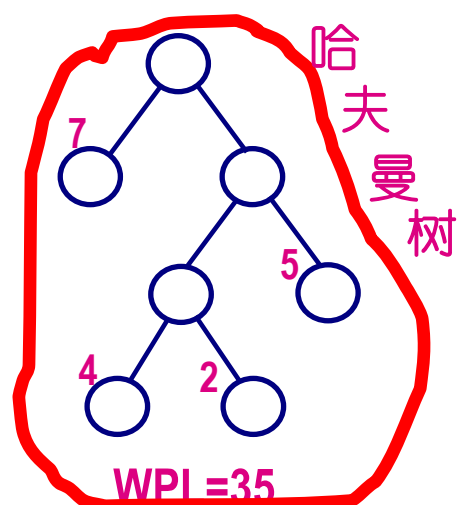
WPL=36



WPL=46



WPL=35



WPL=35

一组权值

{ 7, 5, 2, 4 }



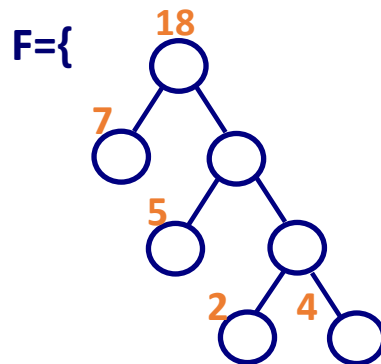
9.2 哈夫曼树的构造

◆核心思想

- ✓ 1. 对于给定的权值 $W = \{w_1, w_2, \dots, w_m\}$, 构造出树林 $F = \{T_1, T_2, \dots, T_m\}$, 其中, $T_i (1 \leq i \leq m)$ 为左、右子树为空, 且根结点(叶结点)的权值为 w_i 的二叉树
- ✓ 2. 将 F 中根结点权值最小的两棵二叉树合并成为一棵新的二叉树, 即把这两棵二叉树分别作为新的二叉树的左、右子树, 并令新的二叉树的根结点权值为这两棵二叉树的根结点的权值之和, 将新的二叉树加入 F 的同时从 F 中删除这两棵二叉树
- ✓ 3. 重复步骤2, 直到 F 中只有一棵二叉树

例

$W = \{7, 5, 2, 4\}$



哈夫曼树



基于哈夫曼编码的数据压缩

在信息和网络时代，数据压缩解压是一个非常重要的技术，现有数据压缩和解压技术很多是基于Huffman研究之上发展而来



9.3 一种熵编码方式

例

要传送的文字

'ABACCDA'

由二进制代码组成的电文

00010010101100

若假设 A=00 B=01 C=10 D=11

(编码等长)

若假设 A=0 B=00 C=1 D=01

(编码不等长)

要传送的文字

'ABACCDA'

由二进制代码组成的电文

000011010

AAAA ABA BB

优点: 电文中出现频率大的字符编码短, 电文总长减少。

缺点: 编码表示不惟一, 致使电文难以翻译。

要求: 任意一个字符的编码不能是另一个字符的编码的前缀。

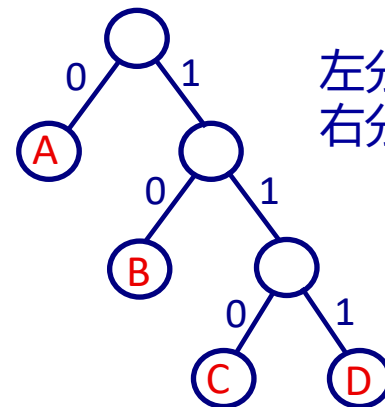
前缀编码



利用二叉树设计二进制前缀编码

从根结点到叶结点的路径上分支字符组成的字符串作为该叶结点字符的编码

A: 0 B: 10 C: 110 D: 111



左分支表示字符0
右分支表示字符1

如何得到使电文
总长最短的编码 ?



哈夫曼编码

设 电文中包含n种字符;
 w_i 为每种字符在电文中出现的次数;
 l_i 为相应的编码长度。

则 电文总长度为 $\sum_{i=1}^n w_i l_i$ (二叉树) w_i 为叶结点的权值;
 l_i 为根结点到叶结点的路径长度

二叉树的带权路径长度



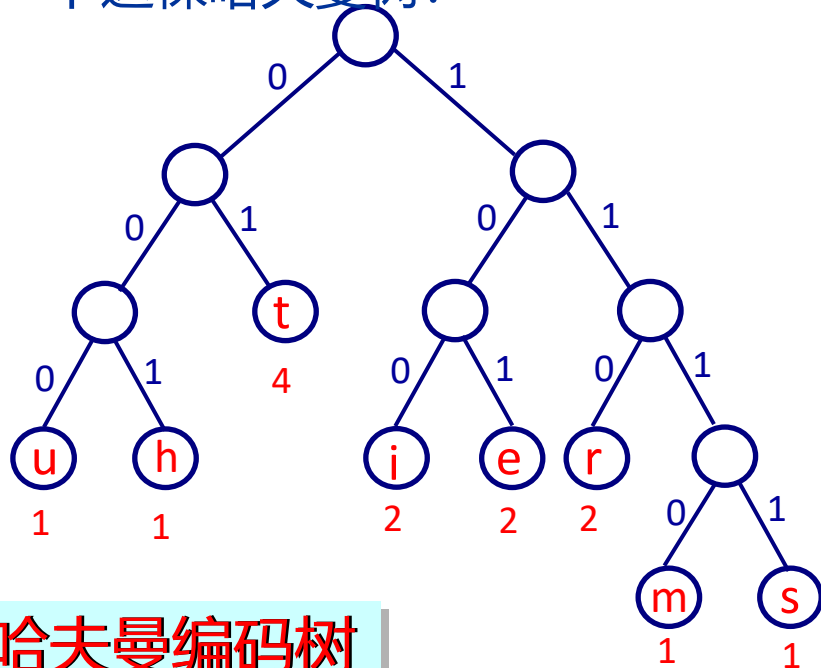
示例：哈夫曼编码

电文: time tries truth

字符集: { t, i, m, e, r, s, u, h }

字符在电文中出现的次数: { 4, 2, 1, 2, 2, 1, 1, 1 }

大家来构造一下这棵哈夫曼树?



哈夫曼编码树

该电文以本Huffman编码传输时传输长度为: 40 位 (5字节)
而该电文以ASCII编码传输时传输长度为: 14 字节

哈夫曼编码

t: 01
i: 100
m: 1110
e: 101
r: 110
s: 1111
u: 000
h: 001

01 100 1110 101 01 110 100 101 1111 01 110 000 01 001



问题4.4-5/6/7：文件压缩和解压-Huffman编码

- ◆【问题描述】编写程序实现利用Huffman编码对一个文件进行压缩和解压工具hzip.exe
- ◆ Huffman压缩文件原理如下：
 - ✓ 1.对正文文件中字符按出现次数（即频率）进行统计
 - ✓ 2.依据字符频率生成相应的Huffman树（未出现的字符不生成）
 - ✓ 3.依据Huffman树生成相应字符的Huffman编码
 - ✓ 4.依据字符Huffman编码压缩文件（即按照Huffman编码依次输出源文件字符）
- ◆说明：
 - ✓ 1. 只对文件中出现的字符生成Huffman码
 - ✓ 2. 采用ASCII码值为0的字符作为压缩文件的结束符（即可将其出现次数设为1来参与编码）
 - ✓ 3. 在生成Huffman树时，初始在对字符频率权重进行（由小至大）排序时，频率相同的字符ASCII编码值小的在前；新生成的权重节点插入到有序权重序列中时，出现相同权重时，插入到其后（采用稳定排序）
 - ✓ 4. 遍历Huffman树生成字符Huffman码时，左边为0右边为1
 - ✓ 5. 源文件是文本文件，字符采用ASCII编码，每个字符占8位；而采用Huffman编码后，最后输出时需要使用C语言中的位运算将字符Huffman码依次输出到每个字节中



应用：文件压缩和解压（续）

◆【输入形式】采用命令行参数：

✓ hzip [-u] <filename.xxx>; （注：xxx是文件扩展名）

✓ 示例：

✓ > hzip myfile.txt

➤ 采用Huffman编码方式压缩文件myfile.txt，并将压缩后结果存到文件myfile.hzip中。

✓ > hzip -u myfile.hzip

➤ myfile.hzip必须是用hzip.exe压缩工具生成的压缩文件，解压后文件名为myfile.txt。当命令行有-u参数时，对当前目录下压缩文件<filename.hzip>进行解压，被解压的文件必须以.hzip为扩展名的文件，解压后的文本文件名同压缩文件，但扩展名为.txt

✓ 具有一定的参数错误处理能力，如参数个数不对、参数格式不正确等：

➤ > hzip Usage: hzip.exe [-u] <filename>

➤ > hzip srcfile.txt objfile.hzip Usage: hzip.exe [-u] <filename>

➤ > hzip -h srcfile.txt Usage: hzip.exe [-u] <filename>

➤ > hzip -u myfile.c File extension error!

➤ > hzip myfile.c File extension error!



应用：文件压缩和解压（续）

◆【输出形式】<filename.hzip>文件格式如下

✓压缩文件由2部分组成，第一部分为ASCII码与Huffman码对照码表，第2部分为以Huffman编码形式的压缩后的文件。如下图所示

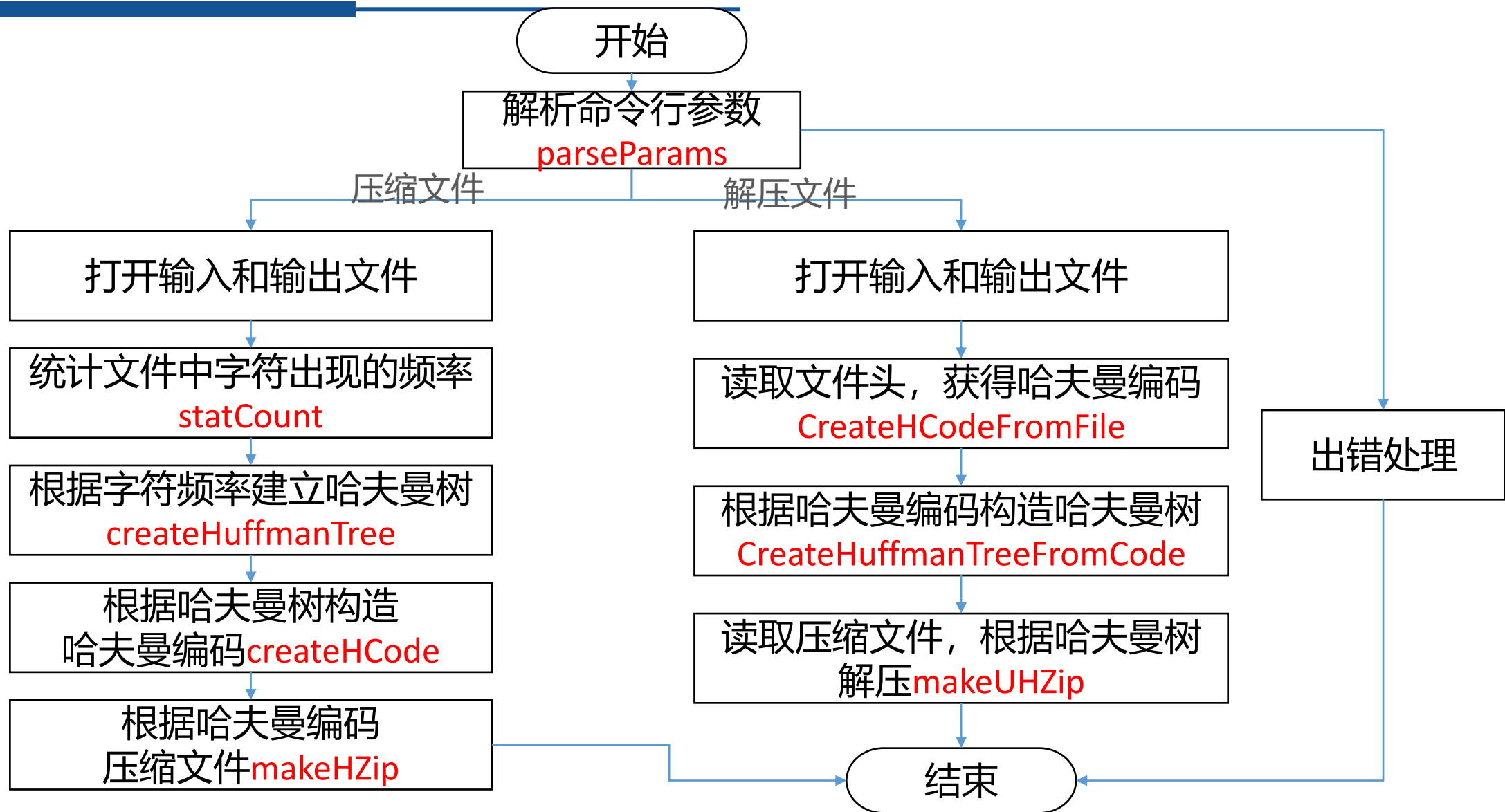
✓文件头结构

- 从文件头开始第1个字节为码表长度，即压缩文件时ASCII字符个数（即字符频率为非0的字符个数，包括文件结束符）
- 从文件头开始第2个字节为相应码表内容，每个码表项依次为字符ASCII码（占一个字节）、第2部分为其对应Huffman码（占不定长位）（为3个以上字节），多余位要补0

码表长度	ASCII	码长	Huffman 码
1 字节	1 字节	1 字节	
	字符 1 编码信息 (3 以上字节)		
			...



问题4.3：问题分析





预备知识：命令行参数

◆C语言中，主函数main还可以带有参数

```
int main( int argc, char *argv[ ])
```

◆其中：

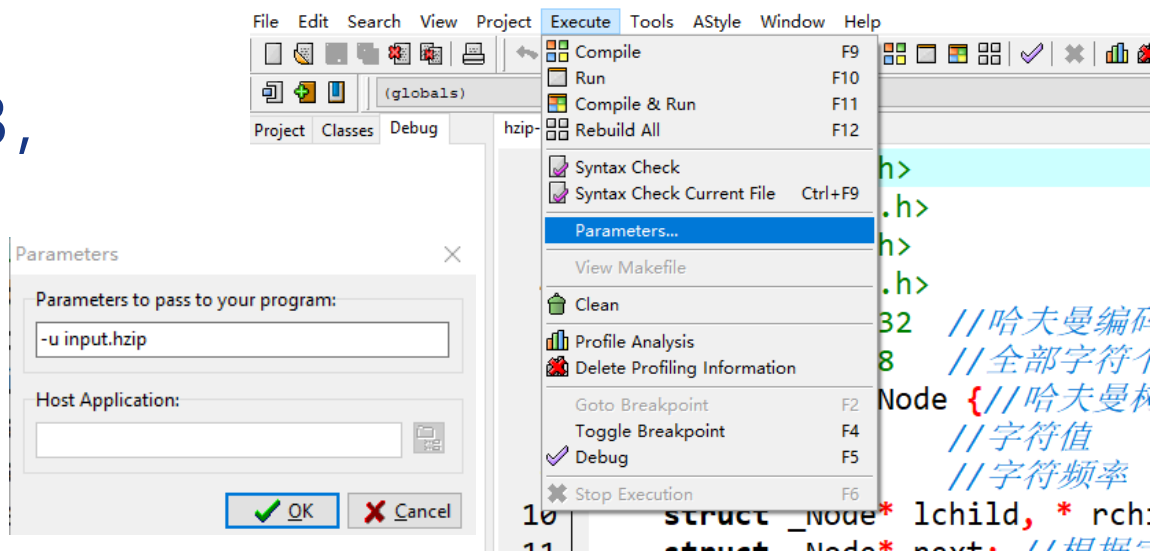
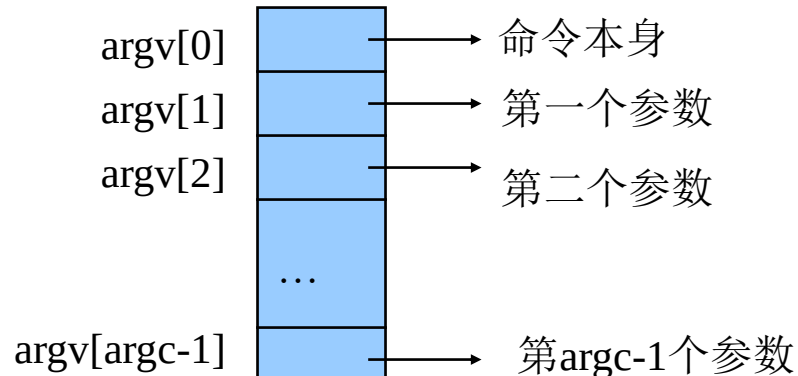
- ✓argc包含命令本身在内的参数个数

- ✓argv指针数组，数组元素为指向各参数（包含命令本身）的指针

◆假设运行程序program时在终端键盘上输入下列命令：

- ✓> **program** f1 6

- ✓在程序program中，argc的值等于3，
argv[0], argv[1]和argv[2]的内容
分别是"program", "f1"和"6"





本题目的命令行参数解析

```
int parseParams(int count, char* params[], char* srcFile, char *destFile) {
    int ans; // 命令行参数解析结果
    char* ext;
    if (count == 2) { // 压缩文件
        strcpy(srcFile, params[1]);
        strcpy(destFile, srcFile);
        ext = getFileExt(destFile);
        if (!isequal(ext, "txt")) ans = -1; // 文件扩展名错误
        else {
            ans = 1; // 参数正确，进行压缩文件的操作
            strcpy(ext, "hzip"); // 修改目标文件的扩展名
        }
    }
}
```

```
// 获得文件扩展名所在的位置
char* getFileExt(char* filename) {
    int len = strlen(filename);
    char* p = filename + len - 1;
    while (*p != '.' && p != filename) p--;
    return p + 1;
}
```



本题目的命令行参数解析 (续)

```
else if (count == 3) {
    strcpy(srcFile, params[2]);
    strcpy(destFile, srcFile);
    if (!isequal(params[1], "-u")) ans = 0; //参数格式错误
    else{
        ext = getFileExt(destFile);
        if (!isequal(ext, "hzip")) ans = -1; //文件扩展名错误
        else {
            ans = 2; //参数正确, 进行解压文件的操作
            strcpy(ext, "txt"); //修改目标文件的扩展名
        }
    }
}
else ans = 0; //参数个数错误
return ans;
}
```

//判断字符串是否相等, 不区分大小写

```
int isequal(const char* s1, const char* s2) {
    int ans = 1, i;
    int len1 = strlen(s1);
    int len2 = strlen(s2);
    if (len1 != len2) ans = 0;
    else{
        for (i = 0; i < len1; i++) {
            if (tolower(s1[i]) != tolower(s2[i])) {
                ans = 0;
                break;
            }
        }
    }
    return ans;
}
```



预备知识：位运算

◆在实际应用中有时需要操作数据对象中的某些位 (bit)

◆位运算符

- ✓按位或 (|) 通常用于给字中某些位赋值。
- ✓按位与 (&) 通常用于取出字中某些位的值。
- ✓按位异或 (^) 通常用于图形/图像运算中

&	0	1
0	0	0
1	0	1

	0	1
0	0	1
1	1	1

^	0	1
0	0	1
1	1	0

~	按位取反, 如~5
&	按位 “与”
	按位 “或”
^	按位 “异或”
>>	按位右移, 低位移出, 对int为算术右移 (高位补符号位), 对unsigned为逻辑右移 (高位补0)
<<	按位左移, 左移, 高位移出, 低位补0



补充知识：二进制文件操作

- ◆ 文本文件按照ASCII编码方式解析和显示文件，并通过回车区分文件的每一行
- ◆ 计算机中文件本质上都是二进制文件，大部分文件（如word、ppt、图像、声音）都是不可见的ASCII内容，按照变量二进制编码存储和访问
- ◆ 思考：
 - ✓ 用fprintf函数写入一个int到文件中？实际写入的是什么？
 - ✓ 是否可以能够把int类型的二进制编码直接写入文件？
 - ✓ 读取过程呢？



示例：文件的文本格式和二进制格式解析

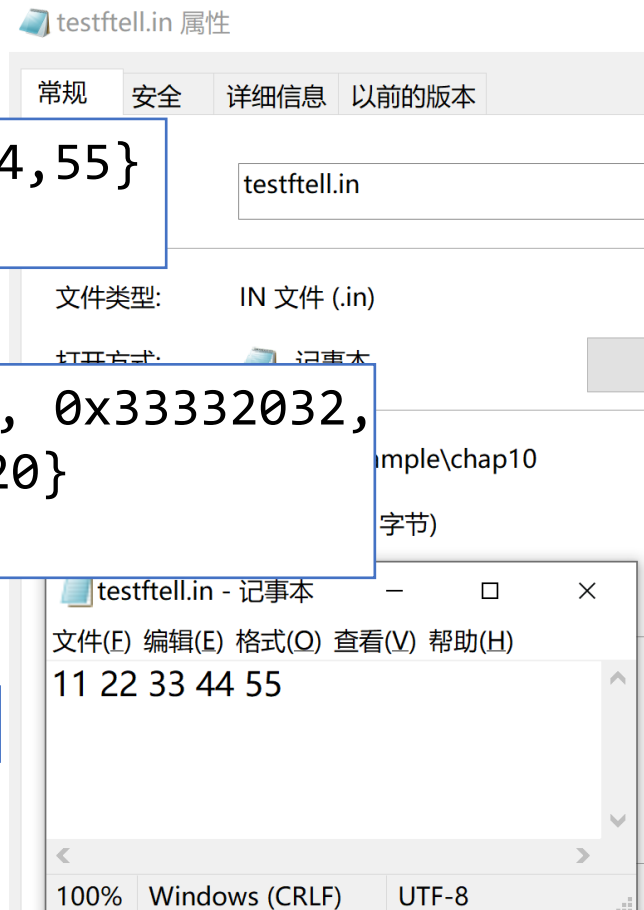
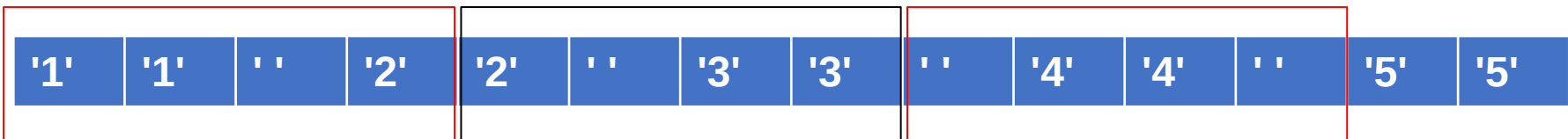
```
int len = 0, buf[BUFSIZ];  
FILE *fp;
```

```
fp = fopen("testftell.in", "r"); //文本打开  
//文本文件方式读写  
while(!fscanf(fp, "%d", &buf[i])) len++;
```

```
fp = fopen("testftell.in", "rb"); //二进制打开  
//二进制方式读写  
len = fread(buf, sizeof(int), BUFSIZ, fp);
```

```
buf {11, 22, 33, 44, 55}  
len 5
```

```
buf {0x32203131, 0x33332032,  
      0x20343420}  
len 3
```





二进制文件读写函数

单字节读写操作: fputc与fgetc

```
int fputc(int c, FILE *fp)
```

```
int fgetc(FILE *fp)
```

- 作用：将一个字节代码**c** 写入到**fp**所指向的文件（从文件中读取一个字节码）
- 返回值：输出成功则返回所写入/读取的字节码**c**，失败则返回**EOF**

多字节读写操作: fwrite与fread

```
size_t fwrite(const void *buf, size_t size, size_t count, FILE *fp);
```

```
size_t fread (void *buf, size_t size, size_t count, FILE *fp);
```

- 作用：将buf缓冲区中的以size大小的count个数据项写入（或读取）到文件（buf）
- buf: 数据存储区（从file读入，或写进file）
- size: 以字节为单位，每个数据项的长度
- count: 读写数据项的个数
- fp: 要读写的文件指针
- 返回值：输出成功则返回所写入/读取的字节码**c**，失败则返回**EOF**



读写定位函数：多次读写、指定位置读写文件

```
int fseek(FILE *stream, long offset, int whence);
```

- ◆ **作用**：将stream所指向文件的位置指针移到以**whence**所指的位置为起始点、以**offset**为位移量的位置，同时清除文件结束标志
- ◆ **返回值**：定位成功则返回非0，否则返回 0
如 `fseek(stream, 0, SEEK_SET);`
- ◆ 位移量**offset**表示以起始点为基准，向前或向后移动的字节数（为正表示向文件尾部的方向的移动，为负则表示向文件头部的方向移动）
- ◆ 起始点**whence**可以是：**SEEK_SET, SEEK_CUR, SEEK_END**三个符号常量，其值分别为 **0, 1, 2**，分别表示文件开始、文件当前位置、文件末尾



哈夫曼树主要数据结构

```
#define MAXSIZE 32 //哈夫曼编码最大长度
#define COUNT 128 //全部字符个数
typedef struct _Node { //哈夫曼树节点
    char ch; //字符值
    int count; //字符频率
    struct _Node* lchild, * rchild; //左孩子、右孩子指针
    struct _Node* next; //根据字符出现的频率排列的有序链表下一个结点地址
}Node, *PNode;

PNode root; //哈夫曼树的根节点
PNode head; //为了便于建立哈夫曼树，增加一个带头结点的有序链表，head指向头结点
int hCodeLength = 0; //码表的长度，初始为0
//字符对应的哈夫曼表，Hcode[0]为文件结束符的编码，
//其他按照ascii下标存储，如hcode['a']为字符a的哈夫曼编码
//以"110"这样的字符串格式存储哈夫曼编码
char hCode[COUNT][MAXSIZE];
char huffman[MAXSIZE]; //临时生成单个字符的哈夫曼编码变量
```



几个工具函数

//创建一个哈夫曼树节点

```
PNode createNode(char ch, int count) {  
    PNode p;  
    p = (PNode)malloc(sizeof(Node));  
    p->ch = ch;  
    p->count = count;  
    p->lchild = p->rchild = p->next = NULL;  
    return p;  
}
```

//统计词频

```
void statCount(FILE* src, int ccount[]){  
    char ch;  
    int i;  
    ccount[0] = 1; //设置文件结束符标记为  
    //初始化词频为0  
    for (i = 1; i < COUNT; i++) ccount[i] = 0;  
    while ((ch = fgetc(src)) != EOF) { //读单字符  
        ccount[ch]++; //对应词频+1  
    }  
    return;  
}
```

//将结点插入到有序链表中

```
void insertSortLink(PNode p) {  
    PNode q, r;  
    r = head; //r指向头结点  
    q = r->next; //q指向头结点后面的正式第一个数据结点  
    while (q != NULL && q->count <= p->count) { //q指针指向要插入的位置  
        r = q;  
        q = q->next;  
    }  
    r->next = p;  
    p->next = q;  
}
```



主函数：解析命令行参数

```
int main(int argc, char* argv[]) {  
    FILE* src; //源文件  
    FILE* obj; //目标文件  
    char srcFileName[50];    char destFileName[50];  
    int op;    int ccount[COUNT];  
    root = NULL;  
    op = parseParams(argc, argv, srcFileName, destFileName);  
    if (op == 0) {  
        printf("Usage: hzip.exe [-u] <filename>\n");  
    }  
    else if (op == -1) {  
        printf("File extension error!\n");  
    }  
}
```



主函数：处理解压操作

```
else if (op == 2) { //解压文件
    src = fopen(srcFileName, "rb"); //打开文件
    if (src == NULL) { //打开失败，报错
        perror("Can't open the file!");
        exit(-1);
    }
    obj = fopen(destFileName, "w"); //打开文件
    if (obj == NULL) { //打开失败，报错
        perror("Can't open the file!");
        exit(-1);
    }
    createHCodeFromFile(src); //读出文件头，得到哈夫曼编码
    createHuffmanTreeFromCode(); //根据哈夫曼编码构造哈夫曼树
    makeUHZip(src, obj); //生成解压文件
    fclose(src);
    fclose(obj);
}
}
```



提纲：树和二叉树

◆树的基本概念、性质和存储结构

◆二叉树

- ✓二叉树的基本概念和性质
- ✓二叉树的存储结构

◆二叉树的遍历

- ✓前序、中序和后续遍历算法
- ✓递归算法的非递归设计
- ✓层次遍历算法
- ✓遍历算法的应用
- ✓（多叉）树的遍历



提纲：树和二叉树

◆*线索二叉树

◆二叉查找树

- ✓二叉查找树的建立、删除
- ✓二叉查找树的应用
- ✓*平衡二叉树

◆*堆和表达式树

◆哈夫曼树

- ✓哈夫曼树的构造原理
- ✓哈夫曼编码